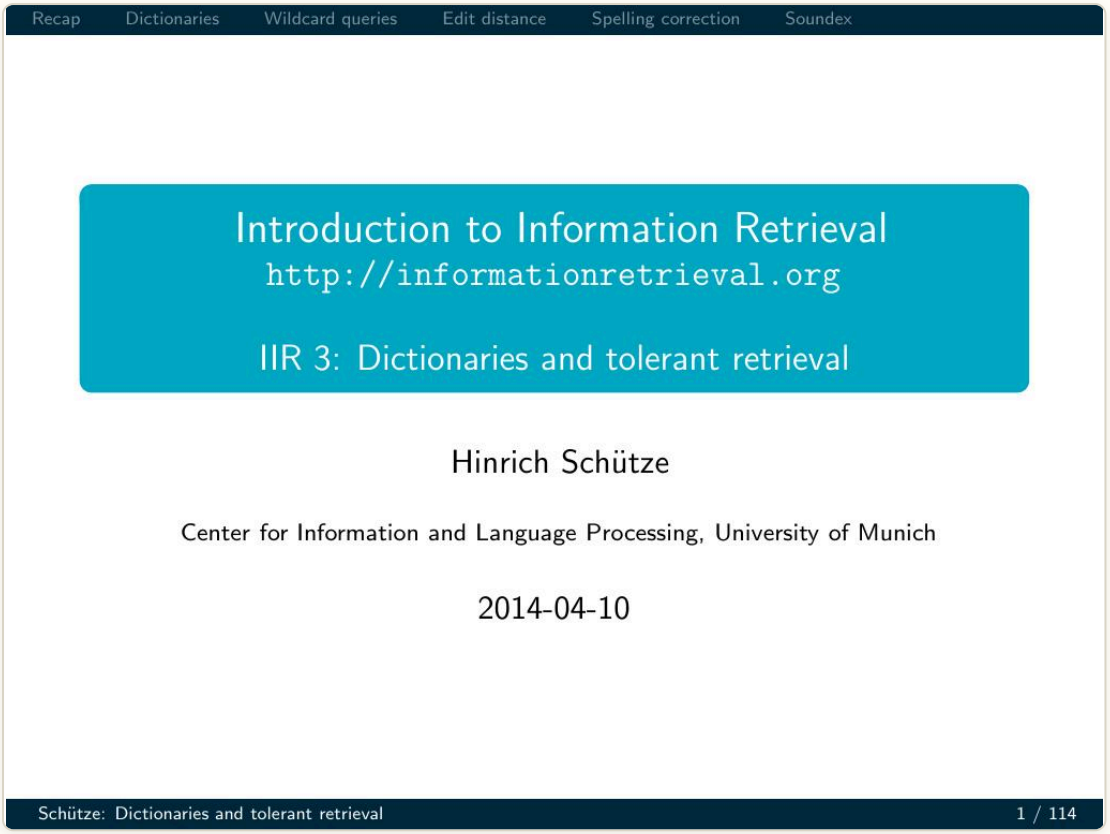
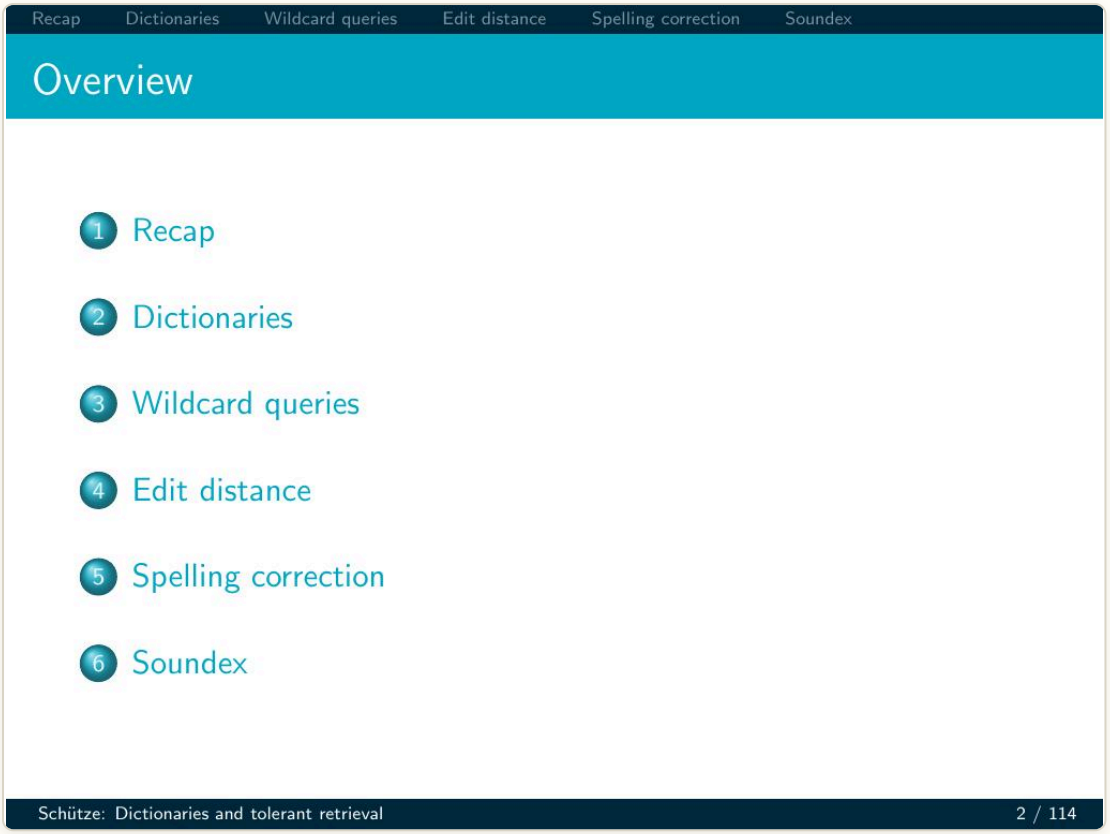


정보검색 입문 — IIR 3: 사전과 오류 허용 검색

- Hinrich Schütze, 뮌헨대 (2014-04-10)

✍️ 메모

💡 이해 3강 핵심: 사전 자료구조(해시 vs 트리) → 와일드카드(permuterm, k-gram) → 편집 거리(DP) → 철자 교정 → Soundex

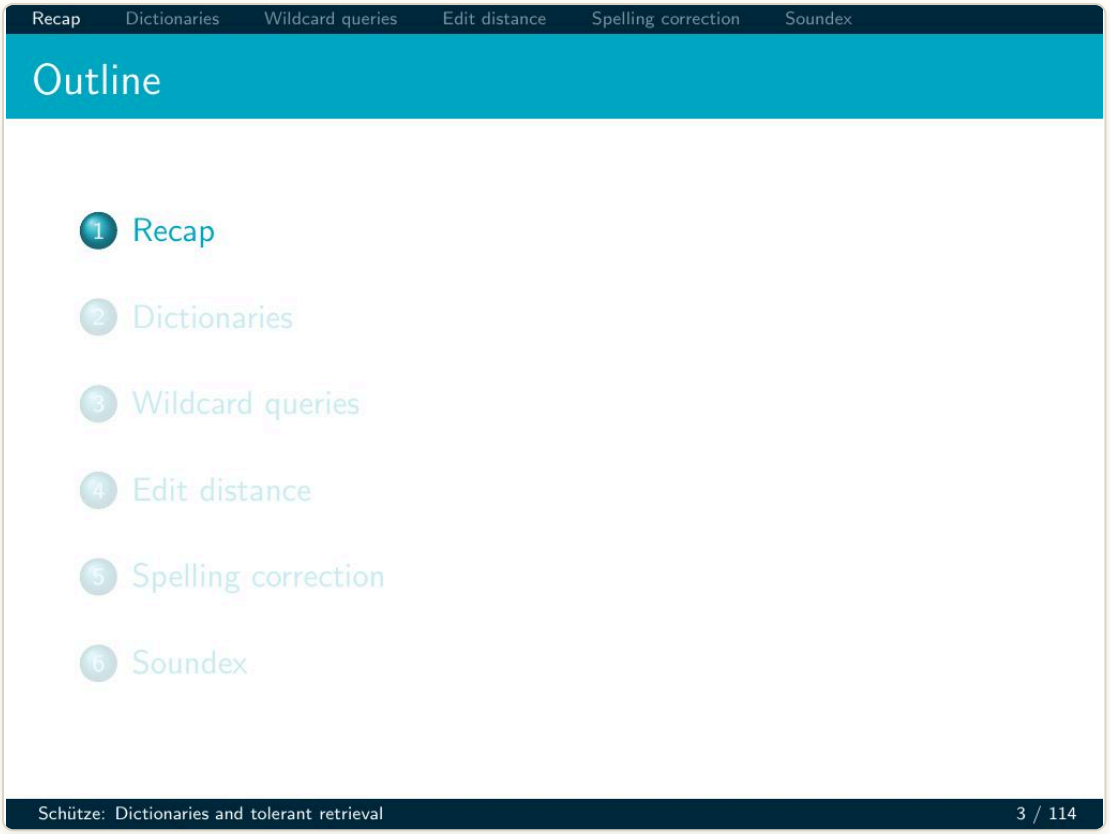


개요

- ① 복습 ② 사전 ③ 와일드카드 질의 ④ 편집 거리 ⑤ 철자 교정 ⑥ Soundex

✍ 메모

— 넘어가도 되는 쪽

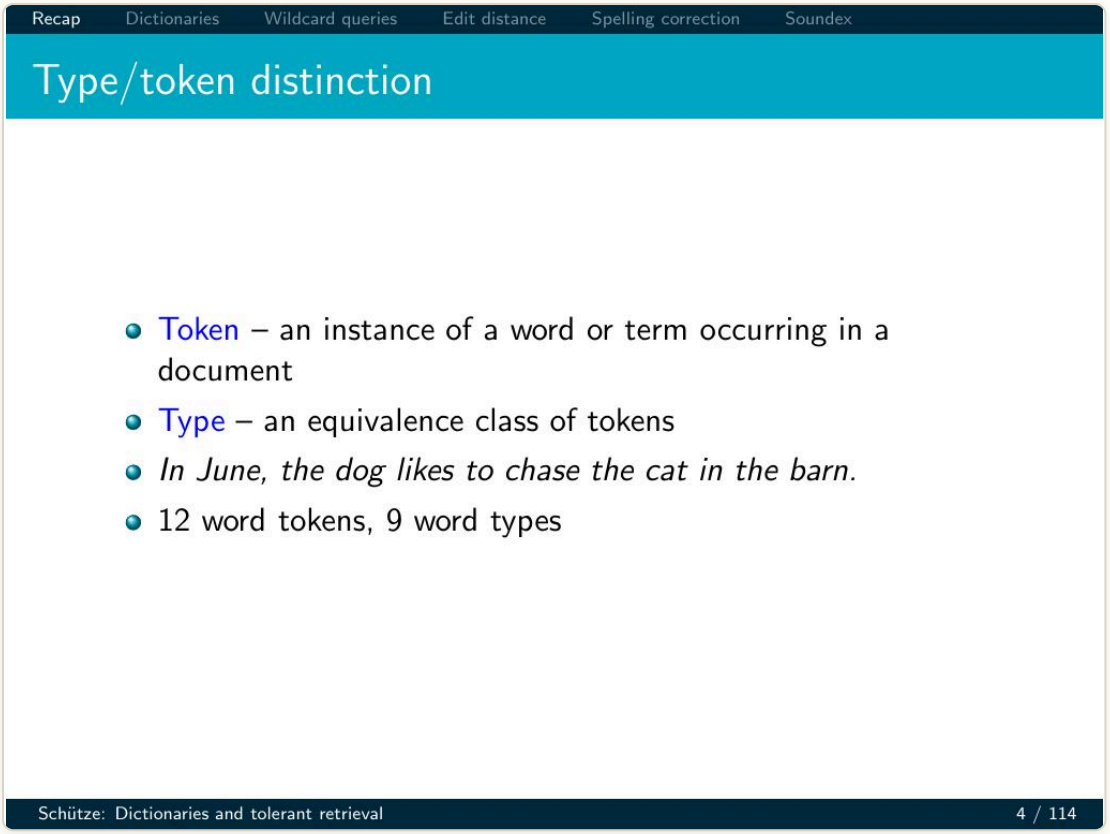


목차

- 복습 → 사전 → 와일드카드 질의 → 편집 거리 → 철자 교정 → Soundex
- 지금: 복습

✍ 메모

— 넘어가도 되는 쪽

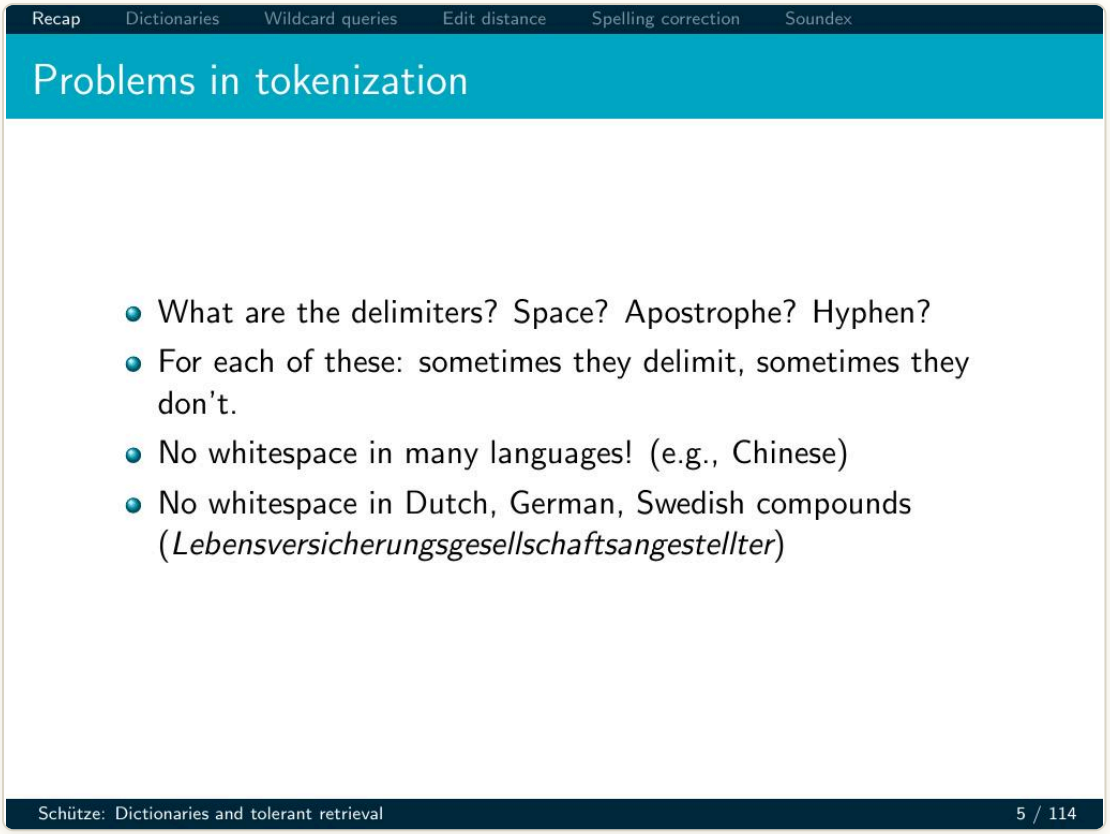


복습: 유형/토큰 구분

- **토큰** — 문서에 나타난 단어·용어의 한 번의 등장
- **유형** — 토큰들의 동치류
- In June, the dog likes to chase the cat in the barn. → **단어 토큰 12개, 단어 유형 9개**

✍️ 메모

★ 시험 2강 연습의 정답이 여기 있다: 12 / 9

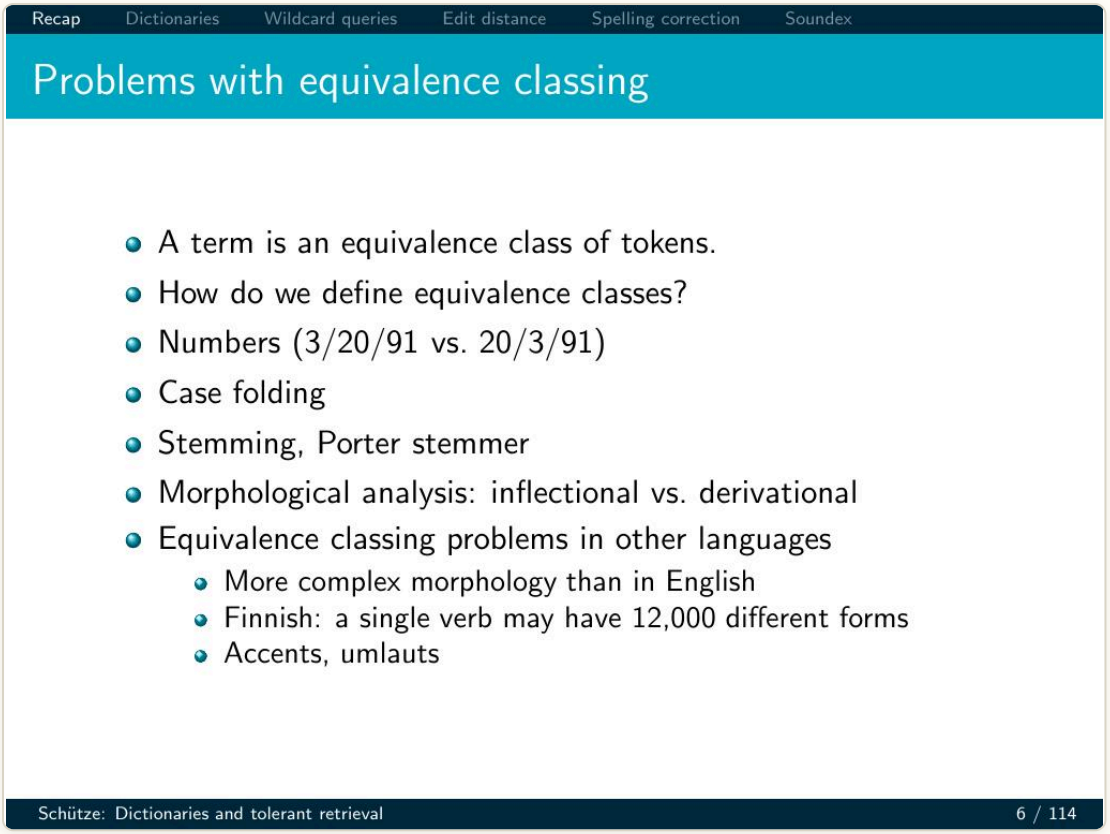


복습: 토큰화의 문제

- 구분자는? 공백? 아포스트로피? 하이픈?
  - 각각 어떨 땐 구분하고 어떨 땐 안 한다
- 많은 언어엔 공백이 없다 (예: 중국어)
- 네덜란드어·독일어·스웨덴어 합성어도 공백 없음
  (Lebensversicherungsgesellschaftsangestellter)

✍ 메모

— 넘어가도 되는 쪽

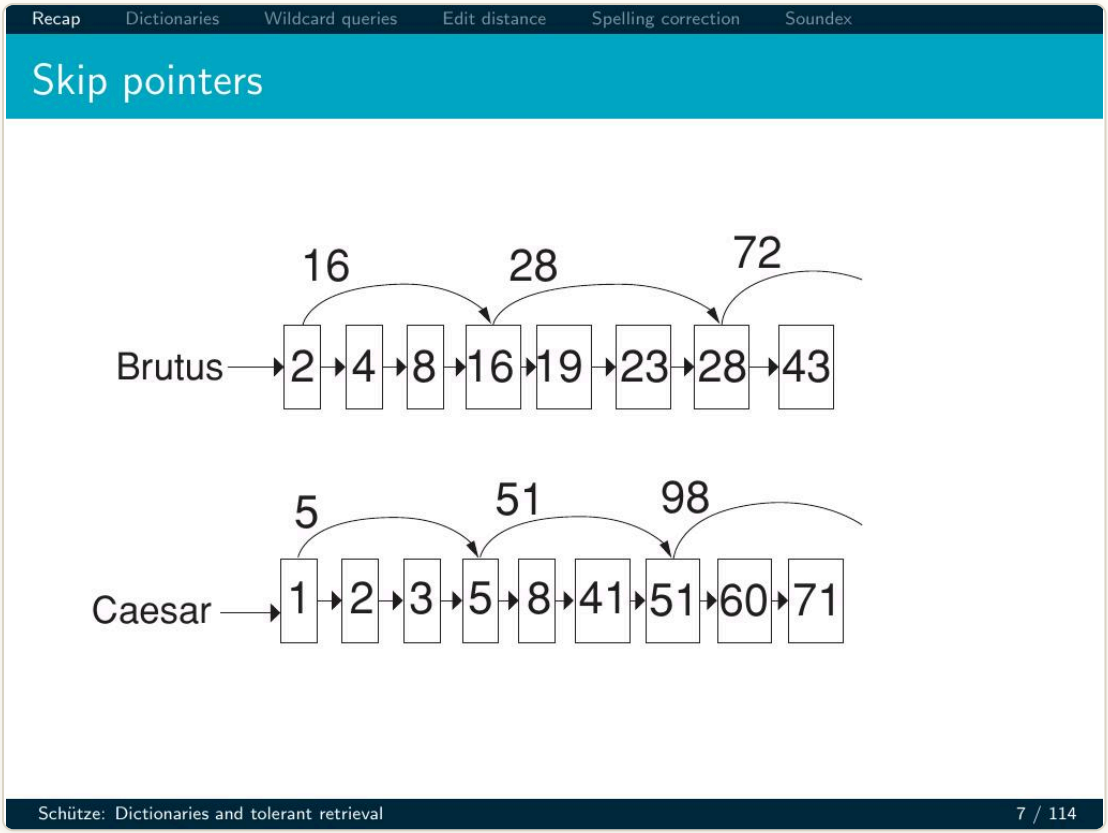


복습: 동치류의 문제

- term = 토큰들의 동치류. 동치류를 어떻게 정의하나?
  - 숫자 (3/20/91 vs. 20/3/91) / 대소문자 통일 / 어간 추출(포터) / 형태 분석: 굴절 vs 파생
- 다른 언어의 동치류 문제
  - 영어보다 복잡한 형태론 — 핀란드어는 동사 하나에 **12,000가지** 형태
  - 악센트, 움라우트

✍️ 메모

— 넘어가도 되는 쪽

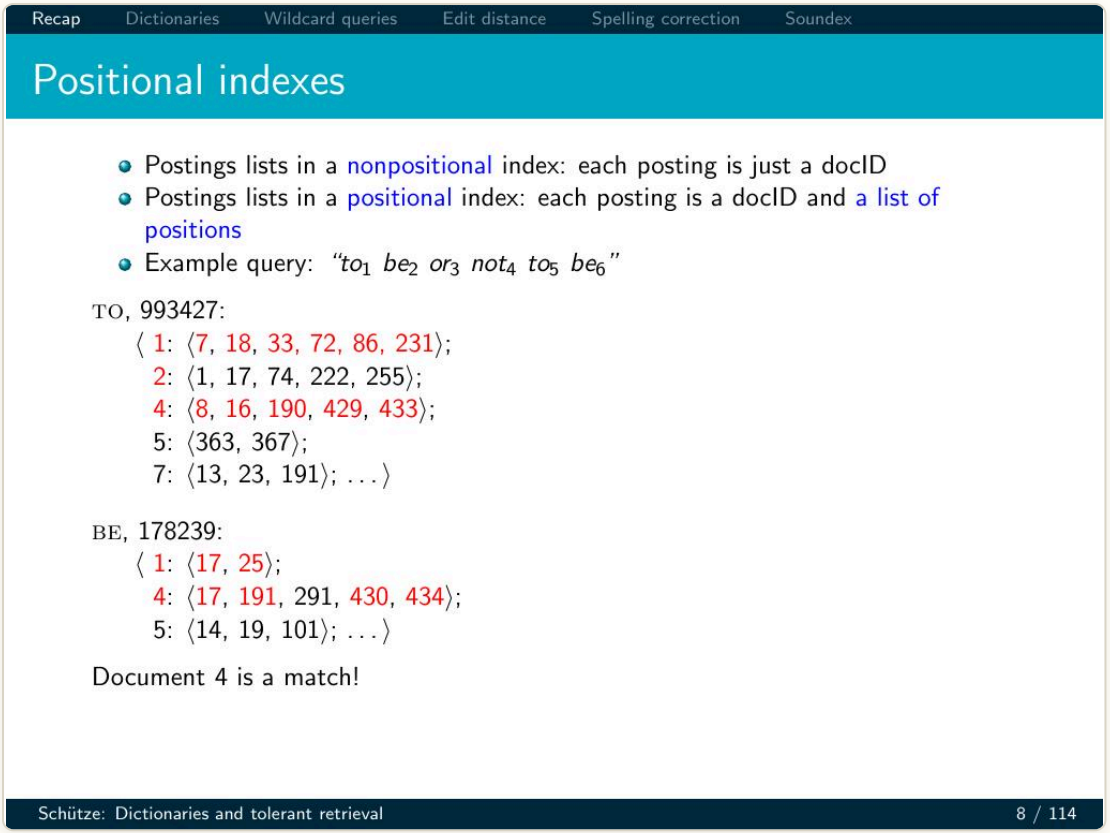


복습: 스킵 포인터

- 그림: Brutus·Caesar 포스팅에 스킵 포인터 — 건너뛸 수 있는 구간을 점프

✍ 메모

— 넘어가도 되는 쪽

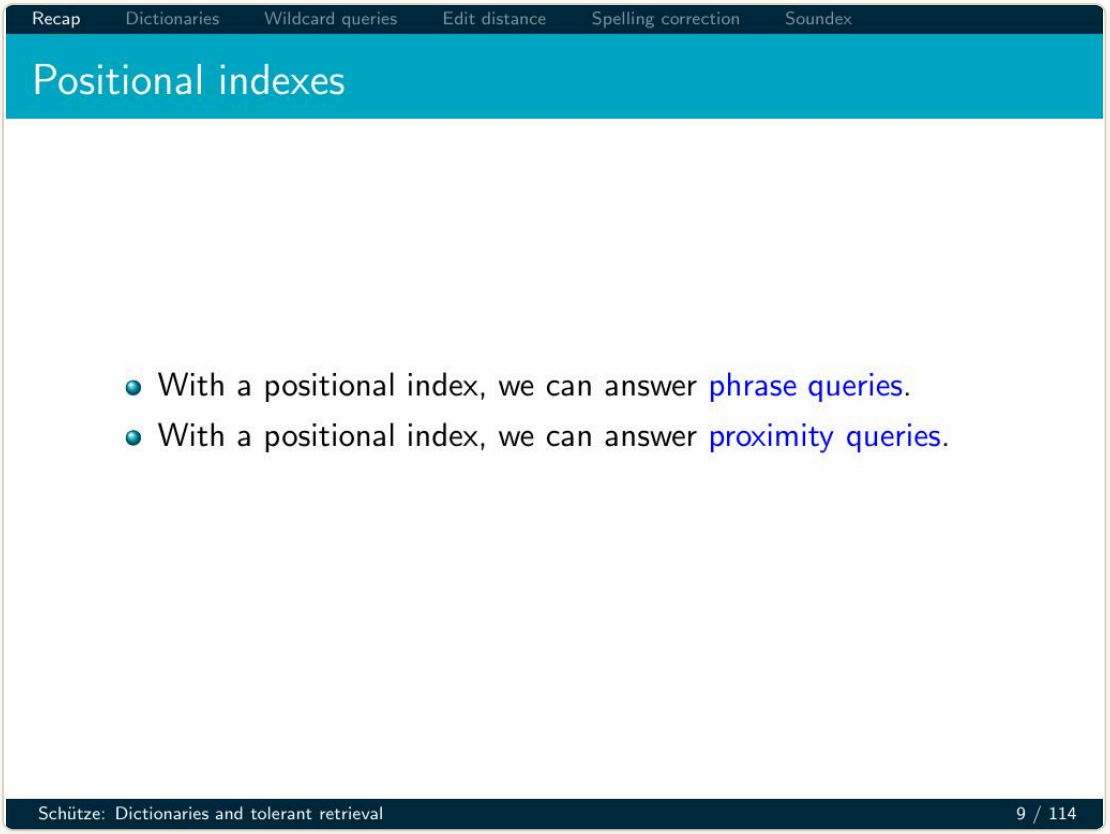


복습: 위치 색인

- 비위치 색인: 포스팅 = docID / 위치 색인: 포스팅 = docID + 위치 목록
- "to be or not to be" — to 4: ⟨...429, 433⟩, be 4: ⟨...430, 434⟩ → **문서 4가 맞음**

✍ 메모

— 넘어가도 되는 쪽

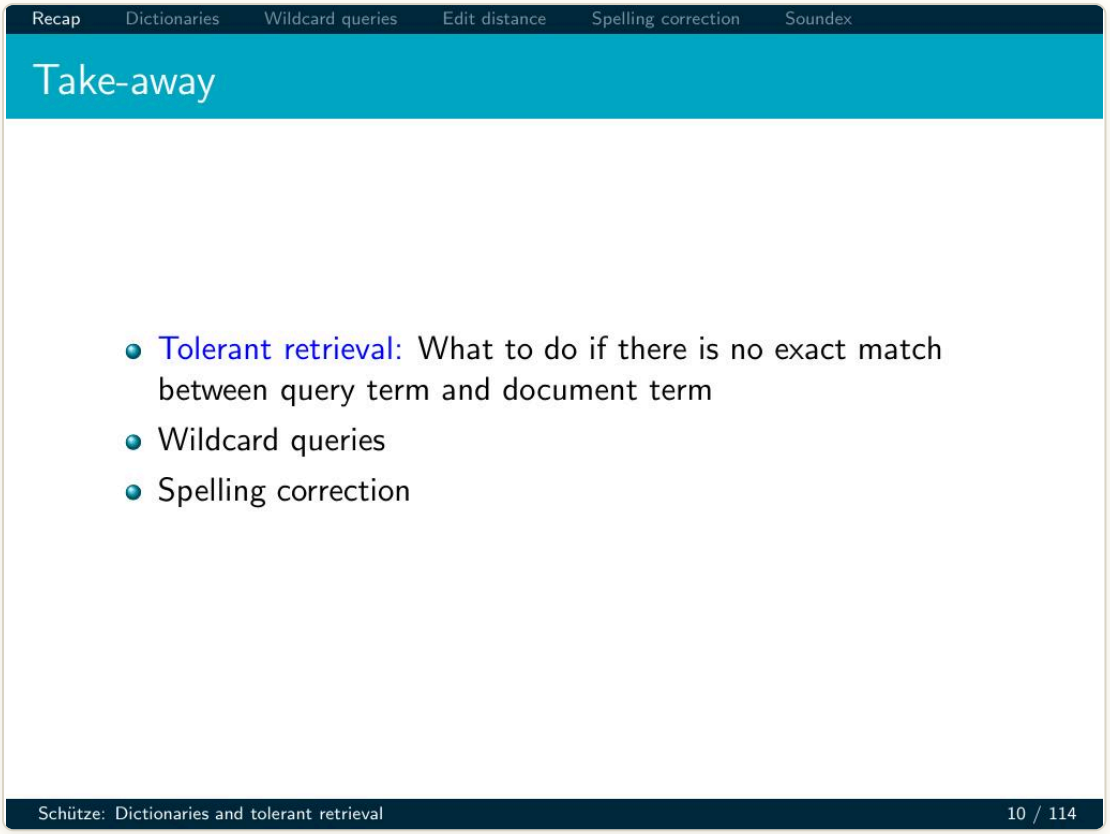


복습: 위치 색인으로 할 수 있는 것

- **구 질의**에 답할 수 있다
- **근접 질의**에 답할 수 있다

✍ 메모

— 넘어가도 되는 쪽

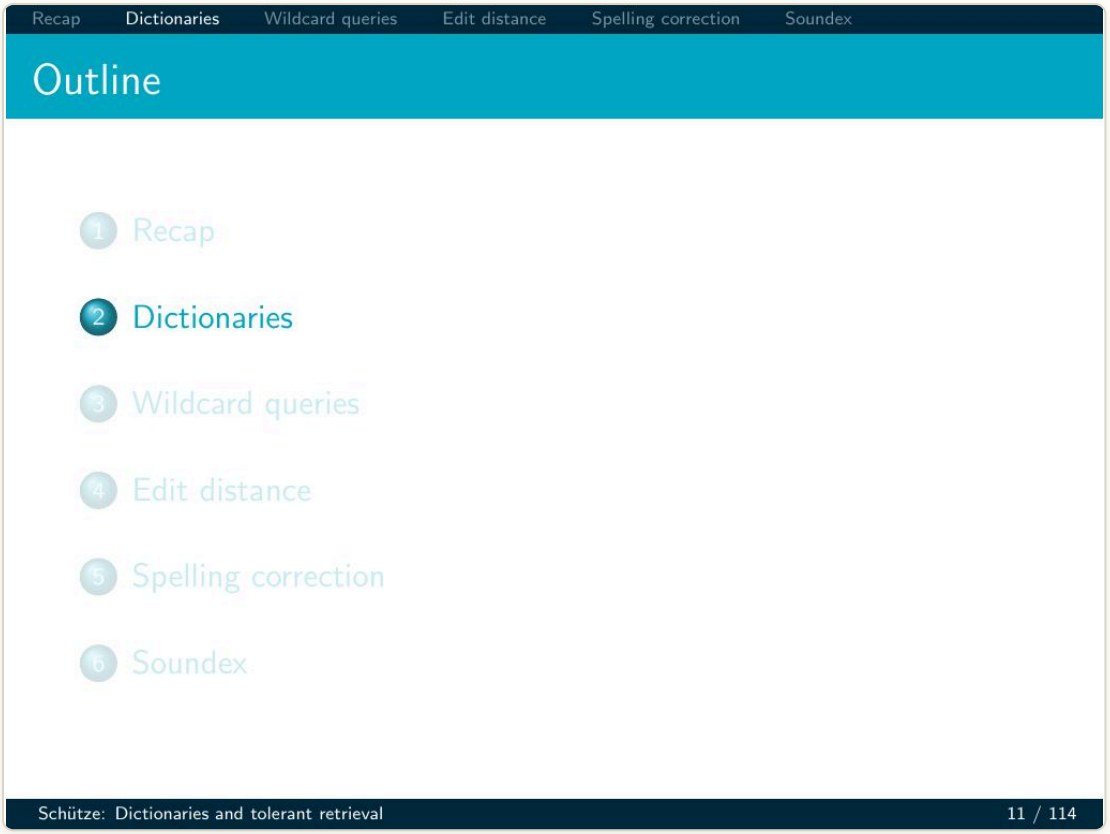


이번 강의에서 가져갈 것

- **오류 허용 검색 (Tolerant retrieval)**: 질의어와 문서 단어가 **정확히 일치하지 않을 때** 어떻게 하나
  - **와일드카드 질의**
  - **철자 교정**

✍️ 메모

— 넘어가도 되는 쪽

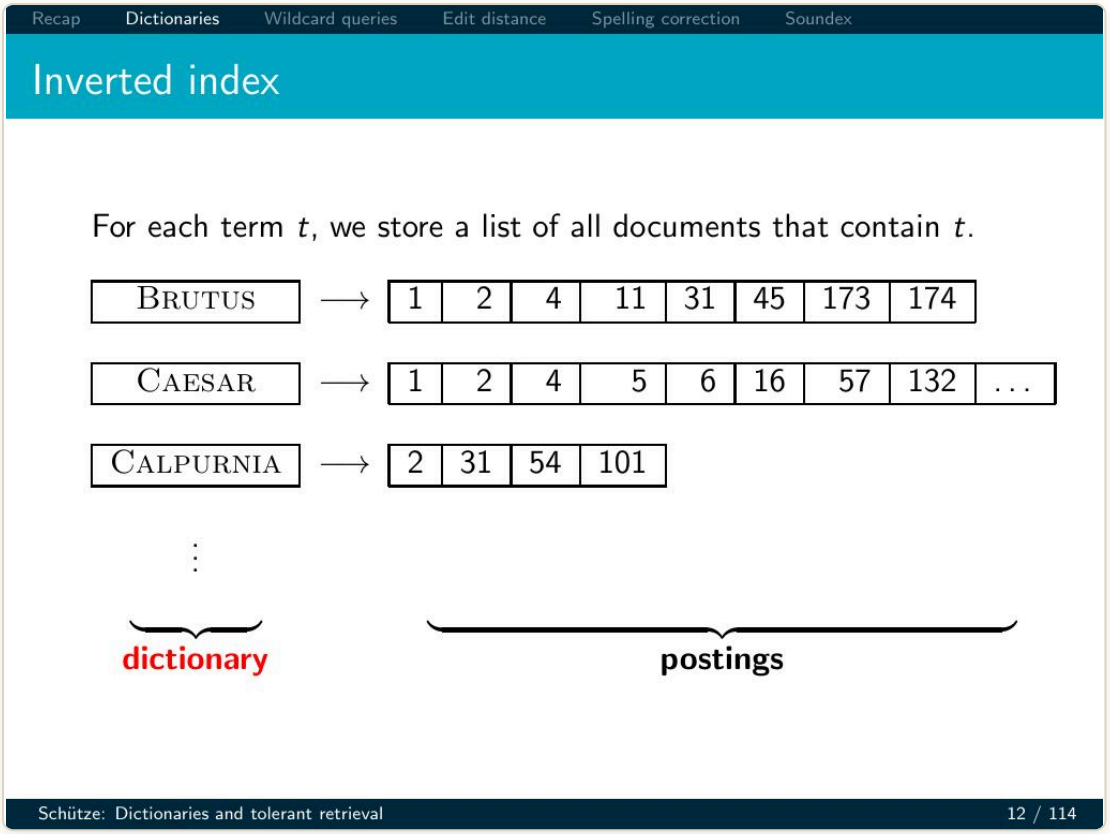


목차

- 복습 → 사전 → 와일드카드 질의 → 편집 거리 → 철자 교정 → Soundex
- 지금: **사전**

✍ 메모

— 넘어가도 되는 쪽

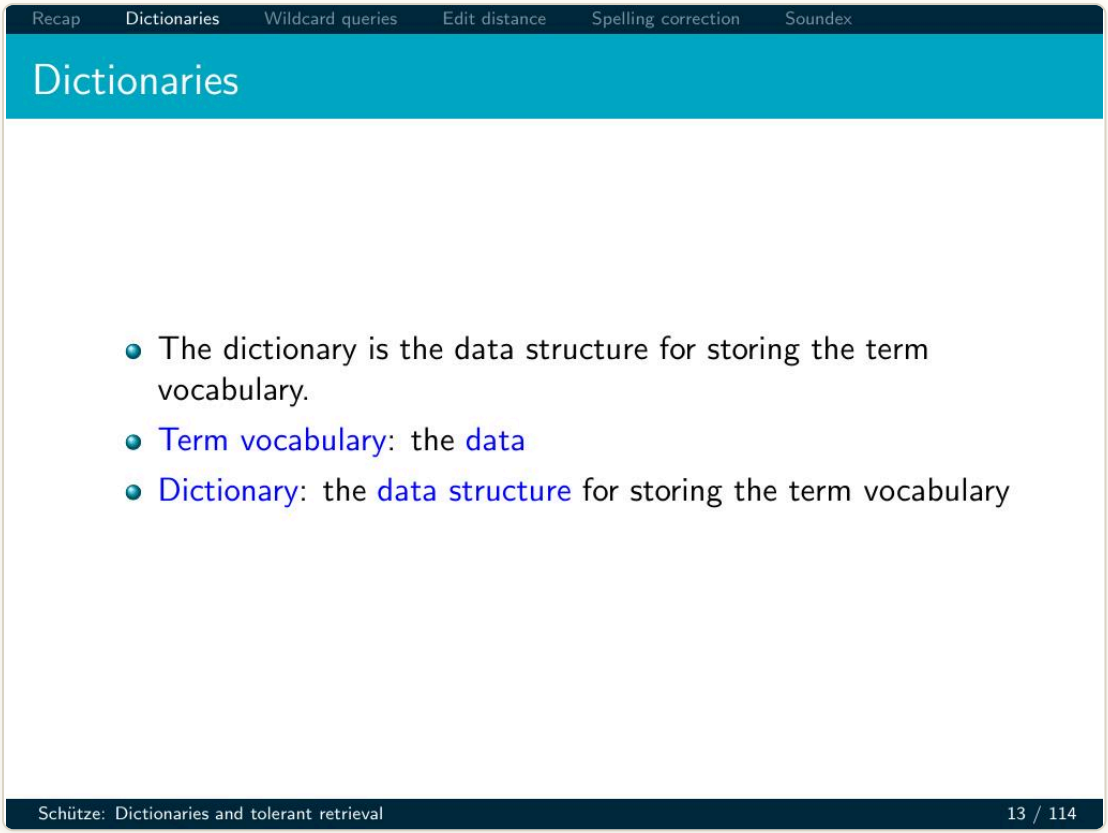


역색인

- Brutus → 1 2 4 11 31 45 173 174 / Caesar → 1 2 4 5 6 16 57 132 … / Calpurnia → 2 31 54 101
- 왼쪽 사전이 이번 절의 주제

✍ 메모

— 넘어가도 되는 쪽

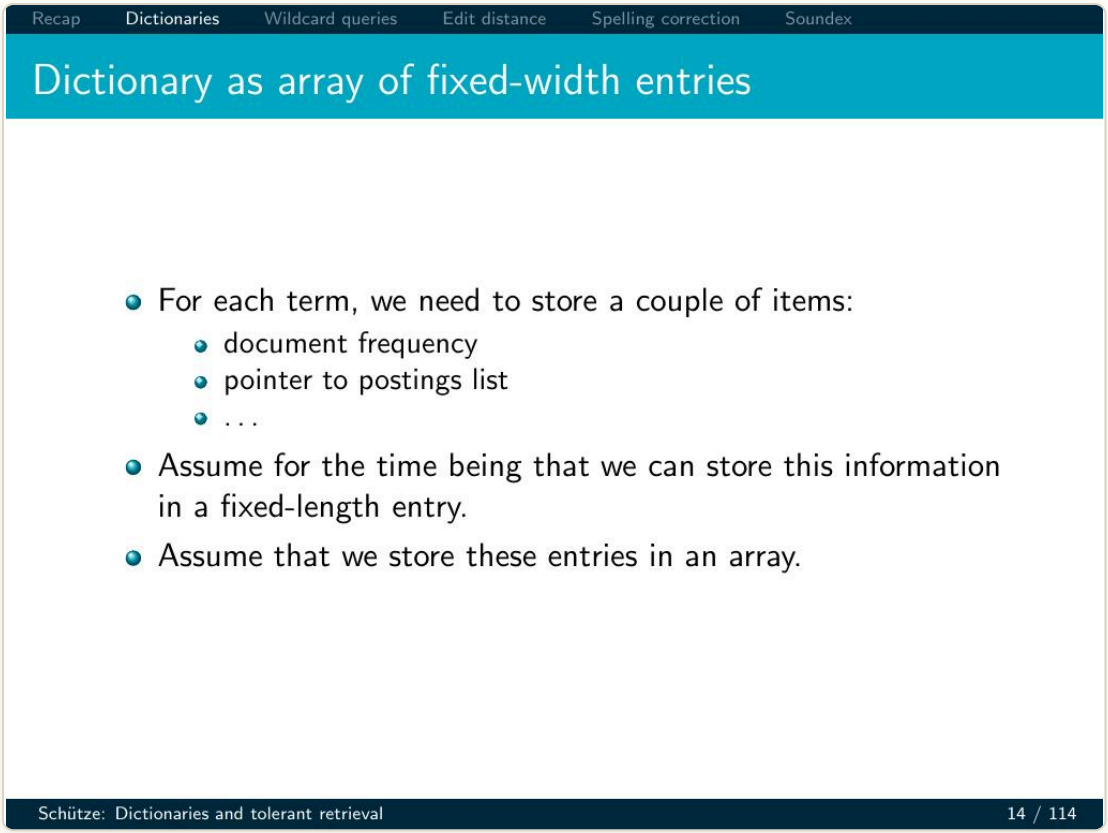


사전 (Dictionaries)

- **사전** = 단어 어휘를 저장하는 **자료구조**
- **단어 어휘(term vocabulary)**: 데이터 그 자체
- **사전(dictionary)**: 그 어휘를 저장하는 자료구조

✍ 메모

★ 시험 vocabulary = 데이터, dictionary = 자료구조



고정 폭 항목 배열로서의 사전

- 단어마다 저장할 것: **문서 빈도(df)**, **포스팅 리스트 포인터**, ...
- 일단 이 정보를 **고정 길이 항목**에 담을 수 있다고 하자
- 이 항목들을 **배열**에 저장한다고 하자
- 표: term(20바이트) · df(4바이트) · 포인터(4바이트) — 예: a 656,265 / aachen 65 / ... / zulu 221

✍ 메모

— 넘어가도 되는 쪽

Dictionary as array of fixed-width entries

	term	document frequency	pointer to postings list
	a	656,265	⟶
	aachen	65	⟶
	. . .	. . .	. . .
	zulu	221	⟶
space needed:	20 bytes	4 bytes	4 bytes

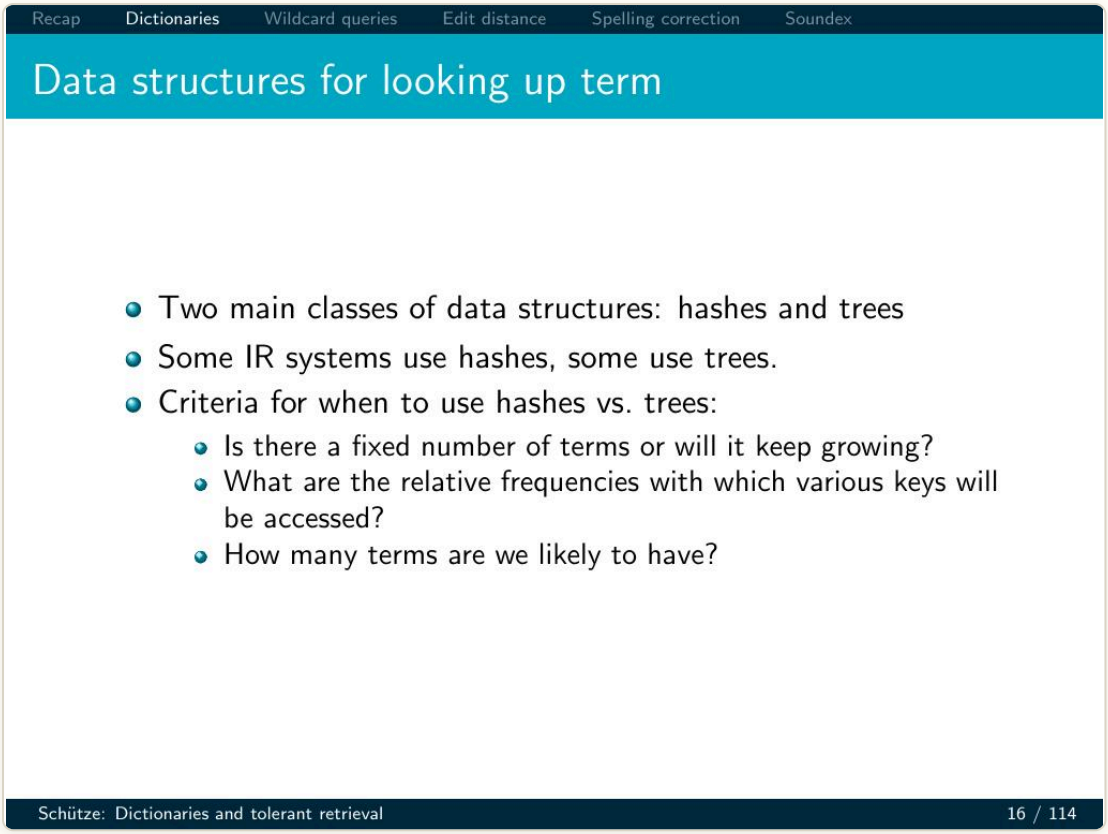
How do we look up a query term q_i in this array at query time?
That is: which data structure do we use to locate the entry (row)
in the array where q_i is stored?

고정 폭 항목 배열 — 찾기

- 질의할 때 배열에서 질의어 qi를 어떻게 찾나?
- 즉 qi가 저장된 행을 찾는 데 **어떤 자료구조**를 쓰나?

✍ 메모

— 넘어가도 되는 쪽

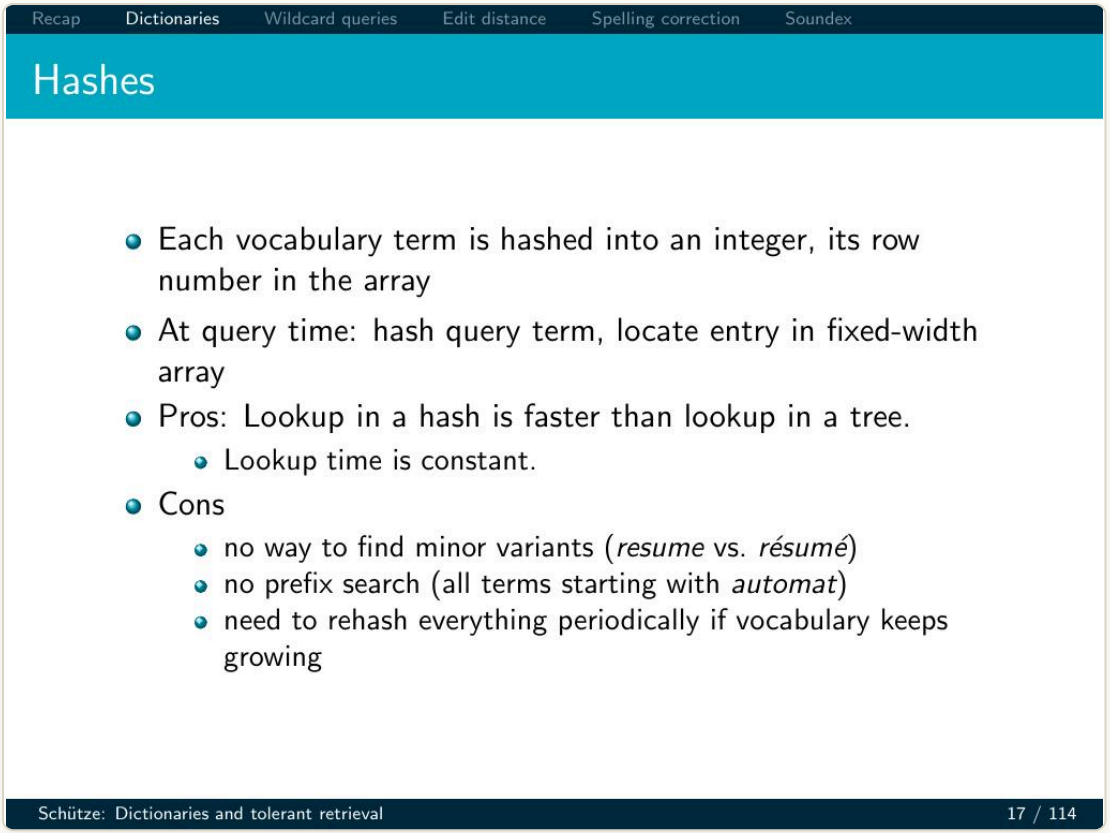


단어 찾기용 자료구조

- 두 가지 큰 부류: **해시**와 **트리**
- 어떤 IR 시스템은 해시, 어떤 건 트리
- 무엇을 쓸지 판단 기준
  - 단어 수가 **고정**인가, 계속 **늘어나는가**?
  - 키마다 접근 **빈도**가 어떻게 다른가?
  - 단어가 **몇 개**나 될까?

✍ 메모

★ 시험 판단 기준 3개: 어휘 증가 여부 · 접근 빈도 · 어휘 크기

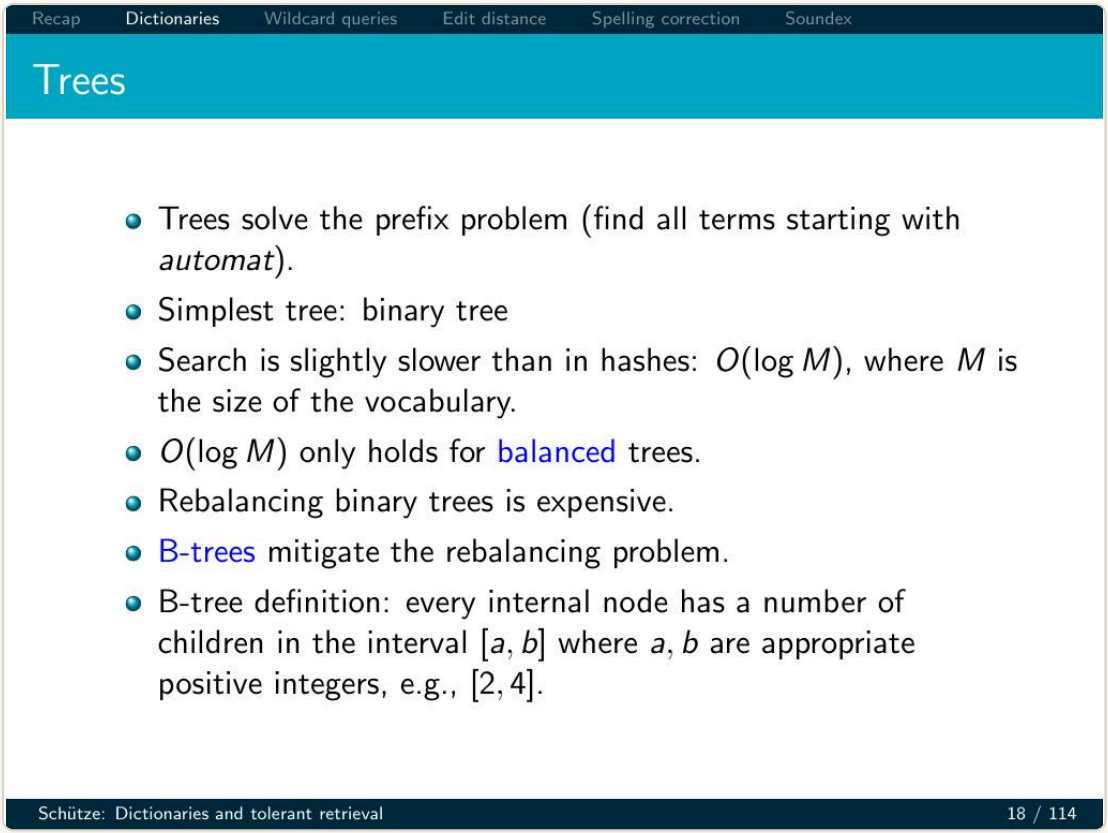


해시 (Hashes)

- 어휘의 각 단어를 정수(배열의 행 번호)로 **해싱**
- 질의 때: 질의어를 해싱해 고정 폭 배열에서 항목 찾기
- **장점**: 해시 조회가 트리보다 **빠르다** — 조회 시간 **상수**
- **단점**
  - 작은 **변형**을 찾을 수 없다 (resume vs. résumé)
  - **접두어 검색 불가** (automat로 시작하는 모든 단어)
  - 어휘가 계속 늘면 주기적으로 **전부 다시 해싱**해야

✍ 메모

★ 시험 해시: O(1) 빠름 / 단점 3개 — 변형·접두어·재해싱

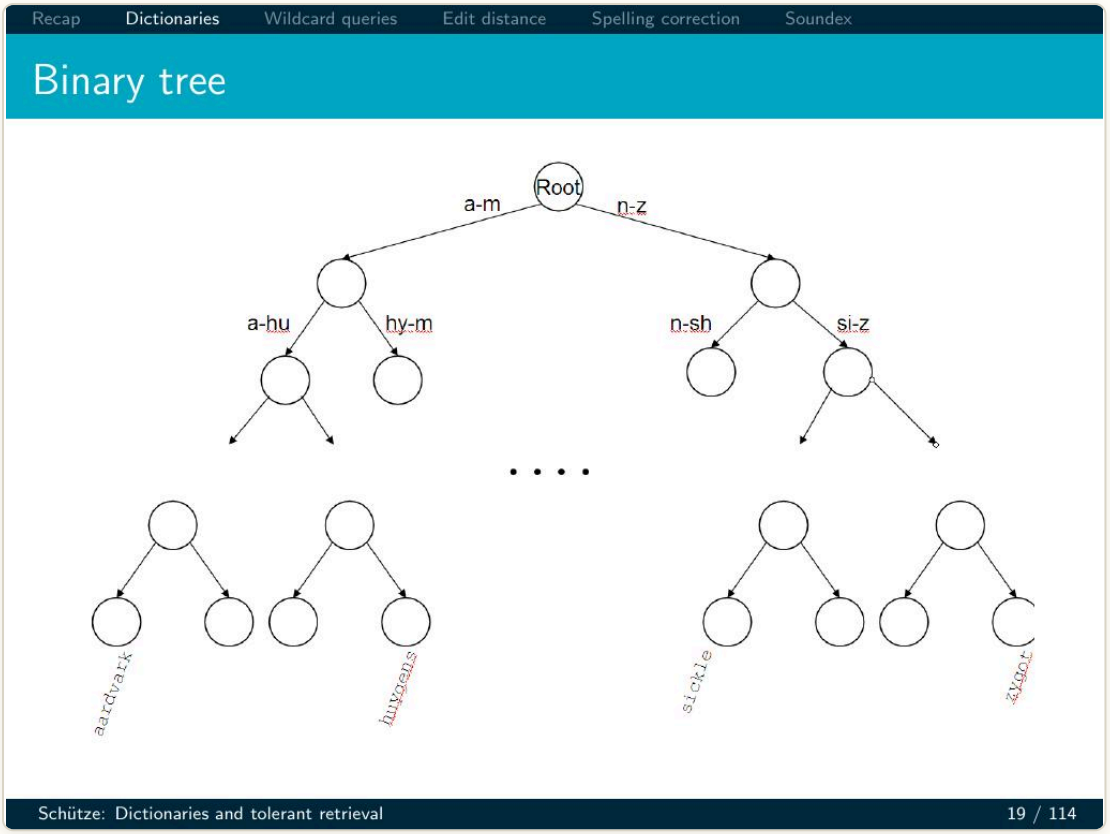


트리 (Trees)

- 트리는 **접두어 문제를 해결** (automat로 시작하는 단어 찾기)
- 가장 단순한 트리: **이진 트리**
- 검색은 해시보다 조금 느림: **O(log M)**, M = 어휘 크기
  - O(log M)은 **균형 트리**일 때만
  - 이진 트리 **재균형은 비싸다**
- **B-트리**가 재균형 문제를 줄인다
- B-트리 정의: 모든 내부 노드의 자식 수가 구간 **[a, b]** 안 (a, b는 적절한 양의 정수, 예: [2, 4])

✍ 메모

★ 시험 트리: O(log M), 접두어 검색 가능, B-트리로 재균형 완화

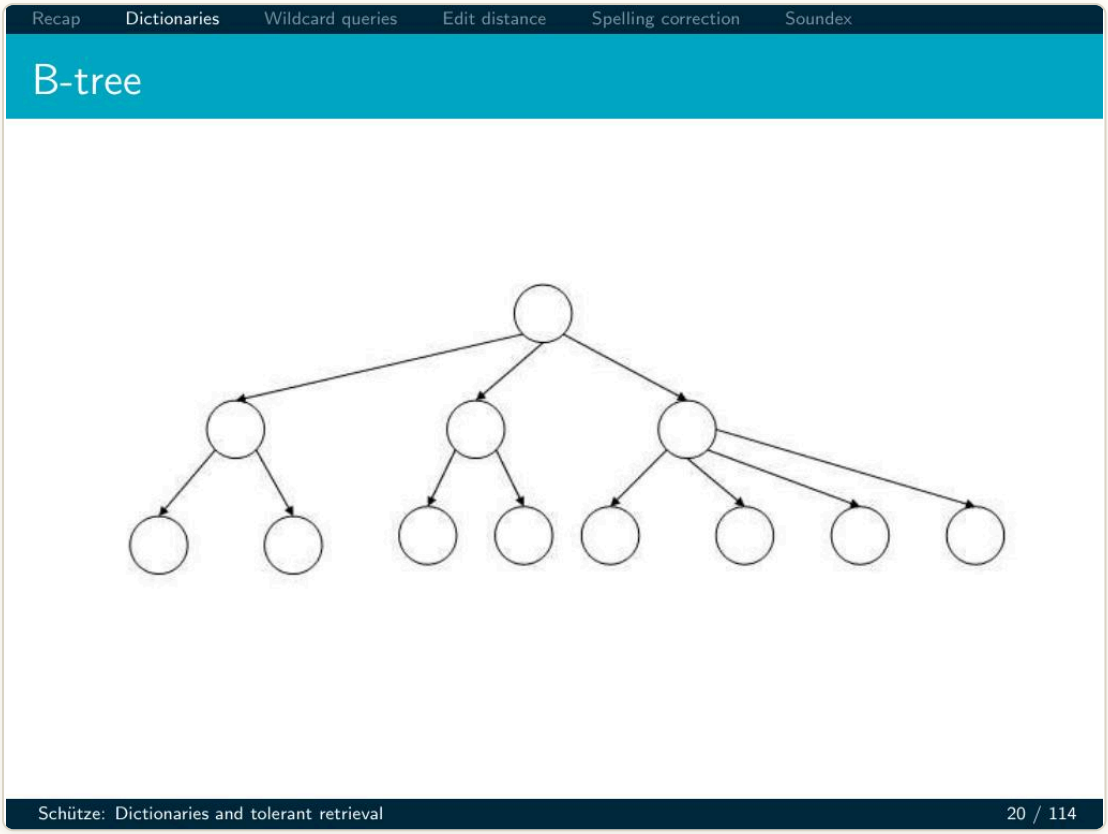


이진 트리 (그림)

- Root → a-m / n-z → a-hu, hy-m, n-sh, si-z → … → 잎에 aardvark, huygens, sickle, zygot 같은 단어

✍ 메모

— 넘어가도 되는 쪽

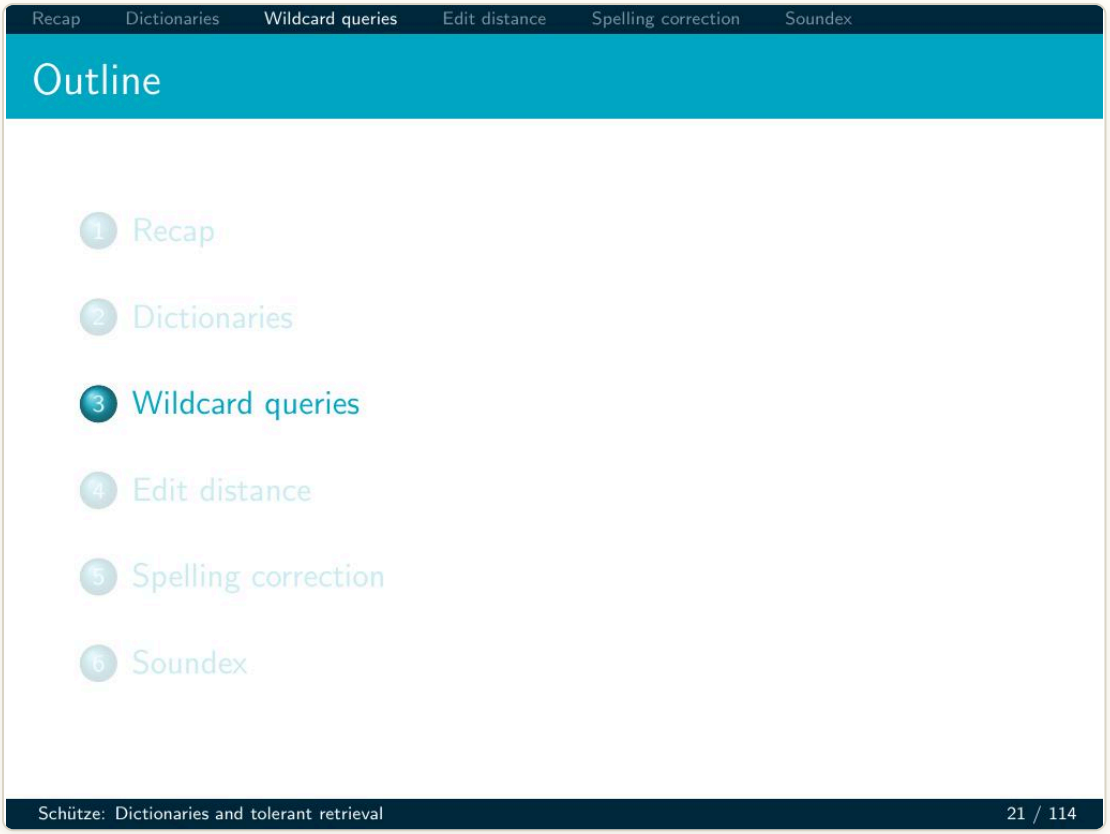


B-트리 (그림)

- 노드마다 자식이 2~4개 — 한 노드가 여러 갈래로 나뉘어 트리가 얕다

✍ 메모

— 넘어가도 되는 쪽

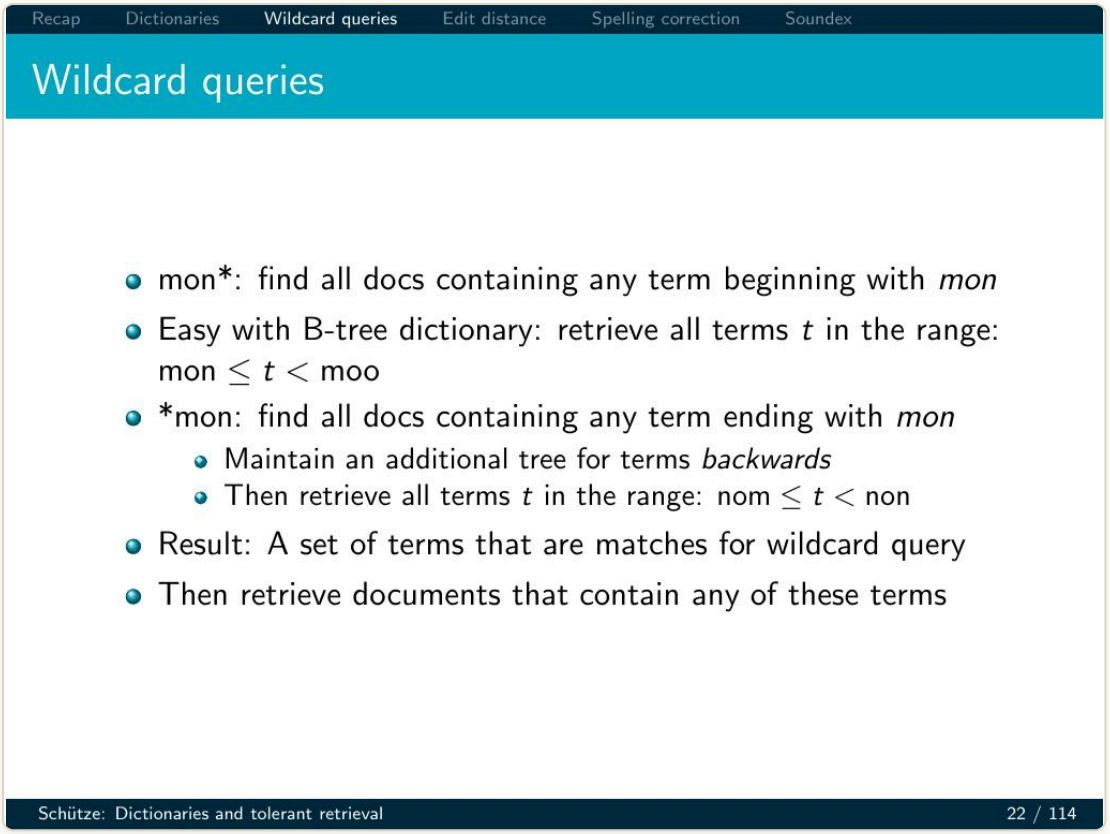


목차

- 복습 → 사전 → 와일드카드 질의 → 편집 거리 → 철자 교정 → Soundex
- 지금: **와일드카드 질의**

✍️ 메모

— 넘어가도 되는 쪽

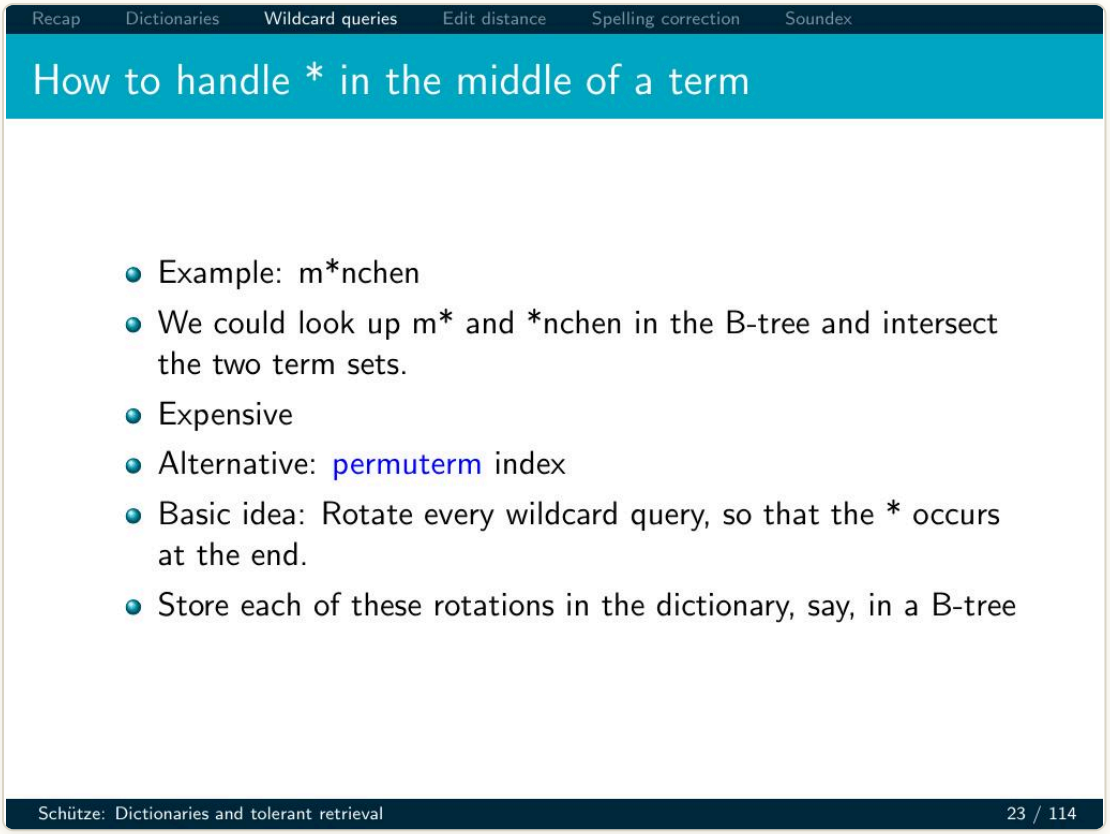


와일드카드 질의

- **mon***: mon으로 시작하는 단어가 있는 모든 문서
  - B-트리 사전이면 쉽다: 범위 **mon ≤ t < moo**의 모든 단어
- ***mon**: mon으로 끝나는 단어가 있는 모든 문서
  - 단어를 **거꾸로** 저장한 트리를 하나 더 둔다
  - 그다음 범위 **nom ≤ t < non**의 단어
- 결과: 와일드카드 질의에 맞는 **단어 집합**
- 그다음 이 단어 중 **하나라도** 있는 문서를 가져온다

✍ 메모

★ 시험 뒤쪽 * → 정방향 B-트리 / 앞쪽 * → 역방향 B-트리

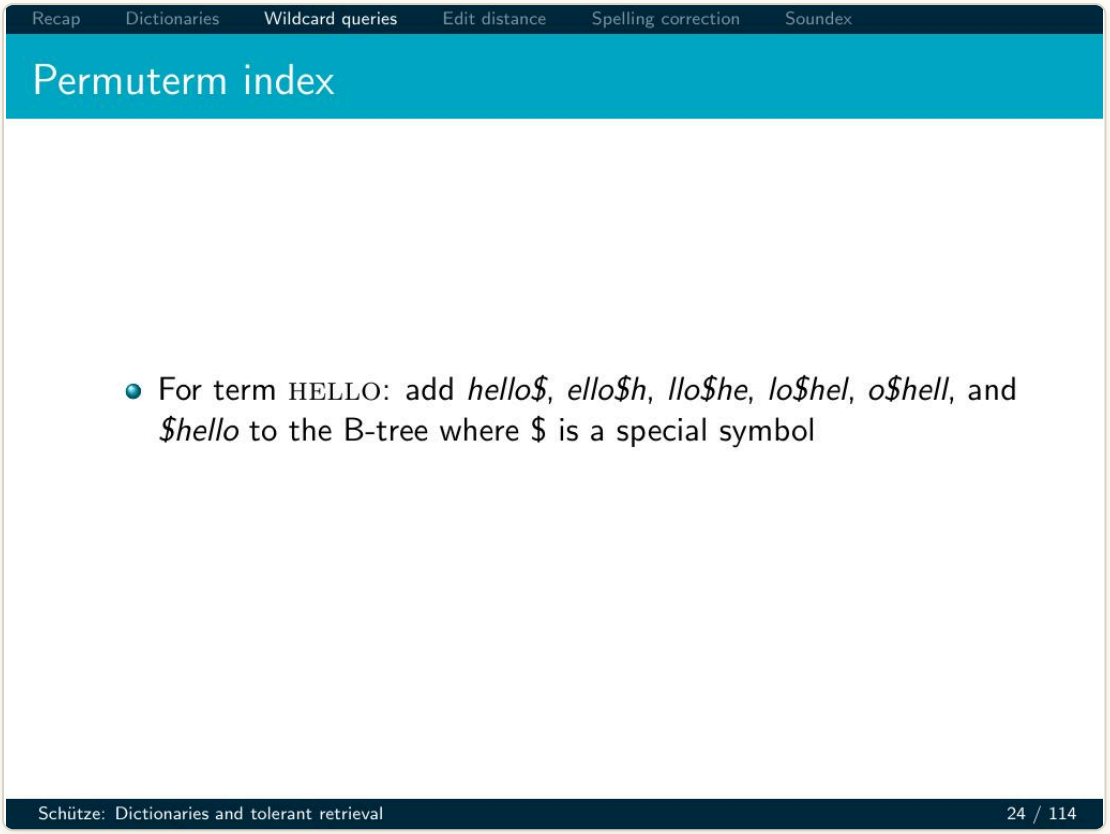


단어 가운데의 * 처리

- 예: **m*nchen**
- B-트리에서 m*와 *nchen을 찾아 두 단어 집합을 **교집합** — 비싸다
- 대안: **permuterm 색인**
  - 기본 아이디어: 모든 와일드카드 질의를 **회전**해 *가 **끝에** 오게 한다
  - 이 회전들을 모두 사전(예: B-트리)에 저장

✍ 메모

— 넘어가도 되는 쪽

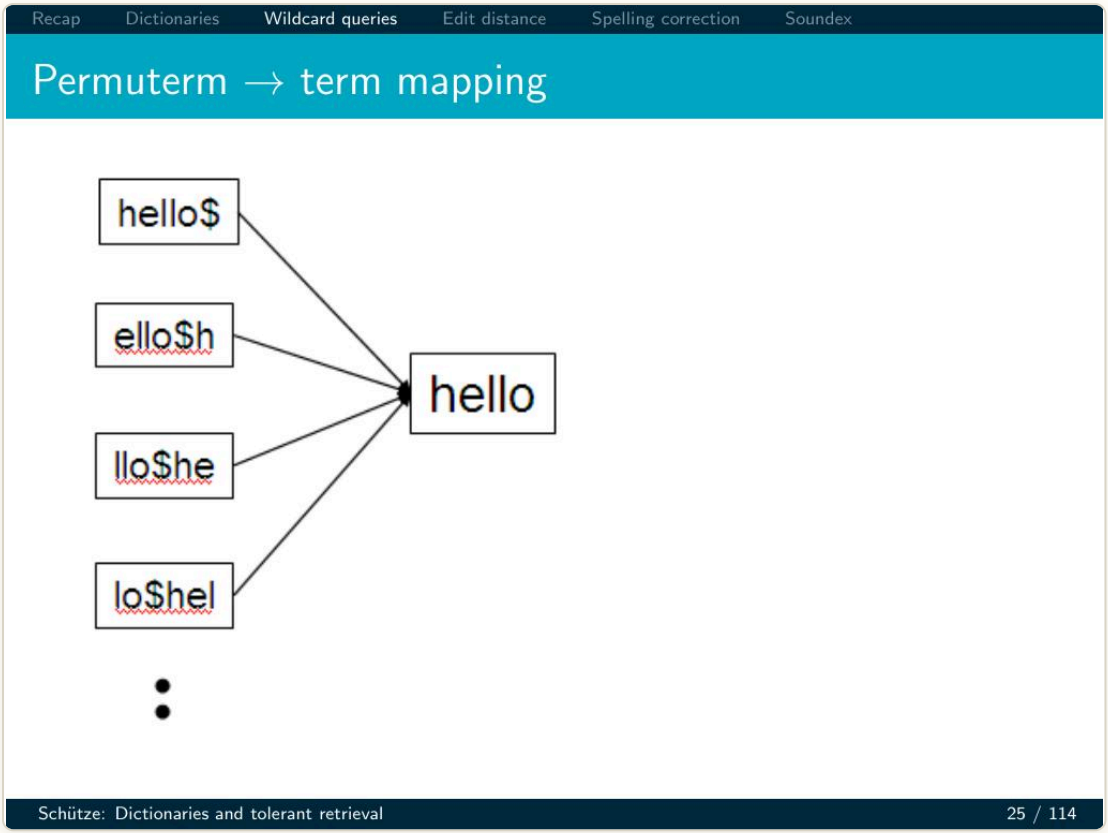


Permuterm 색인

- 단어 hello에 대해 **hello$, ello$h, llohe, lohel, o$hell, $hello**를 B-트리에 추가 ($ = 특수 기호)

✍️ 메모

★ 시험 단어 + $의 모든 회전을 저장 (글자 수 + 1개)

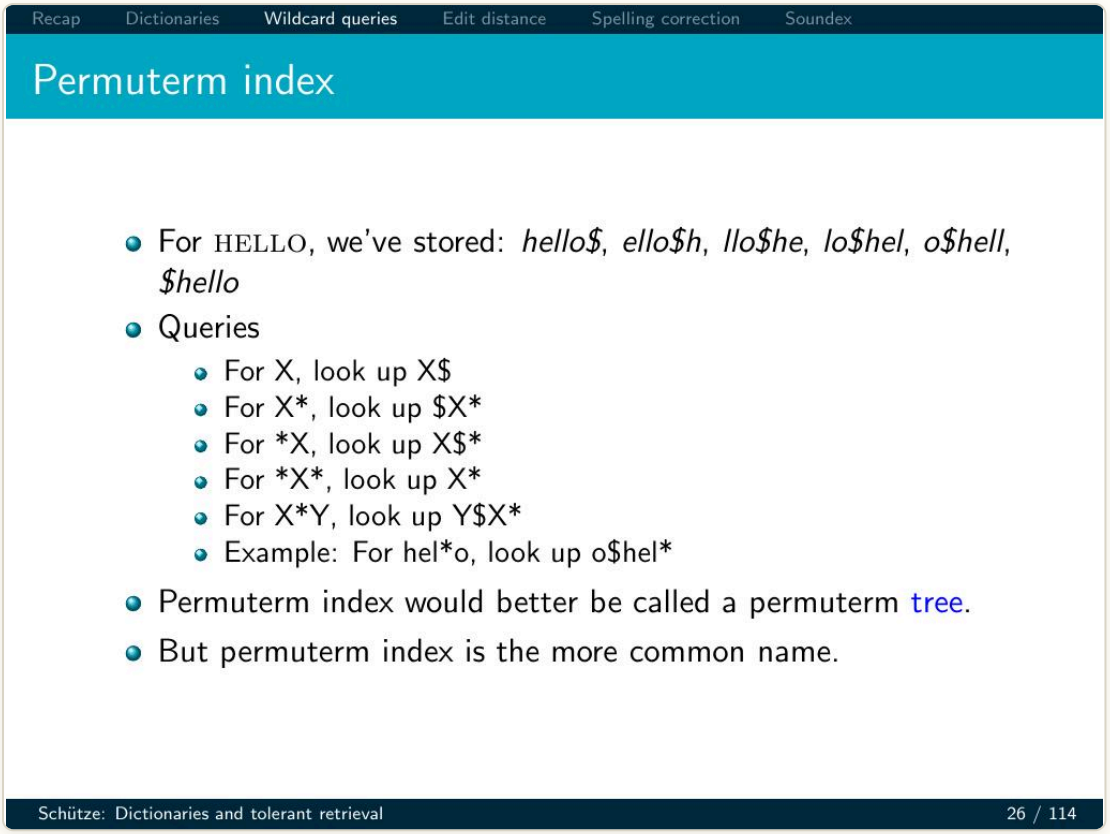


Permuterm → 단어 대응 (그림)

- hello$, ello$h, llohe, lohel, … 모두 원래 단어 **hello**를 가리킨다

✍ 메모

— 넘어가도 되는 쪽



Permuterm 색인: 질의 규칙

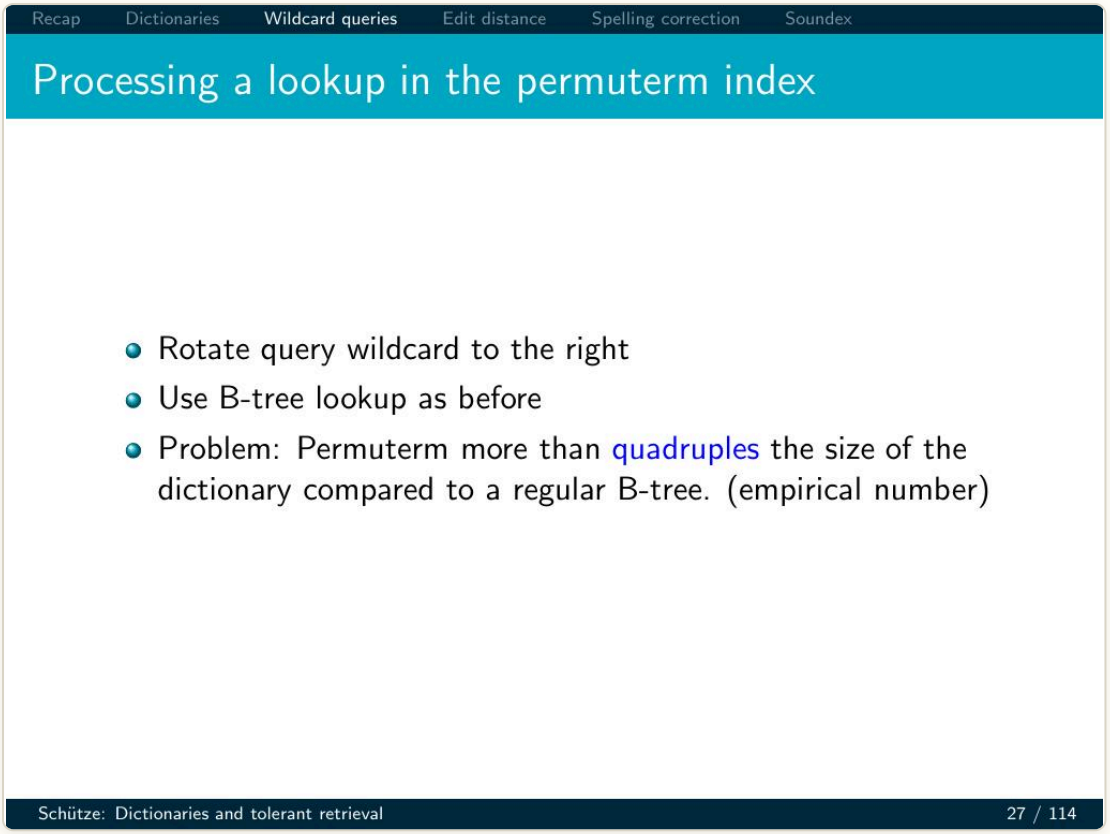
- hello에 대해 저장: hello$, ello$h, llohe, lohel, o$hell, $hello

```
X     → X$ 찾기
X*    → $X* 찾기
*X    → X$* 찾기
*X*   → X* 찾기
X*Y   → Y$X* 찾기
```

- 예: hel*o → **o$hel*** 찾기
- permuterm '트리'라 불러야 맞지만 permuterm '색인'이 더 흔한 이름

✍ 메모

★ 시험 규칙: $를 붙이고 *가 끝에 오도록 회전 — X*Y → Y$X* (변환 문제 단골)

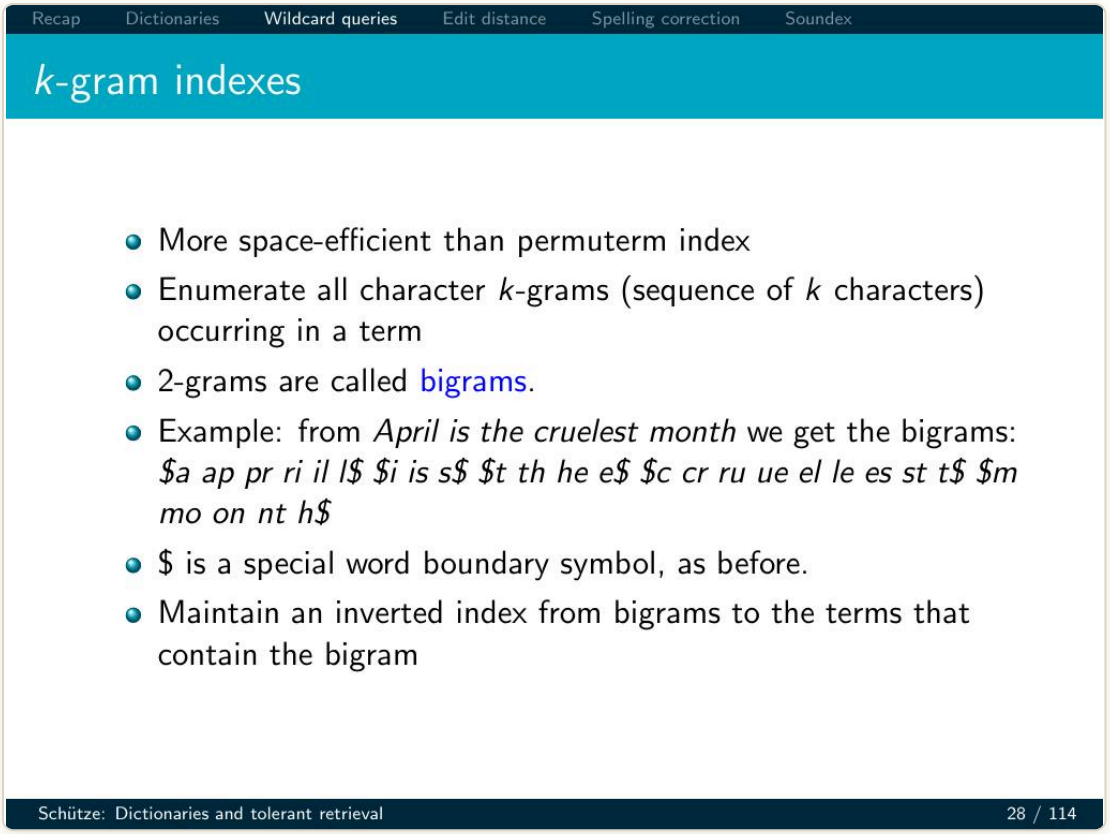


Permuterm 색인 조회 과정

- 질의 와일드카드를 **오른쪽으로 회전**
- 앞과 같이 B-트리 조회
- **문제**: permuterm은 일반 B-트리보다 사전 크기를 **4배 넘게** 키운다 (경험적 수치)

✍ 메모

⚠ 주의 permuterm 단점 = 공간 (4배 이상)



k-gram 색인

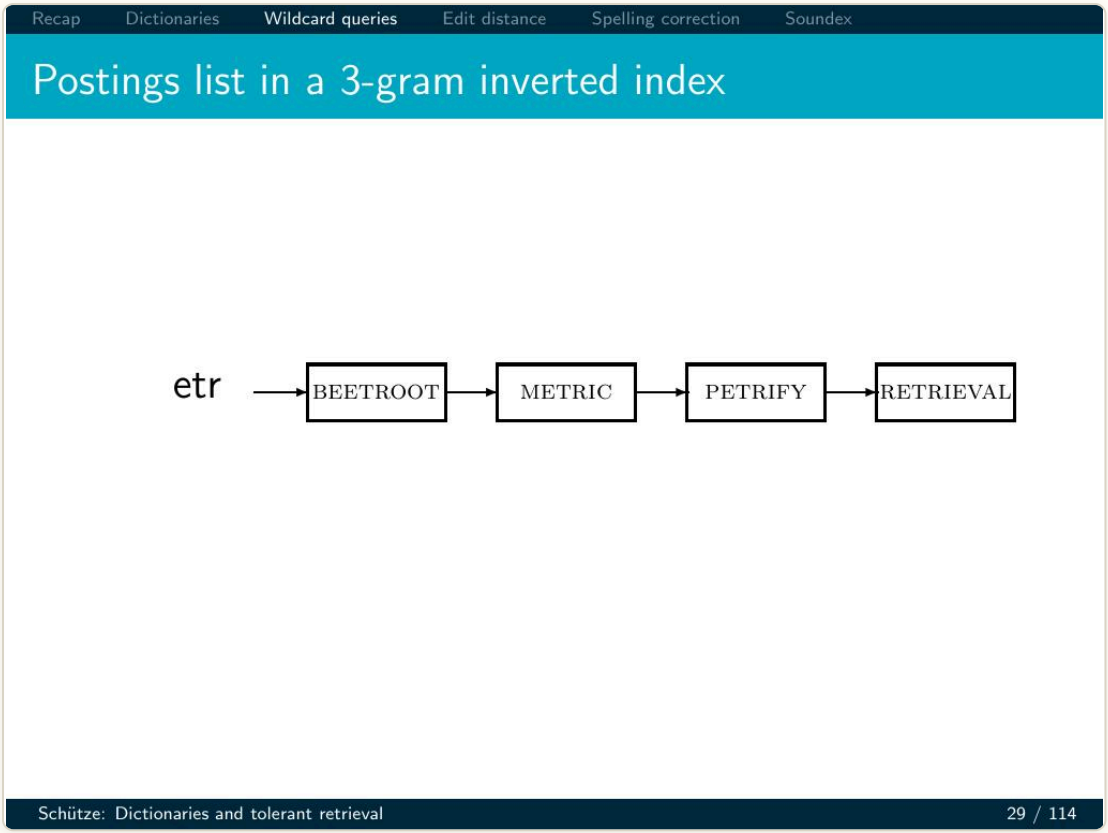
- permuterm보다 **공간 효율적**
- 단어에 나오는 모든 문자 **k-gram**(k글자 연속)을 나열
- 2-gram = **바이그램**
- 예: April is the cruelest month의 바이그램
  - $a ap pr ri il l$ $i is s$ $t th he e$ $c cr ru ue el le es st t$ $m mo on nt th h$

= 단어 경계 기호

- 바이그램 → 그 바이그램을 포함한 **단어**로 가는 역색인을 유지

✍ 메모

— 넘어가도 되는 쪽

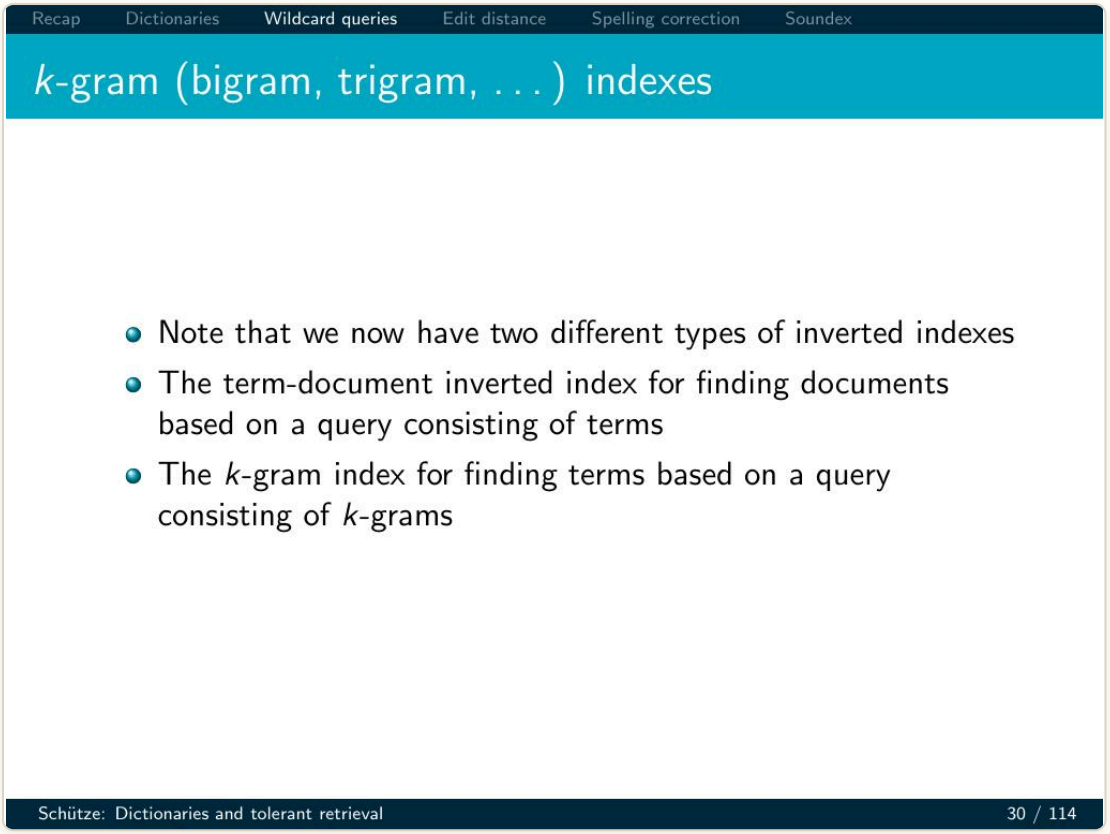


3-gram 역색인의 포스팅 리스트

- etr → BEETROOT → METRIC → PETRIFY → RETRIEVAL

✍️ 메모

💡 이해 **k-gram 색인의 포스팅은 docID가 아니라 '단어'**

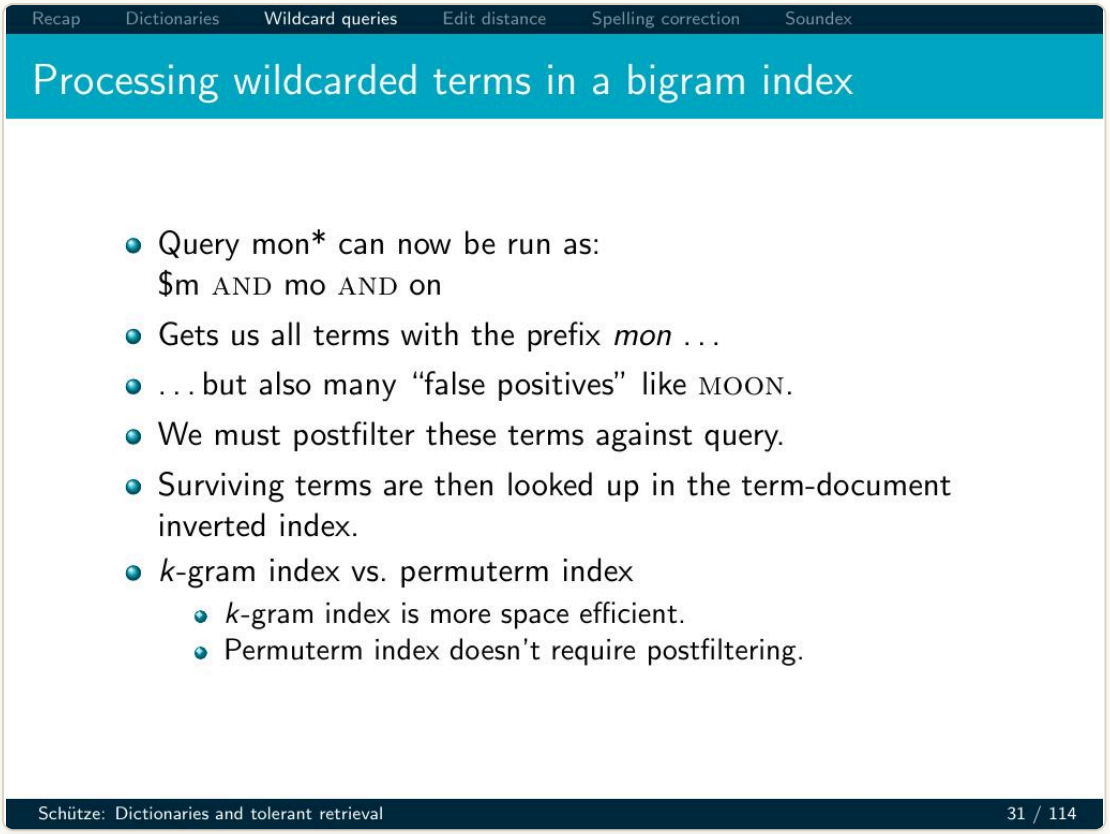


k-gram 색인 (바이그램, 트라이그램 ...)

- 이제 역색인이 **두 종류**
  - **단어-문서 역색인**: 단어로 된 질의로 **문서**를 찾음
  - **k-gram 색인**: k-gram으로 된 질의로 **단어**를 찾음

✍ 메모

★ 시험 **k-gram 색인 = k-gram → 단어 / 일반 역색인 = 단어 → 문서**



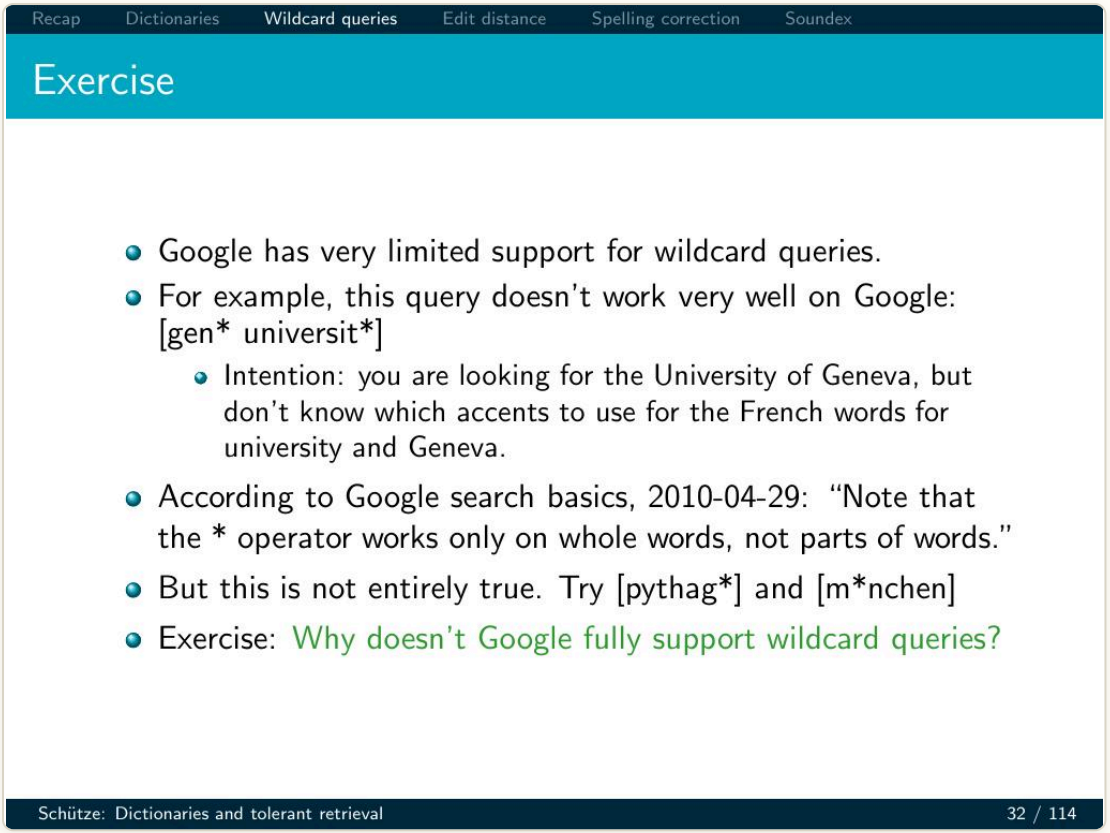
바이그램 색인으로 와일드카드 처리

- 질의 mon*는 이제 **$m AND mo AND on**으로 실행
- mon으로 시작하는 단어를 모두 얻지만 ...
- ... **moon** 같은 **거짓 양성**도 많다
- 이 단어들을 질의와 다시 비교해 **후처리(postfilter)**해야
- 살아남은 단어로 단어-문서 역색인을 조회
- **k-gram vs permuterm**
  - k-gram이 **공간 효율적**
  - permuterm은 **후처리가 필요 없다**

✍ 메모

★ 시험 비교 한 줄: k-gram = 공간↓ 후처리 필요 / permuterm = 공간↑ 후처리 불필요

✏ 예·풀이 moon: m, mo, oo, on, n → $m·mo·on을 다 가져서 걸림

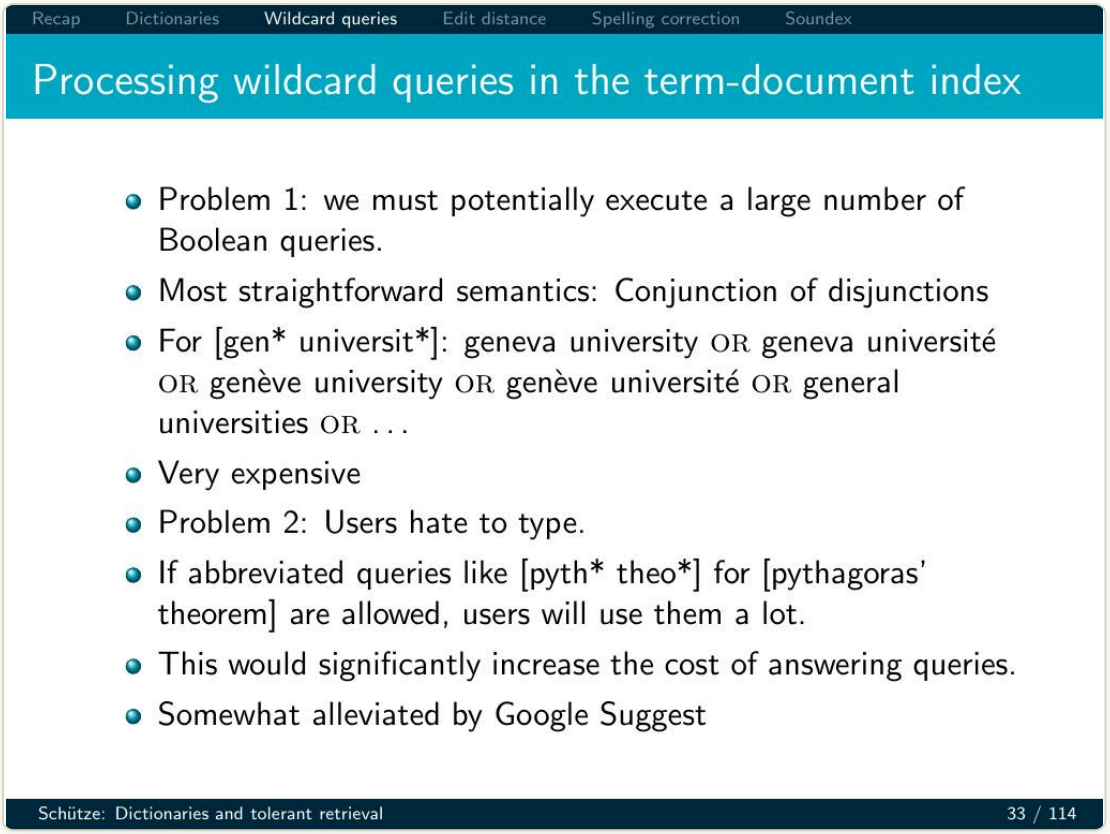


연습

- 구글은 와일드카드 질의를 **아주 제한적으로** 지원
  - 예: [gen* universit*]는 구글에서 잘 안 된다
  - 의도: 제네바 대학을 찾는데 프랑스어 '대학'·'제네바'의 악센트를 모름
- 구글 검색 도움말(2010-04-29): '* 연산자는 단어의 일부가 아니라 **단어 전체**에만 쓰인다'
  - 하지만 꼭 그렇진 않다. [pythag*]와 [m*nchen]을 해 보라
- 연습: 구글은 왜 와일드카드 질의를 완전히 지원하지 않을까?

✍️ 메모

💡 이해 답은 다음 쪽: 불리언 질의 폭증 + 사용자가 남용

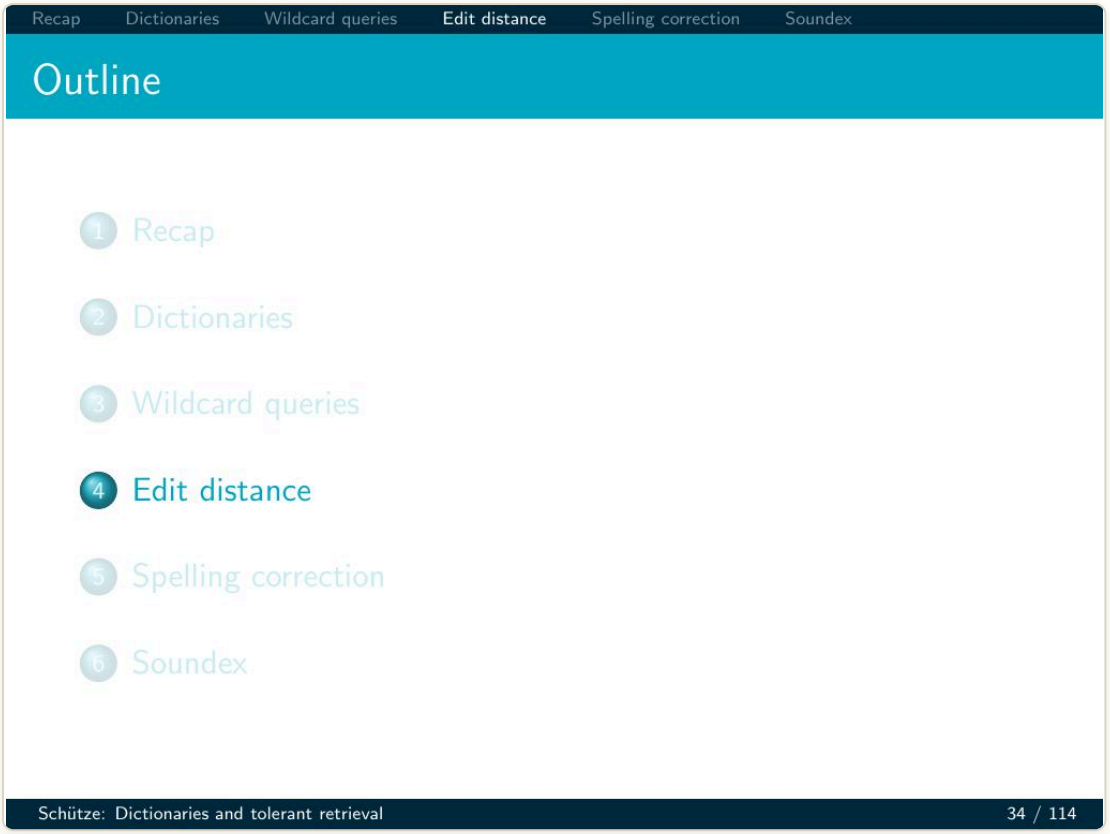


단어-문서 색인에서의 와일드카드 처리

- **문제 1**: 엄청나게 많은 불리언 질의를 실행해야 할 수 있다
  - 가장 단순한 의미: **OR들의 AND**
  - [gen* universit*] → geneva university OR geneva université OR genève university OR genève université OR general universities OR ...
  - **매우 비싸다**
- **문제 2**: 사용자는 타이핑을 싫어한다
  - [pythagoras' theorem] 대신 [pyth* theo*] 같은 줄임 질의를 허용하면 사용자가 **많이 쓸 것**
  - 질의 처리 비용이 크게 늘어난다
- 구글 자동완성(Google Suggest)이 어느 정도 완화

✍ 메모

★ 시험 구글이 와일드카드를 제한하는 이유 2개

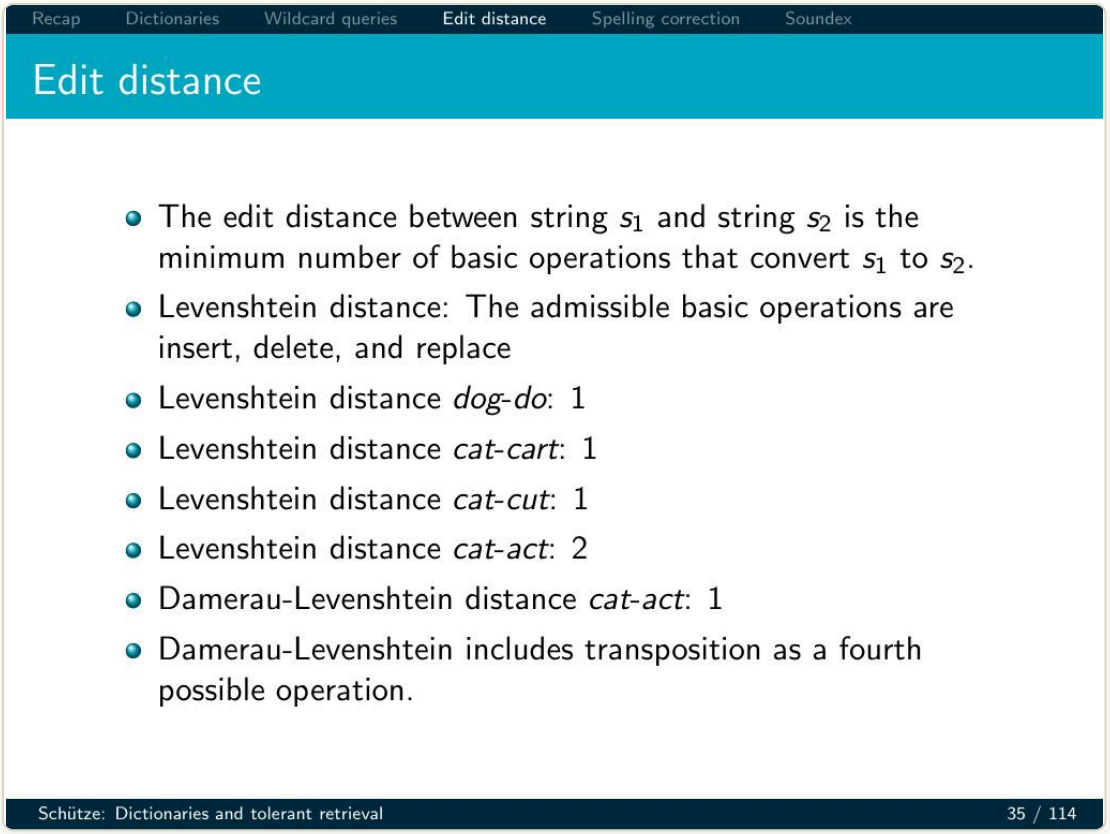


목차

- 복습 → 사전 → 와일드카드 질의 → 편집 거리 → 철자 교정 → Soundex
- 지금: **편집 거리**

✍ 메모

— 넘어가도 되는 쪽

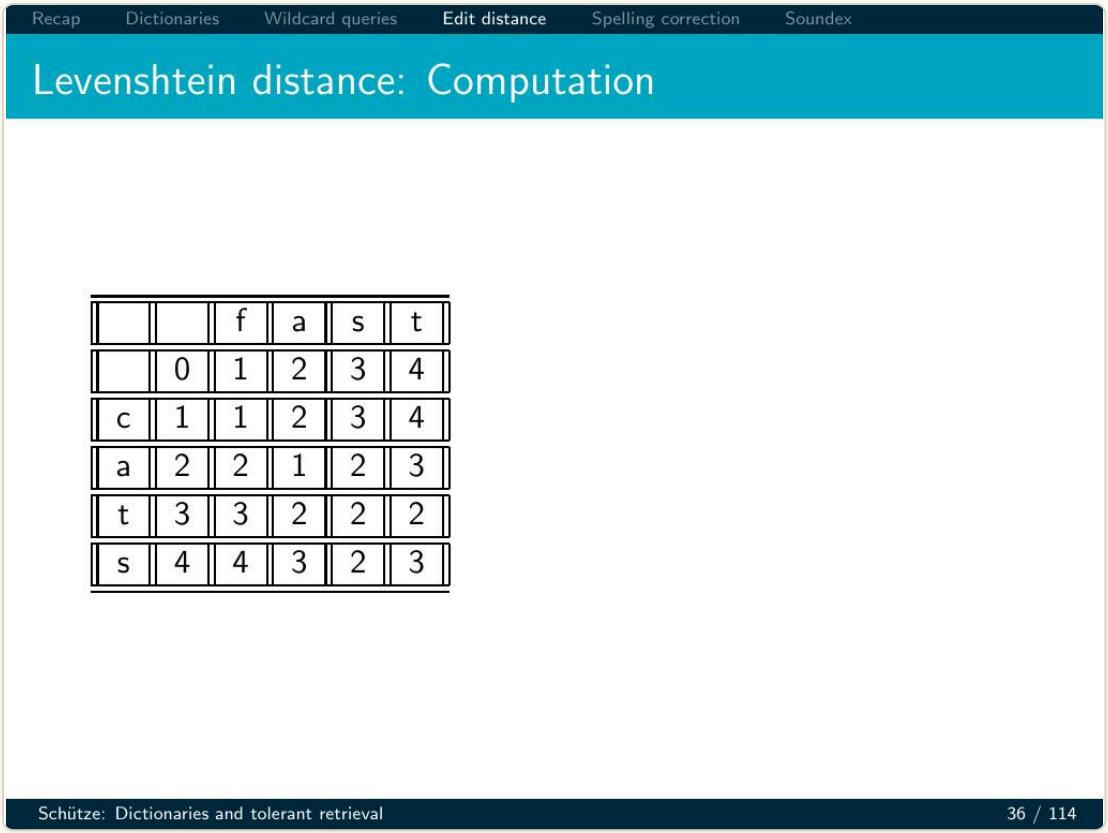


편집 거리 (Edit distance)

- 문자열 s1과 s2의 편집 거리 = s1을 s2로 바꾸는 **기본 연산의 최소 횟수**
- **레벤슈타인 거리**: 허용 연산은 **삽입, 삭제, 교체**
  - dog–do: 1 / cat–cart: 1 / cat–cut: 1 / cat–act: **2**
- **다메라우-레벤슈타인** 거리 cat–act: **1**
  - 다메라우-레벤슈타인은 **전치(transposition, 이웃 글자 맞바꾸기)**를 넷째 연산으로 포함

✍ 메모

★ 시험 cat→act: 레벤슈타인 2 (교체 2번) / 다메라우 1 (ca↔ac 전치)



		f	a	s	t
	0	1	2	3	4
c	1	1	2	3	4
a	2	2	1	2	3
t	3	3	2	2	2
s	4	4	3	2	3

레벤슈타인 거리: 계산 (표)

- CATS(행) × FAST(열)
- 첫 행 0 1 2 3 4, 첫 열 0 1 2 3 4
- c: 1 1 2 3 4 / a: 2 2 1 2 3 / t: 3 3 2 2 2 / s: 4 4 3 2 3
- → cats와 fast의 거리 = **3**

✍ 메모

— 넘어가도 되는 쪽

Levenshtein distance: Algorithm

LevenshteinDistance(s_1, s_2)

```
 1  for i ← 0 to |s1|
 2  do m[i, 0] = i
 3  for j ← 0 to |s2|
 4  do m[0, j] = j
 5  for i ← 1 to |s1|
 6  do for j ← 1 to |s2|
 7      do if s1[i] = s2[j]
 8             then m[i, j] = min{m[i-1, j]+1, m[i, j-1]+1, m[i-1, j-1]}
 9             else m[i, j] = min{m[i-1, j]+1, m[i, j-1]+1, m[i-1, j-1]+1}
10  return m[|s1|, |s2|]
```

Operations: insert (cost 1), delete (cost 1), replace (cost 1), copy (cost 0)

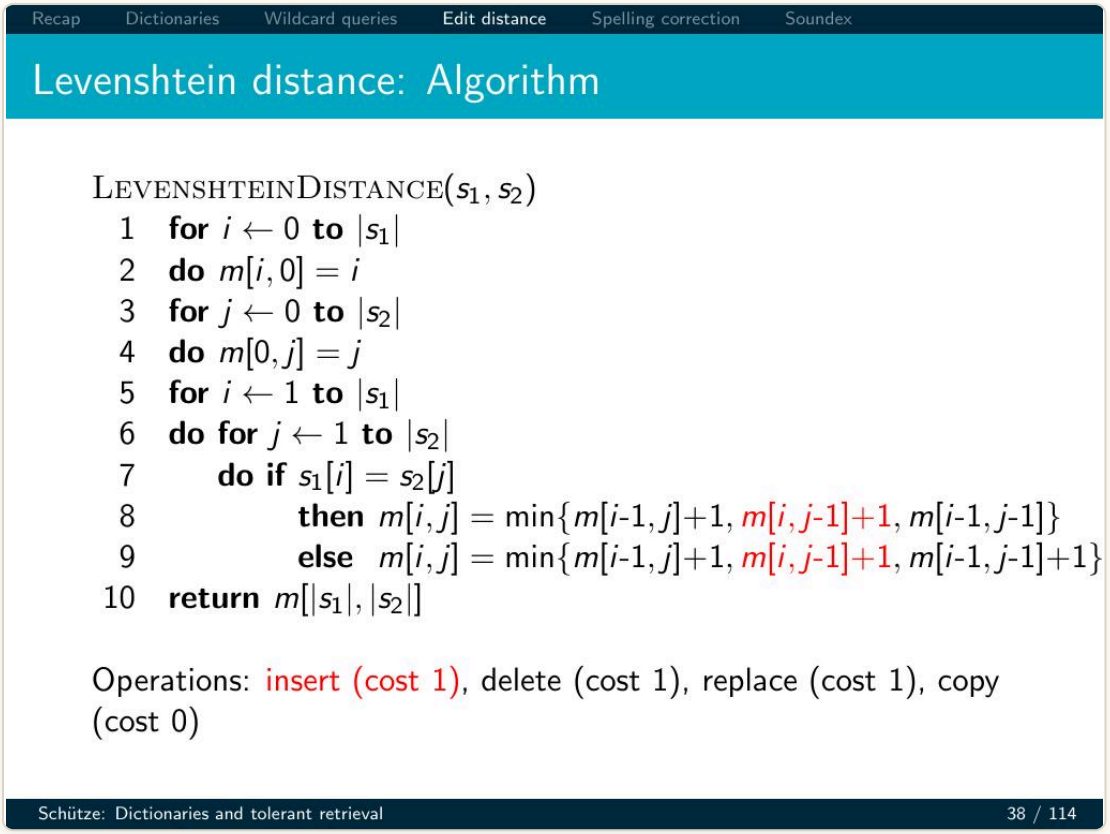
레벤슈타인 거리: 알고리즘

```
for i ← 0 to |s1|: m[i, 0] = i
for j ← 0 to |s2|: m[0, j] = j
for i ← 1 to |s1|, j ← 1 to |s2|:
  s1[i] = s2[j]이면 m[i,j] = min{m[i-1,j]+1, m[i,j-1]+1, m[i-1,j-1]}
  아니면          m[i,j] = min{m[i-1,j]+1, m[i,j-1]+1, m[i-1,j-1]+1}
return m[|s1|, |s2|]
```

- 연산 비용: 삽입 1, 삭제 1, 교체 1, **복사 0**

✍ 메모

★ 시험 점화식 그대로 외우기 — 손으로 표 채우는 문제 확실히 나옴



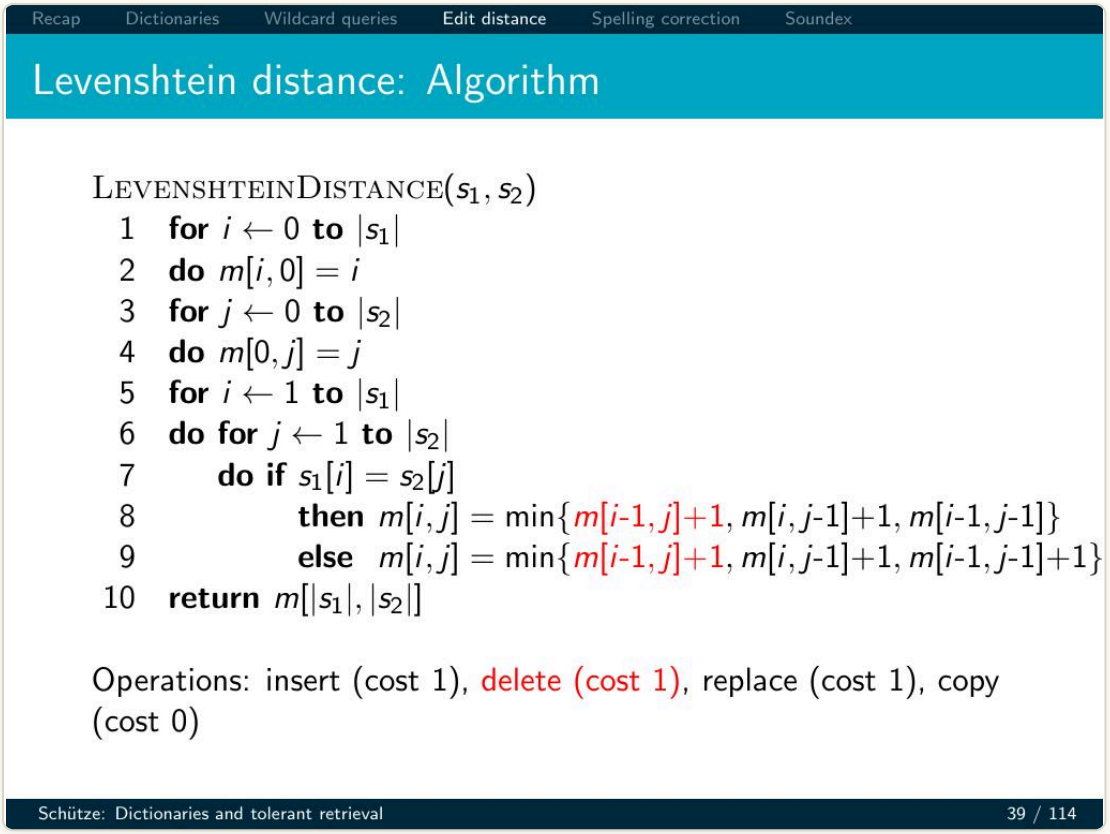
레벤슈타인 거리: 알고리즘 (계속)

- 앞 쪽과 같은 알고리즘 — 슬라이드가 한 줄씩 강조하며 넘어간다

```
1~4행: 첫 열 m[i,0] = i, 첫 행 m[0,j] = j (빈 문자열과의 거리)
7~9행: 글자가 같으면 대각선 + 0, 다르면 대각선 + 1, 위·왼쪽은 + 1, 셋 중 최소
```

✍ 메모

— 넘어가도 되는 쪽



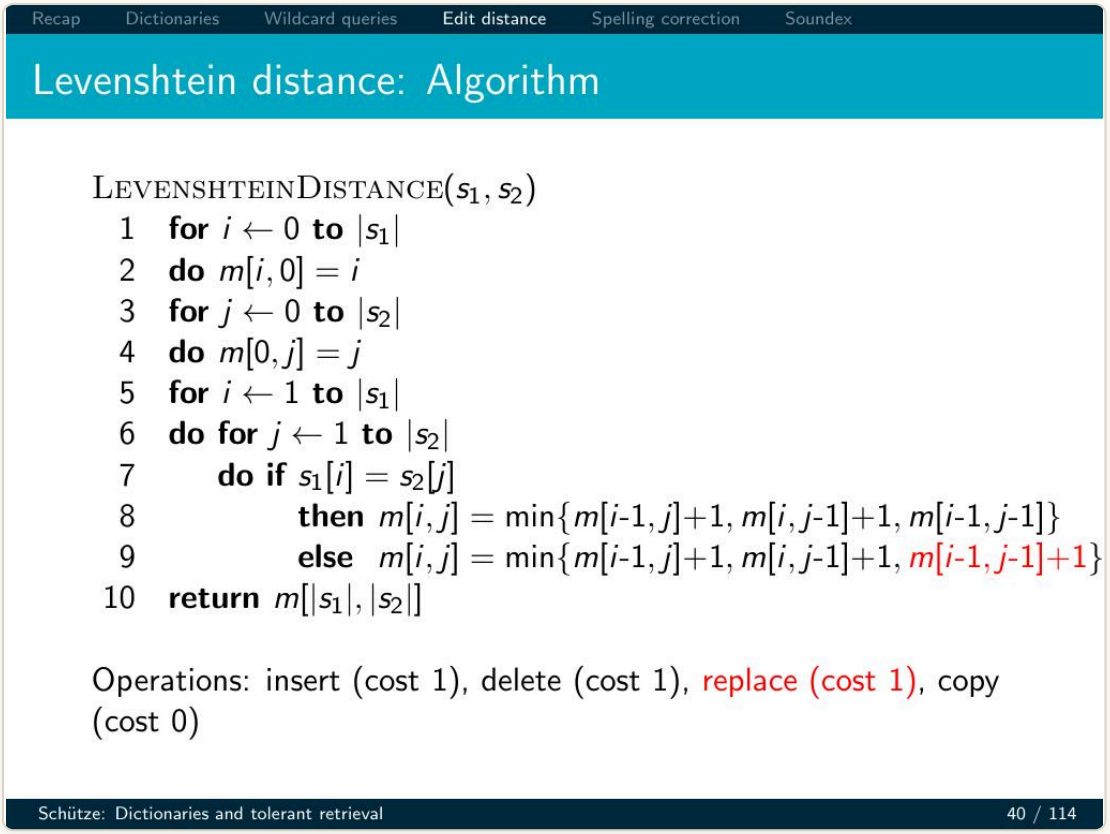
레벤슈타인 거리: 알고리즘 (계속)

- 앞 쪽과 같은 알고리즘 — 슬라이드가 한 줄씩 강조하며 넘어간다

```
1~4행: 첫 열 m[i,0] = i, 첫 행 m[0,j] = j (빈 문자열과의 거리)
7~9행: 글자가 같으면 대각선 + 0, 다르면 대각선 + 1, 위·왼쪽은 + 1, 셋 중 최소
```

✍ 메모

— 넘어가도 되는 쪽



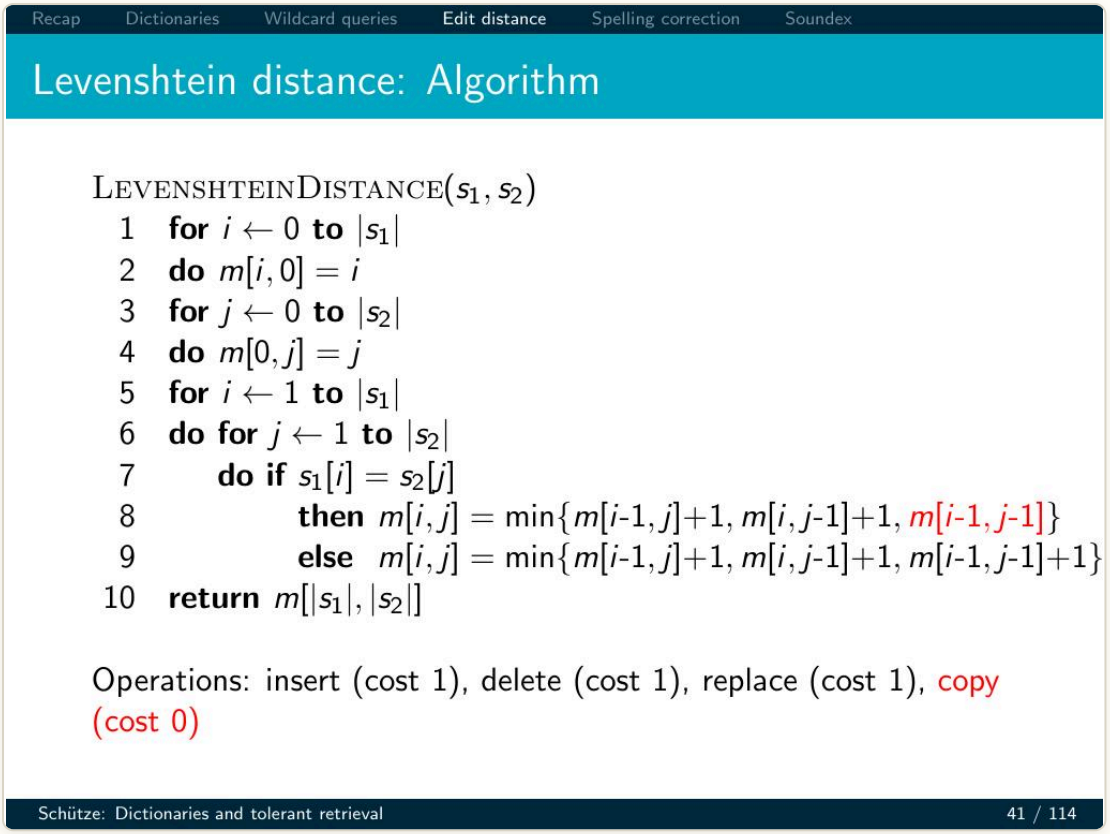
레벤슈타인 거리: 알고리즘 (계속)

- 앞 쪽과 같은 알고리즘 — 슬라이드가 한 줄씩 강조하며 넘어간다

```
1~4행: 첫 열 m[i,0] = i, 첫 행 m[0,j] = j (빈 문자열과의 거리)
7~9행: 글자가 같으면 대각선 + 0, 다르면 대각선 + 1, 위·왼쪽은 + 1, 셋 중 최소
```

✍ 메모

— 넘어가도 되는 쪽



레벤슈타인 거리: 알고리즘 (계속)

- 앞 쪽과 같은 알고리즘 — 슬라이드가 한 줄씩 강조하며 넘어간다

```
1~4행: 첫 열 m[i,0] = i, 첫 행 m[0,j] = j (빈 문자열과의 거리)
7~9행: 글자가 같으면 대각선 + 0, 다르면 대각선 + 1, 위·왼쪽은 + 1, 셋 중 최소
```

✍ 메모

— 넘어가도 되는 쪽

Levenshtein distance: Example

			f		a		s		t	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
c	**1**	**1**	*1*	*2*	**2**	3	**3**	4	**4**	5
	1		*2*	*1*	**2**	**2**	**3**	**3**	**4**	**4**
a	**2**	**2**	**2**	**2**	*1*	*3*	*3*	*4*	4	5
	2		3	**2**	*3*	*1*	*2*	*2*	**3**	**3**
t	**3**	**3**	**3**	**3**	3	**2**	*2*	*3*	*2*	*4*
	3		4	**3**	4	**2**	*3*	*2*	*3*	*2*
s	**4**	**4**	**4**	**4**	4	**3**	**2**	3	*3*	*3*
	4		5	**4**	5	**3**	4	**2**	*3*	*3*

레벤슈타인 거리: 예 (칸마다 4개 값)

- CATS × FAST를 칸마다 네 숫자로 보여 줌
  - 왼쪽 위 = 대각선에서 온 비용(복사·교체), 오른쪽 위 = 위에서 온 비용(삭제)
  - 왼쪽 아래 = 왼쪽에서 온 비용(삽입), 오른쪽 아래 = 셋 중 **최소** (실제 칸 값)

✍ 메모

— 넘어가도 되는 쪽

Each cell of Levenshtein matrix

cost of getting here from my upper left neighbor (copy or replace)	cost of getting here from my upper neighbor (delete)
cost of getting here from my left neighbor (insert)	the minimum of the three possible "movements"; the cheapest way of getting here

레벤슈타인 행렬의 각 칸

- 왼쪽 위: **왼쪽 위 이웃**에서 오는 비용 (복사 또는 교체)
- 오른쪽 위: **위 이웃**에서 오는 비용 (삭제)
- 왼쪽 아래: **왼쪽 이웃**에서 오는 비용 (삽입)
- 오른쪽 아래: 세 가지 '이동' 중 **최소** — 여기 오는 가장 싼 방법

✍ 메모

★ 시험 대각선 = 복사/교체, 위 = 삭제, 왼쪽 = 삽입

Levenshtein distance: Example

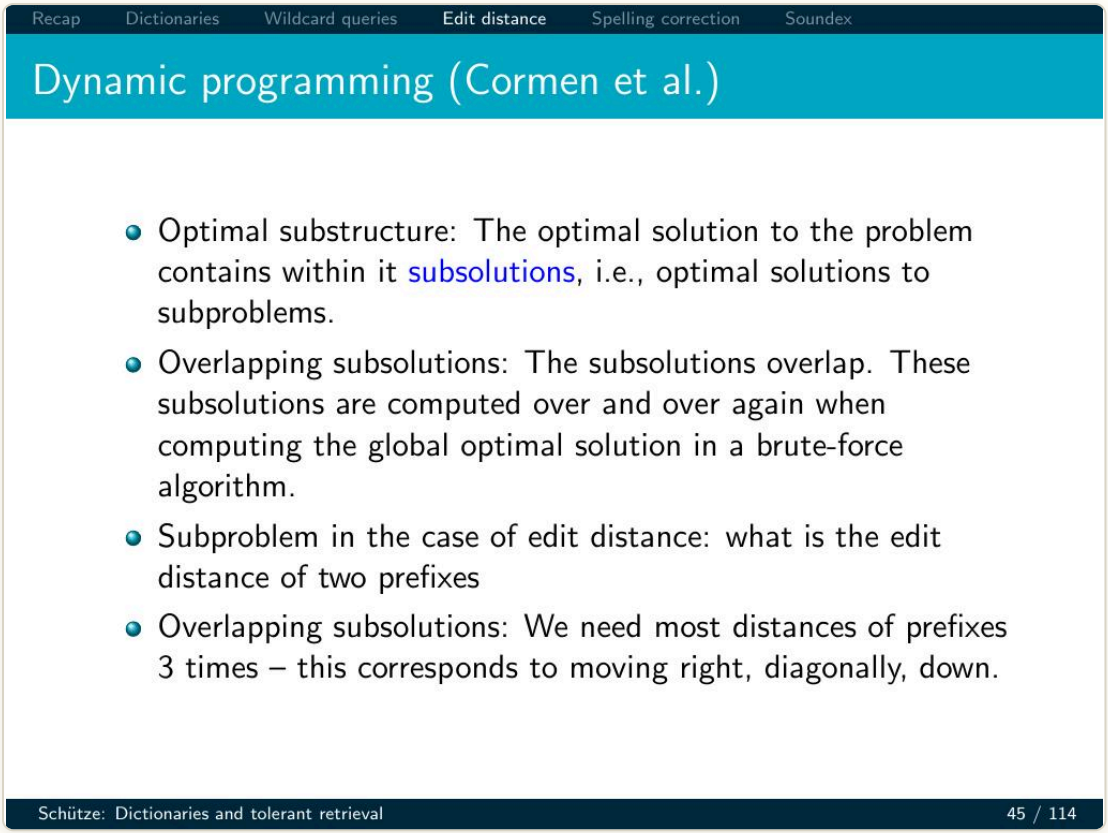
			f		a		s		t	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
c	**1**	**1**	*1*	*2*	**2**	3	**3**	4	**4**	5
	1		*2*	*1*	**2**	**2**	**3**	**3**	**4**	**4**
a	**2**	**2**	**2**	**2**	*1*	*3*	*3*	*4*	4	5
	2		3	**2**	*3*	*1*	*2*	*2*	**3**	**3**
t	**3**	**3**	**3**	**3**	3	**2**	*2*	*3*	*2*	*4*
	3		4	**3**	4	**2**	*3*	*2*	*3*	*2*
s	**4**	**4**	**4**	**4**	4	**3**	**2**	3	*3*	*3*
	4		5	**4**	5	**3**	4	**2**	*3*	*3*

레벤슈타인 거리: 예 (다시)

- p.42와 같은 표 — 오른쪽 아래 최종값 **3**

✍ 메모

— 넘어가도 되는 쪽

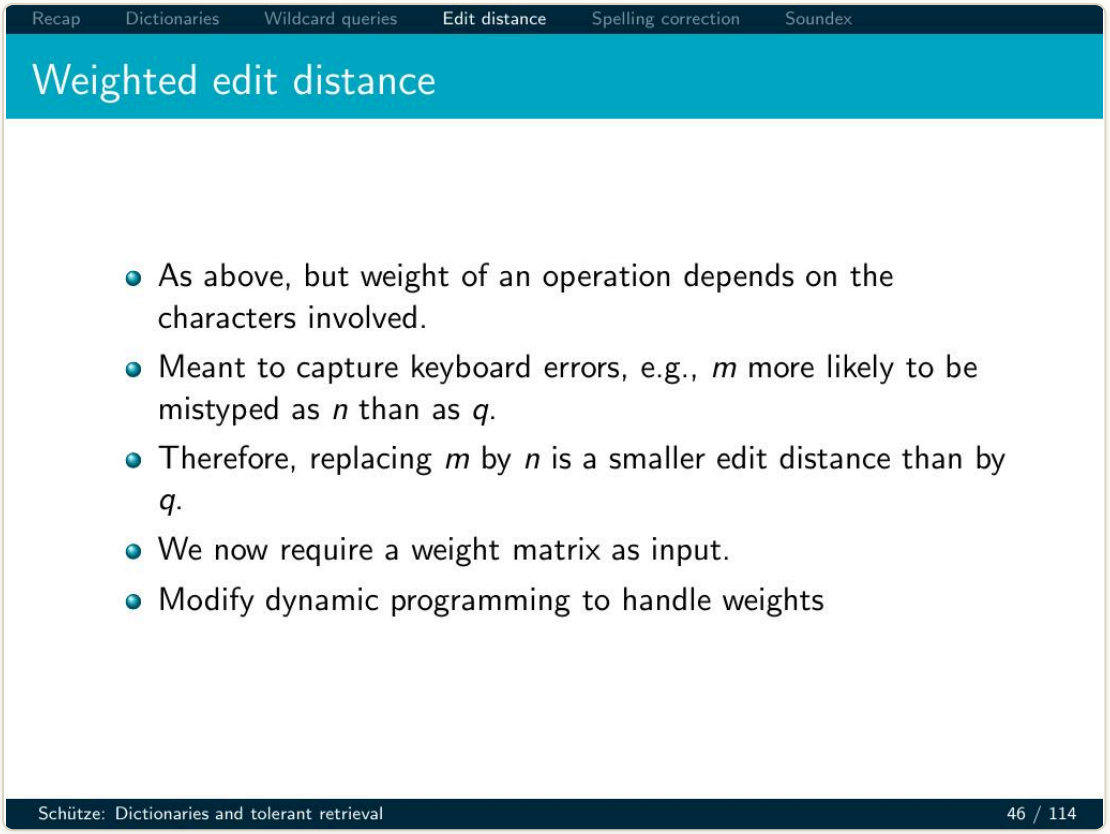


동적 계획법 (Cormen 등)

- **최적 부분 구조**: 문제의 최적해가 부분 문제의 최적해를 포함한다
- **겹치는 부분 해**: 부분 해들이 겹친다 — 완전 탐색이면 같은 부분 해를 계속 다시 계산
- 편집 거리의 부분 문제: **두 접두어**의 편집 거리
- 겹치는 부분 해: 접두어 거리 대부분을 **3번** 쓴다 — 오른쪽, 대각선, 아래로 이동할 때

✍ 메모

★ 시험 DP 조건 2개: 최적 부분 구조 + 겹치는 부분 문제

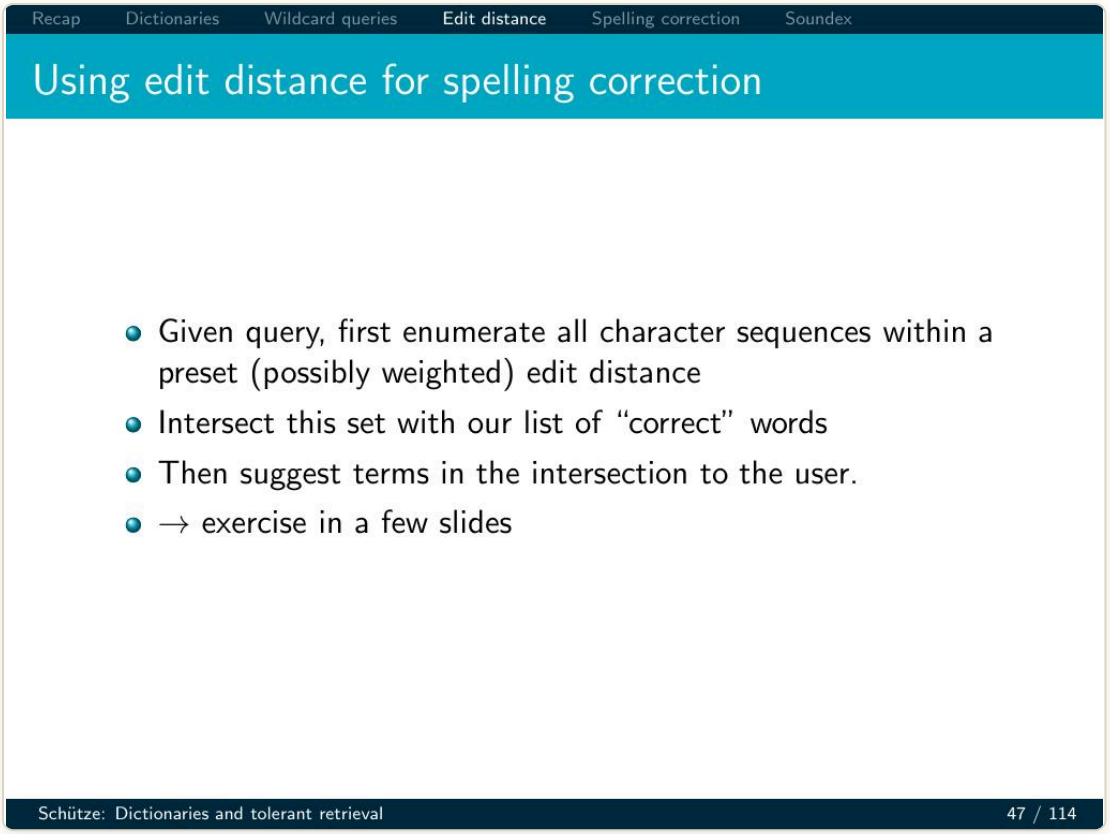


가중 편집 거리 (Weighted edit distance)

- 앞과 같지만 연산의 **가중치가 관련 글자에 따라** 달라진다
- **키보드 오타**를 반영하려는 것 — m은 q보다 n으로 잘못 칠 가능성이 크다
- 그래서 m→n 교체가 m→q보다 편집 거리가 **작다**
- 입력으로 **가중치 행렬**이 필요
- 동적 계획법을 가중치를 다루도록 수정

✍ 메모

— 넘어가도 되는 쪽

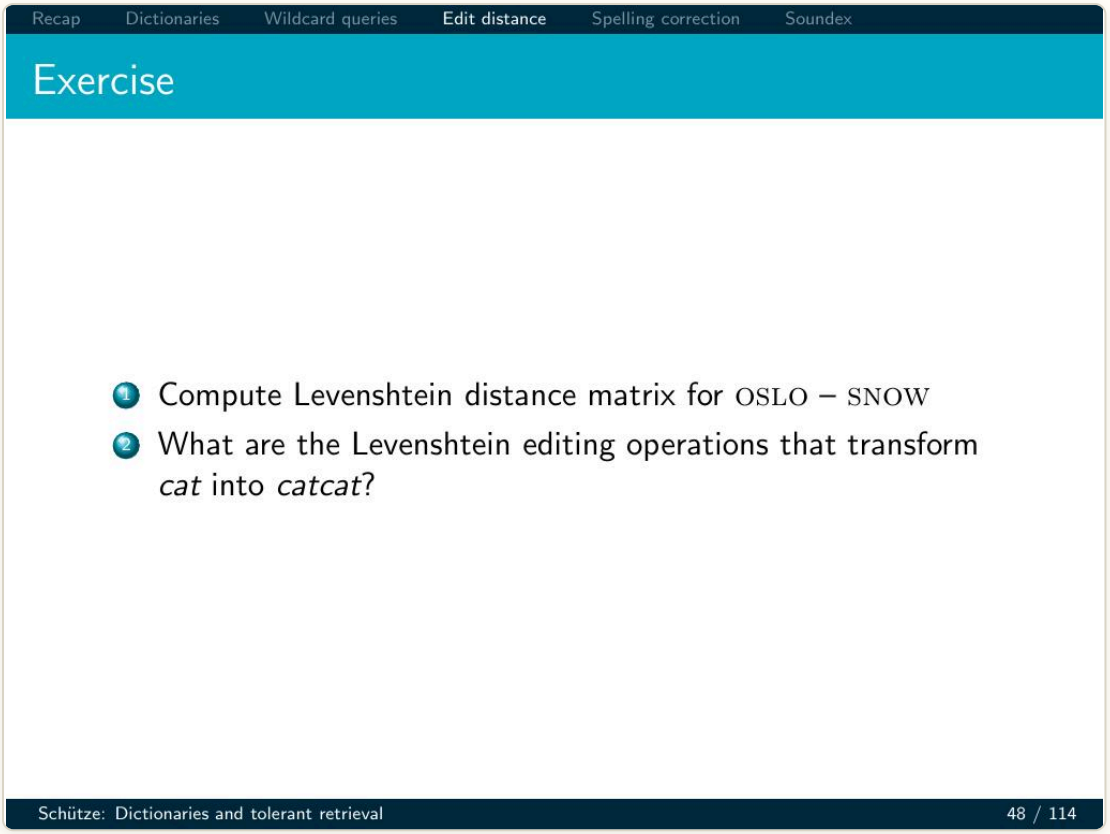


철자 교정에 편집 거리 쓰기

- 질의가 주어지면, 정해진 (가중) 편집 거리 **안의 모든 문자열**을 나열
- 그 집합을 '올바른' 단어 목록과 **교집합**
- 교집합의 단어를 사용자에게 **제안**
- → 몇 쪽 뒤 연습

✍ 메모

— 넘어가도 되는 쪽

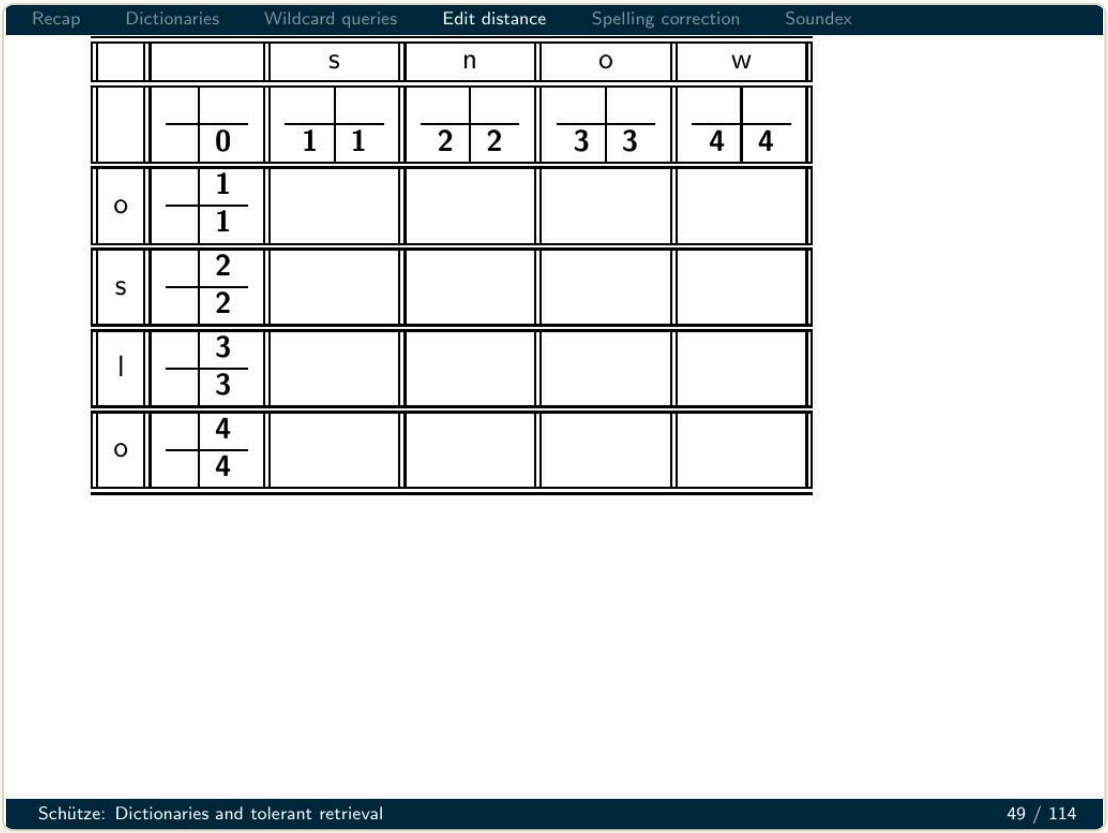


연습

- ① OSLO – SNOW의 레벤슈타인 거리 행렬을 계산하라
- ② cat을 catcat으로 바꾸는 레벤슈타인 편집 연산은?

✍️ 메모

✏ 예·풀이 답: ① 거리 3 (p.82) ② 거리 3 = 복사 3번 + 삽입 3번 (p.93)

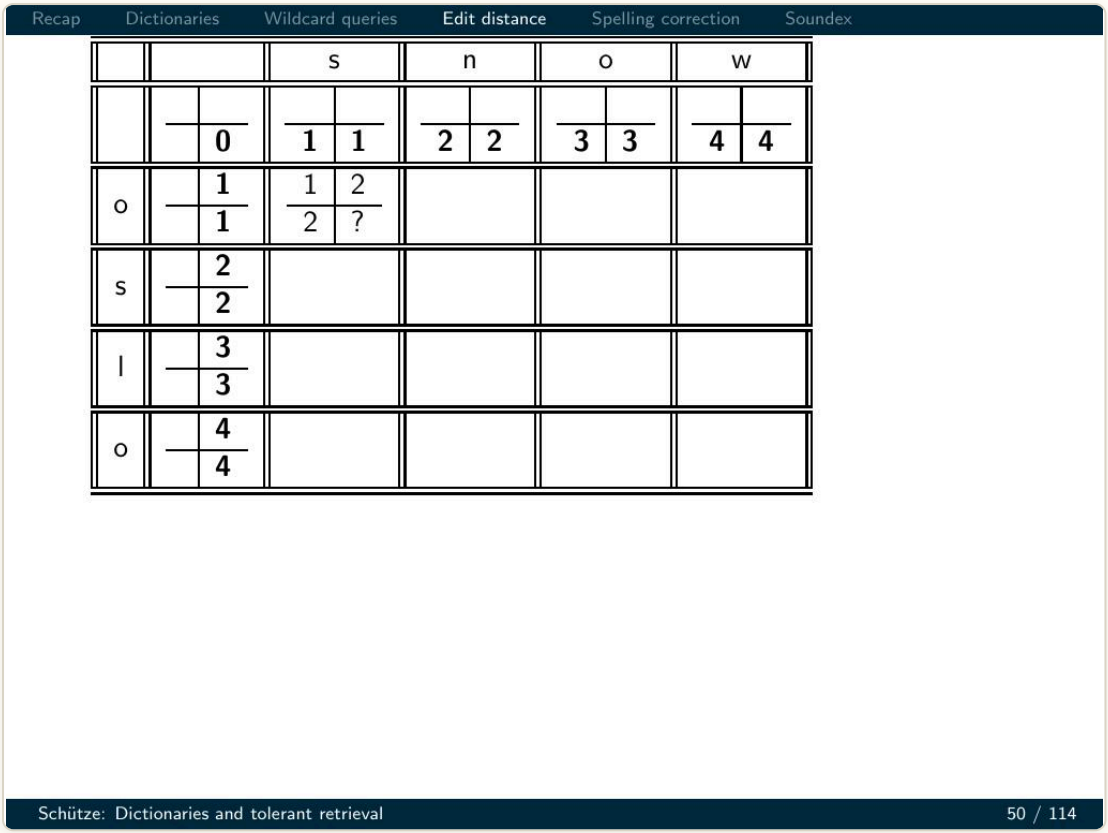


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

💡 이해 p.49~82는 같은 표를 채워 가는 장면 — 최종 결과는 p.82

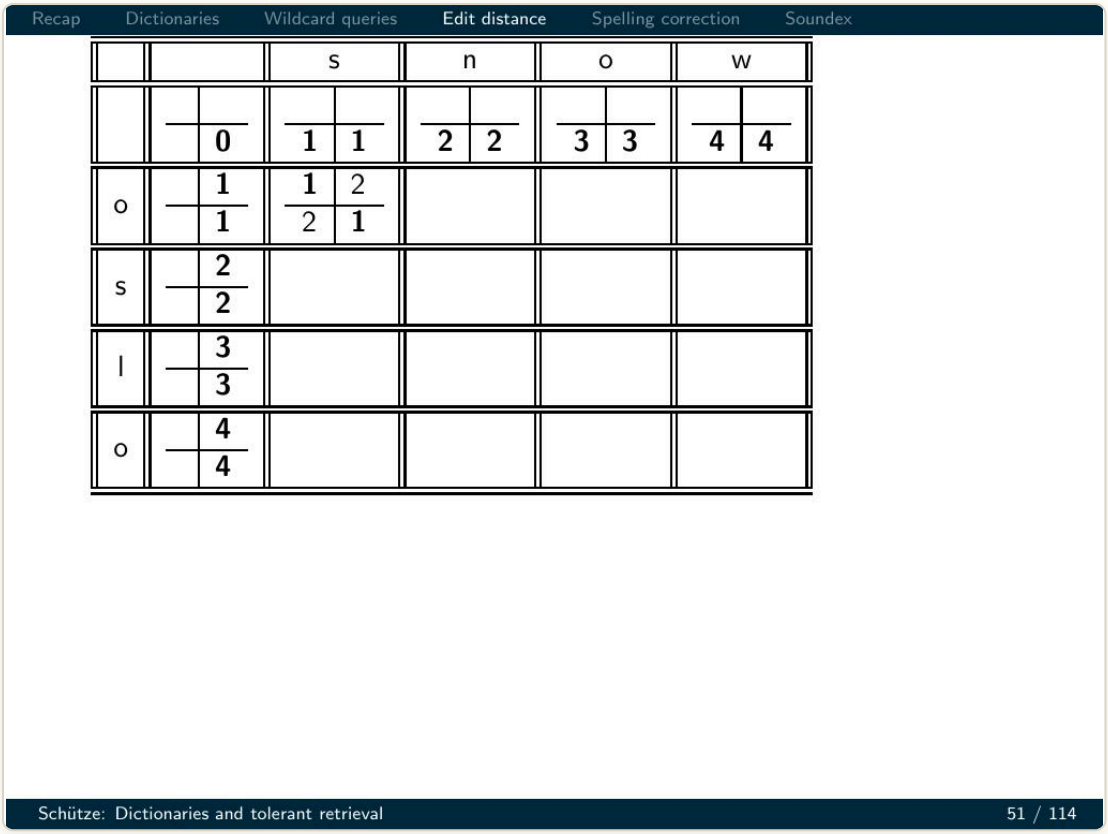


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

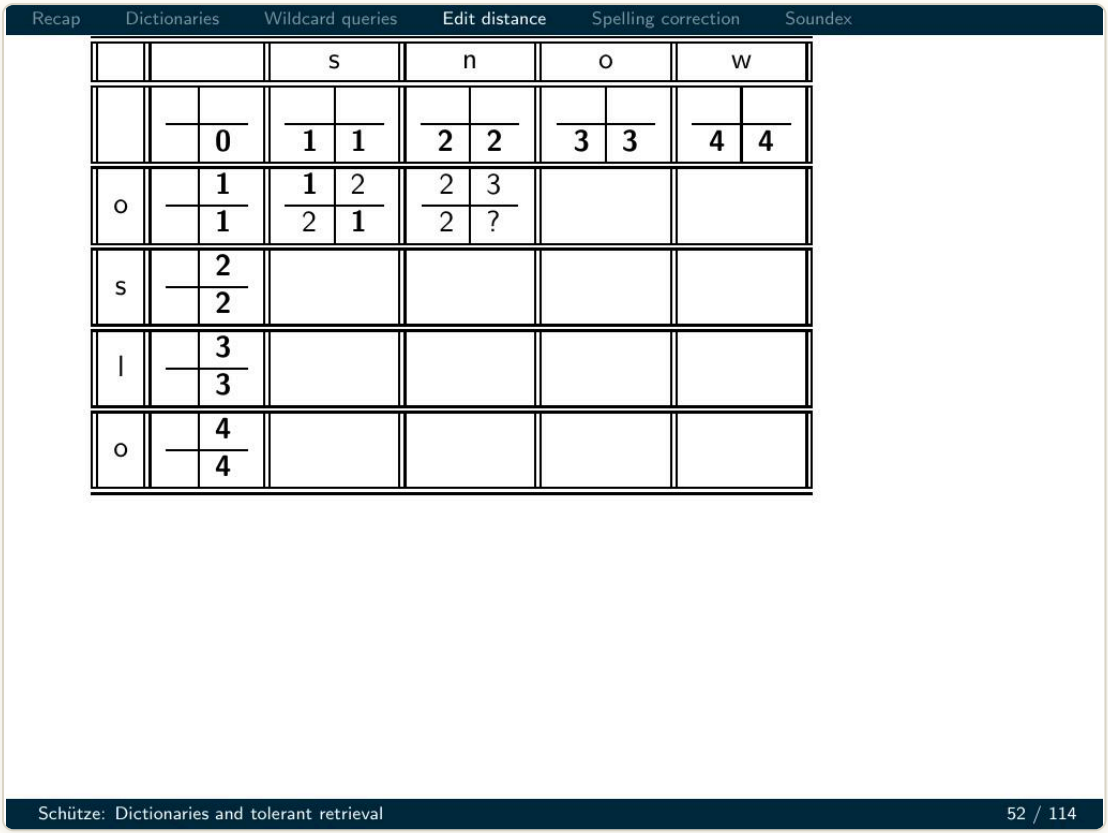


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

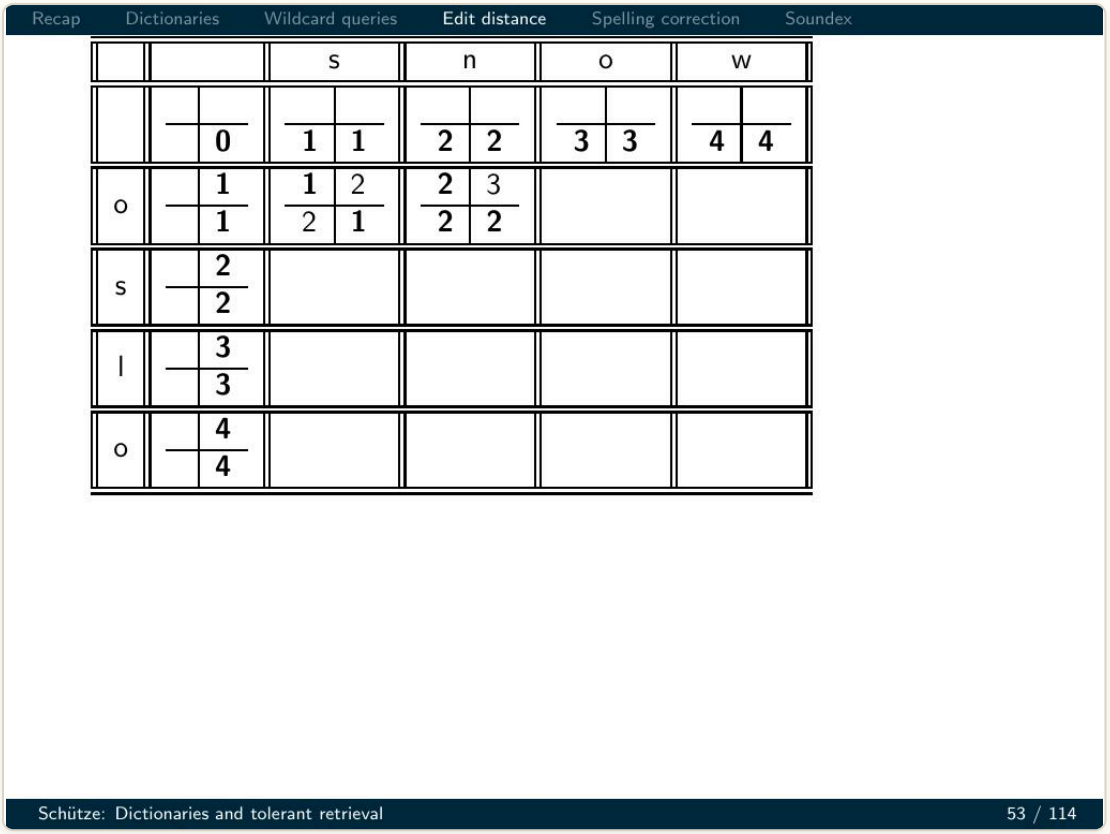


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

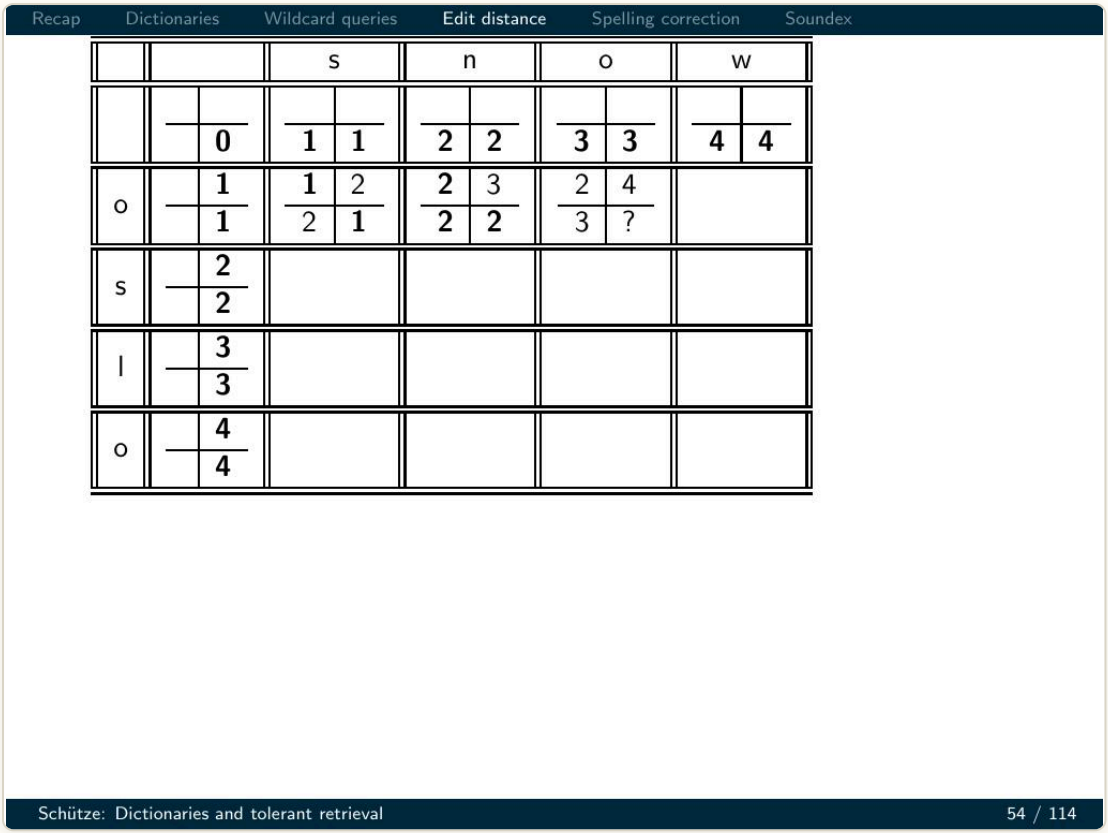


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

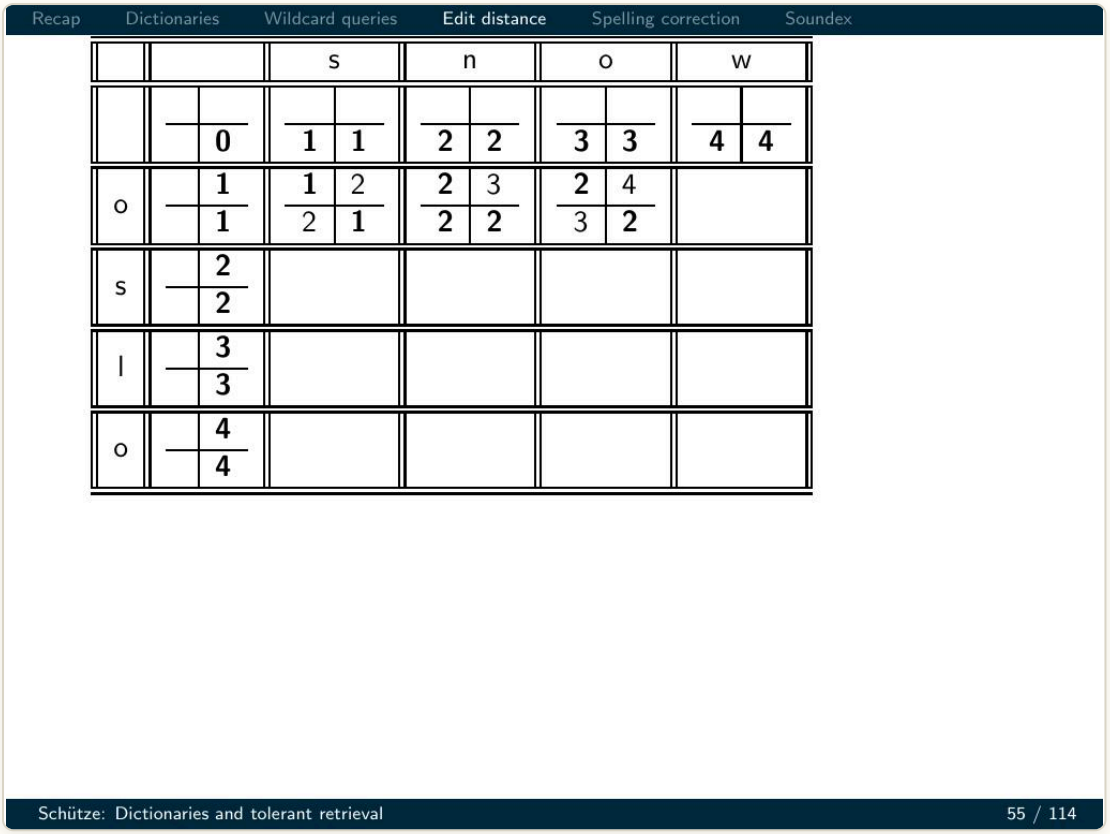


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

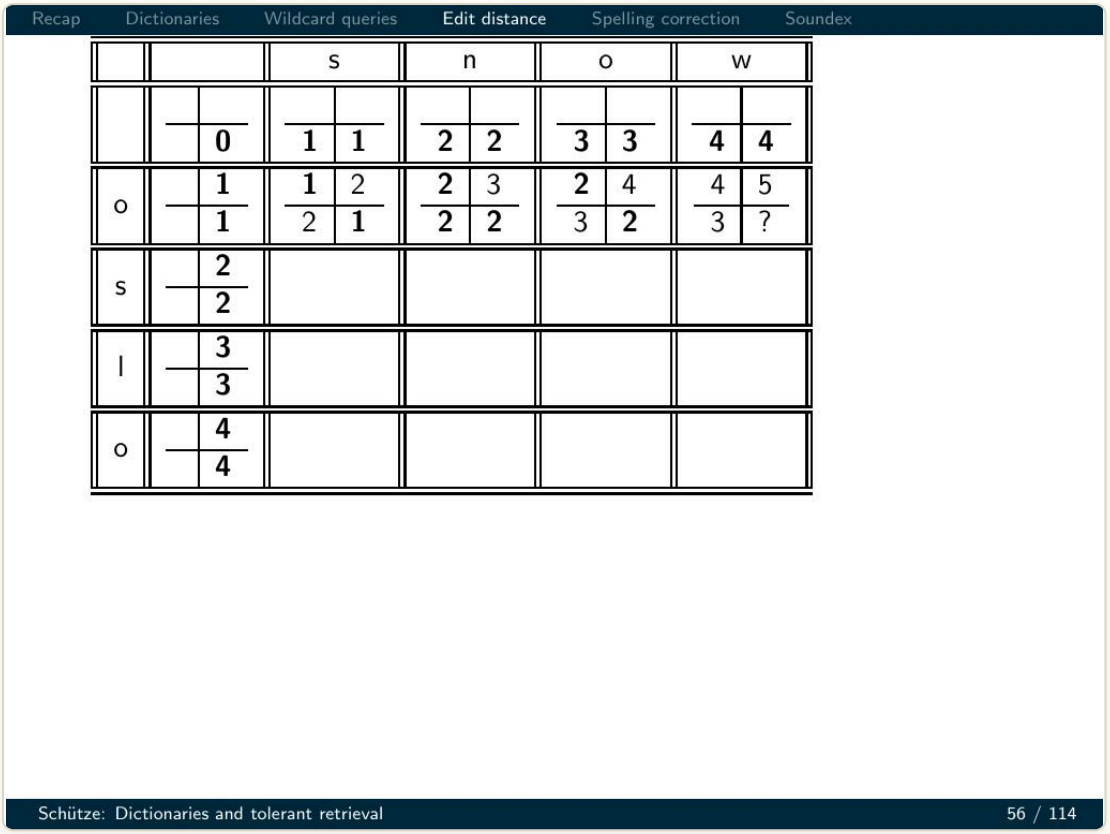


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

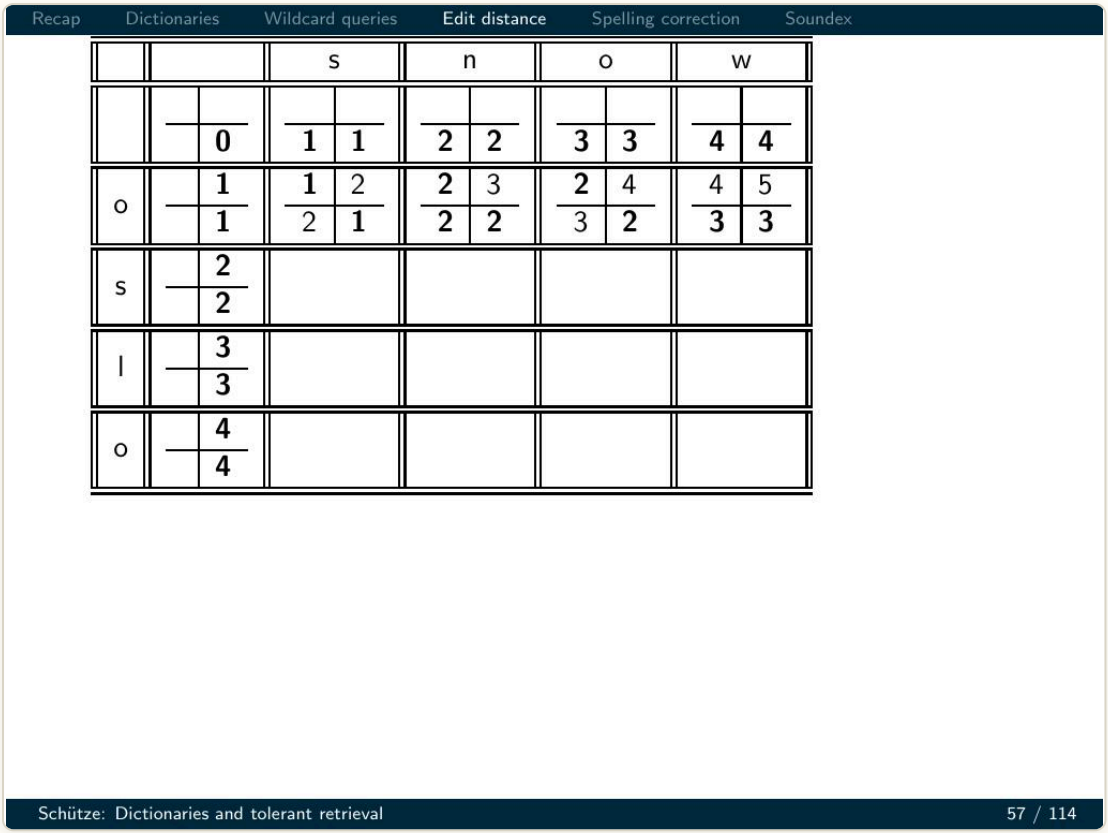


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

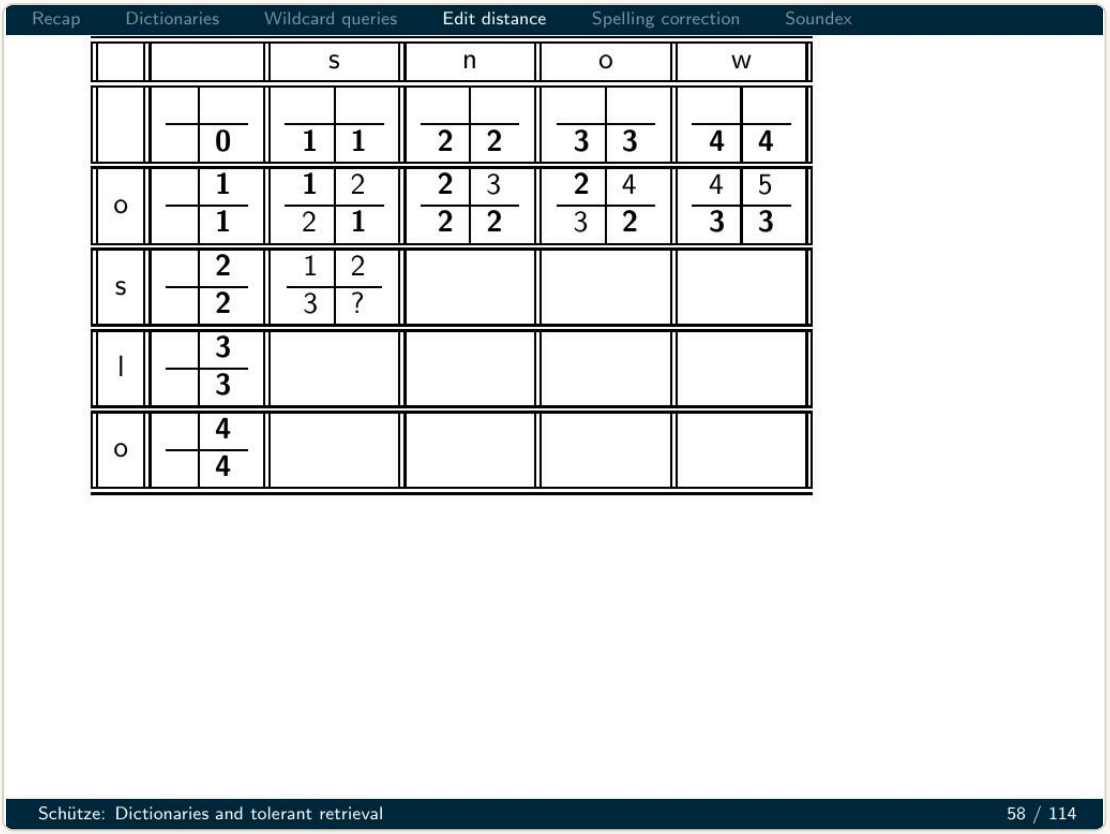


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

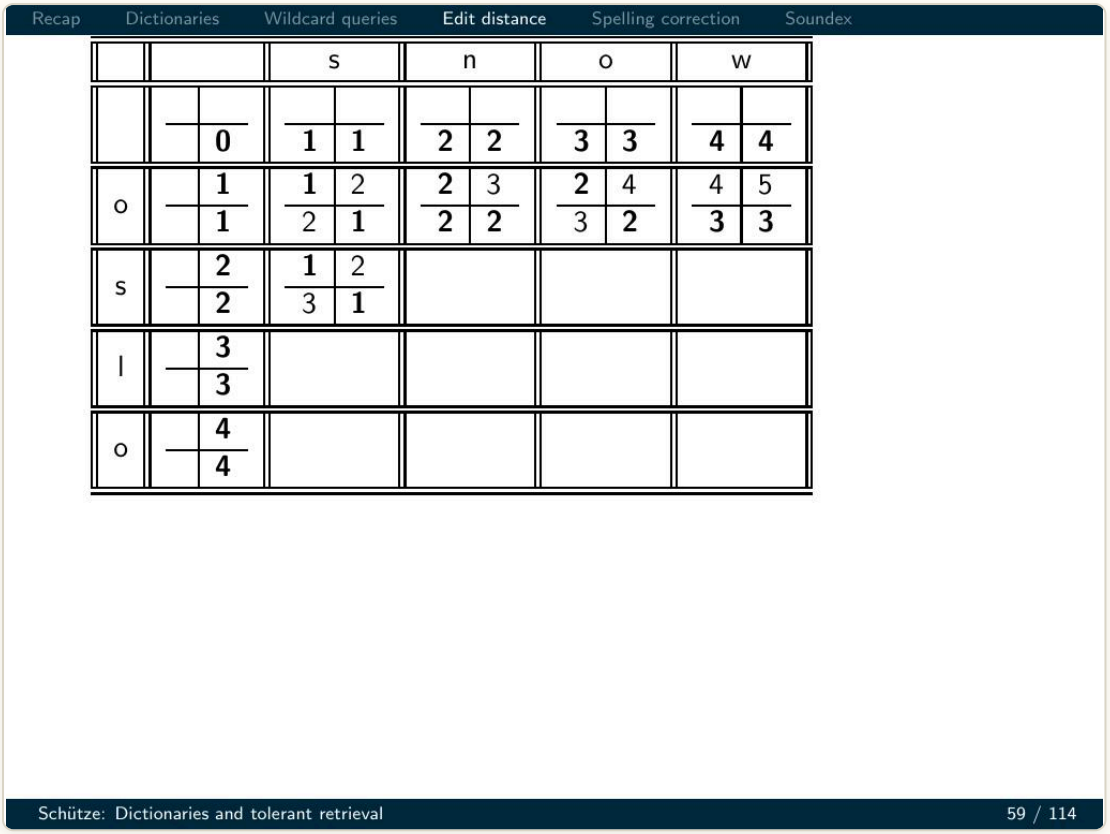


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

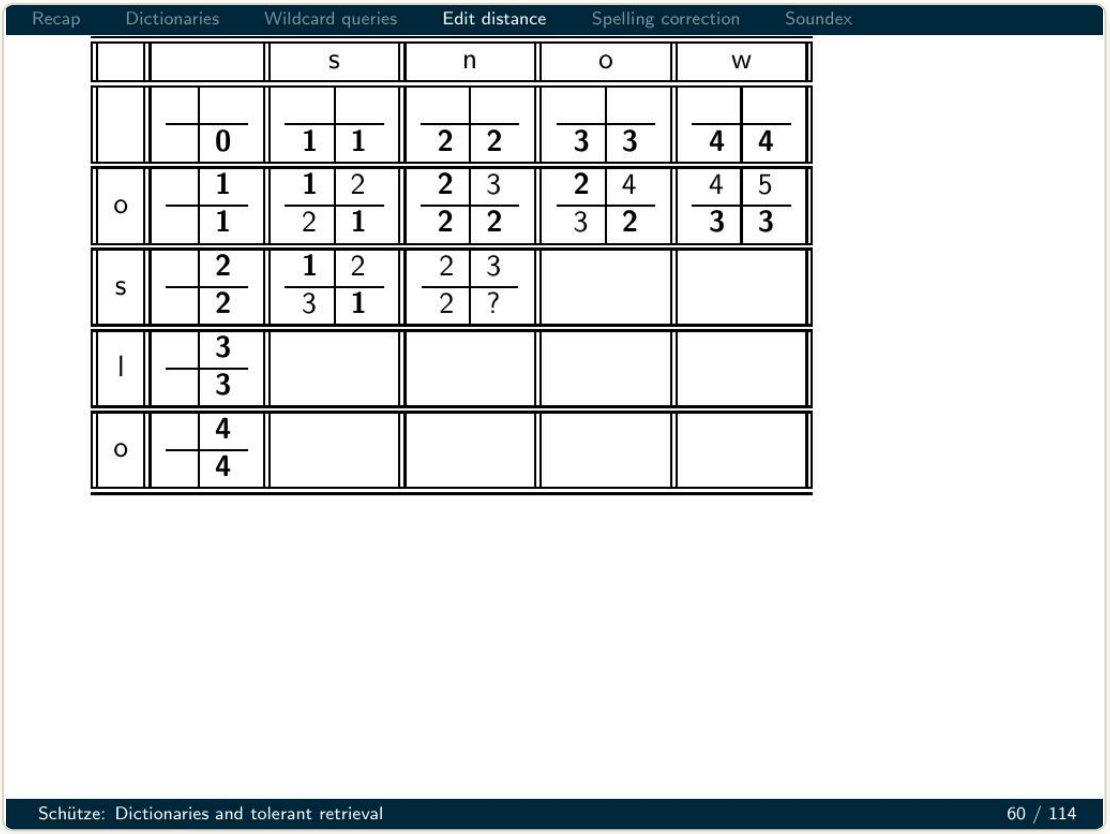


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

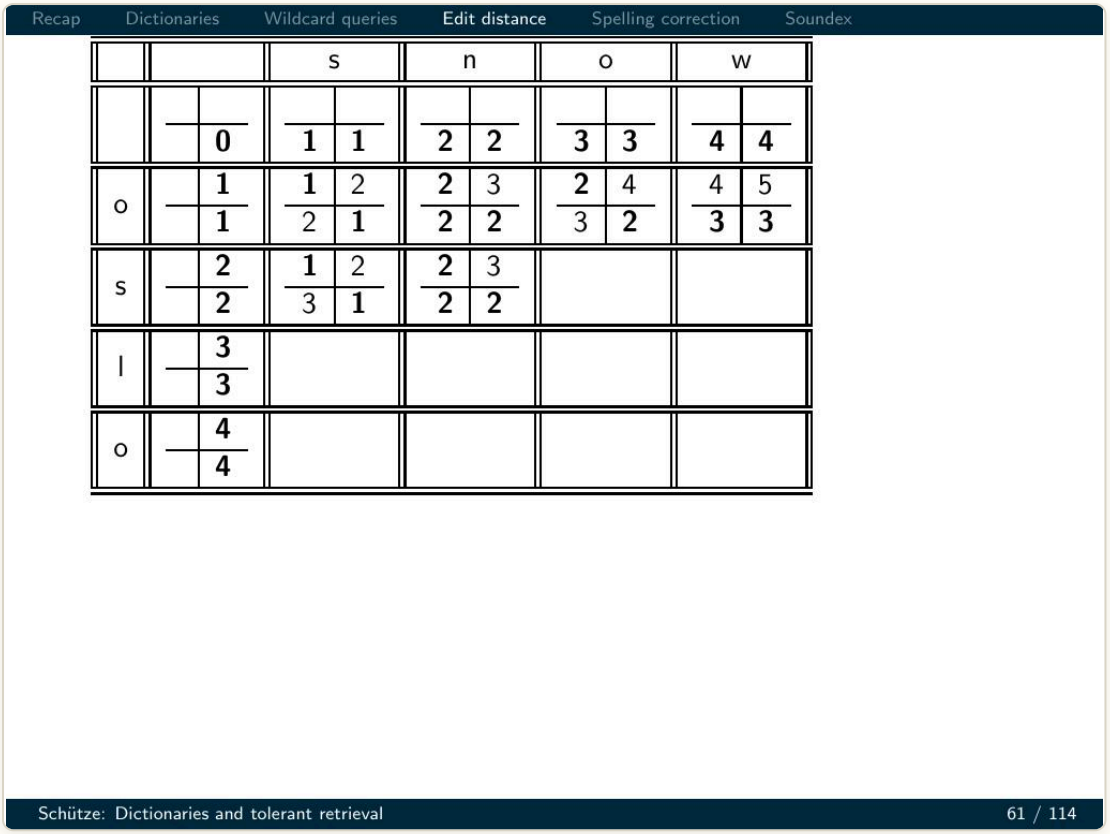


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

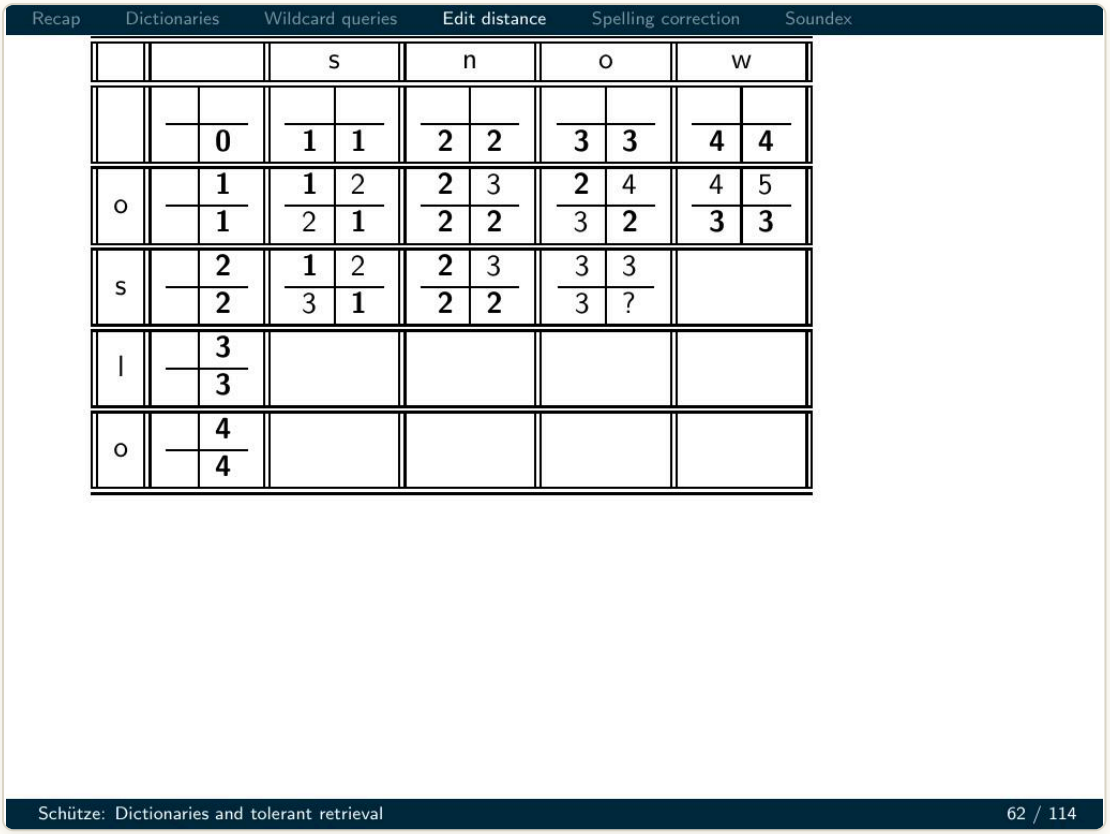


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

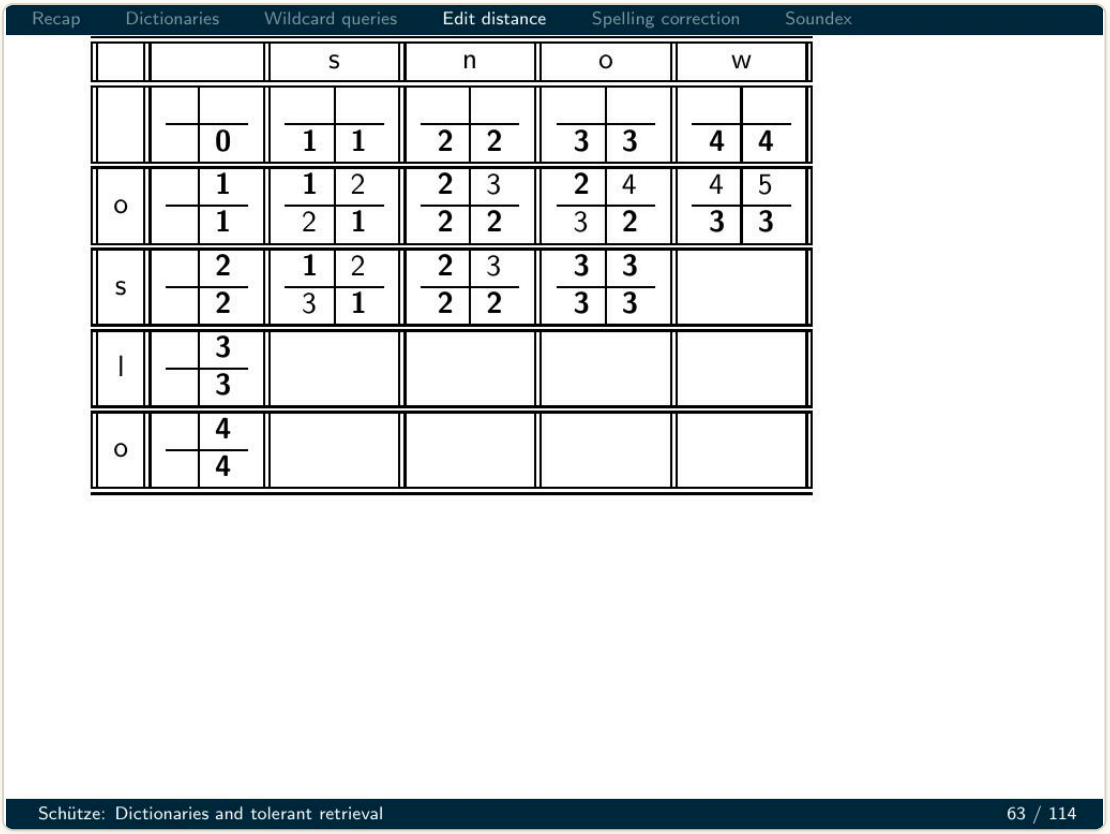


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽



연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

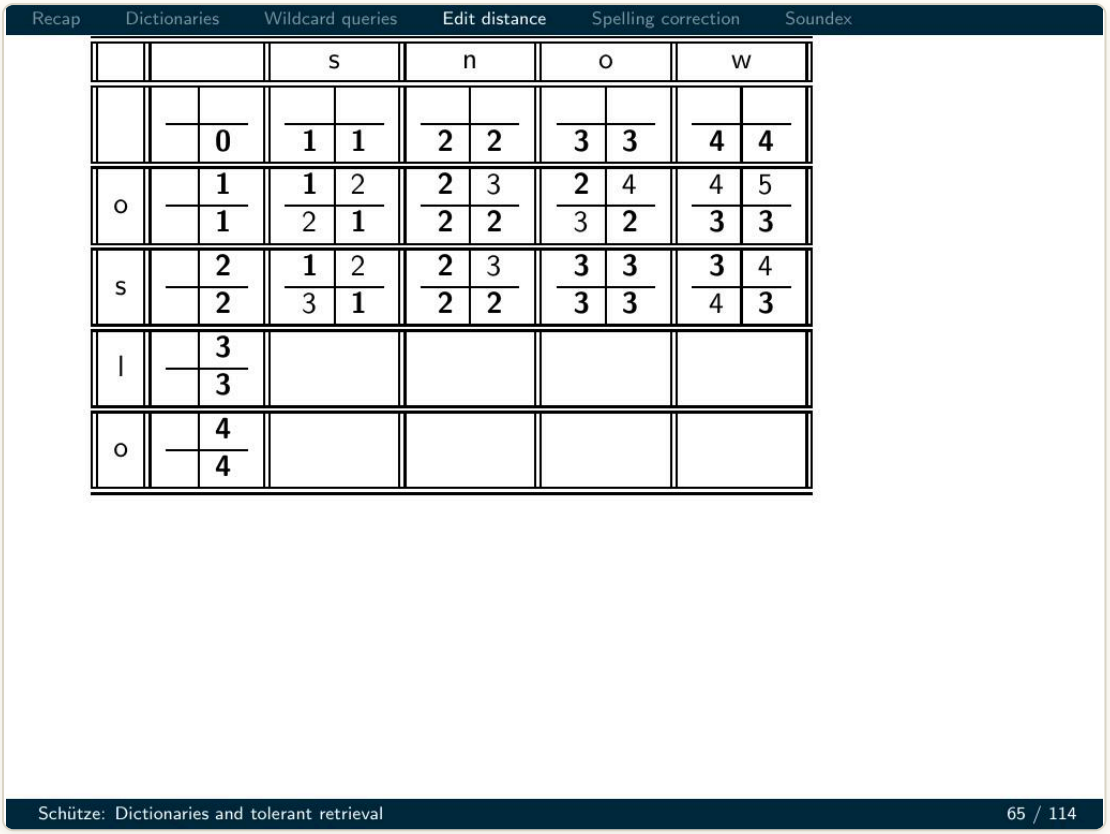
		s		n		o		w	
	0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1** / **1**	**1** / 2	2 / **1**	**2** / **2**	3 / **2**	**2** / 3	4 / **2**	4 / **3**	5 / **3**
s	**2** / **2**	**1** / 3	2 / **1**	**2** / **2**	3 / **2**	**3** / **3**	3 / **3**	3 / 4	4 / ?
l	**3** / **3**								
o	**4** / **4**								

연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

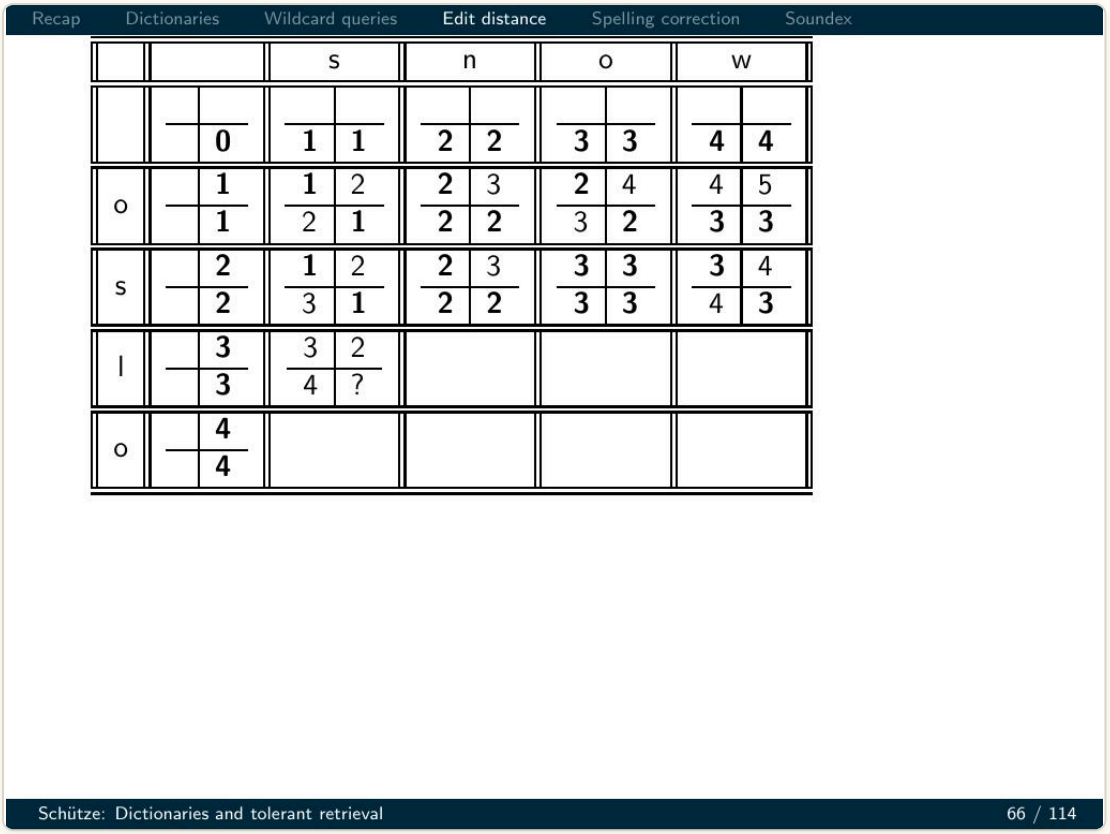


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

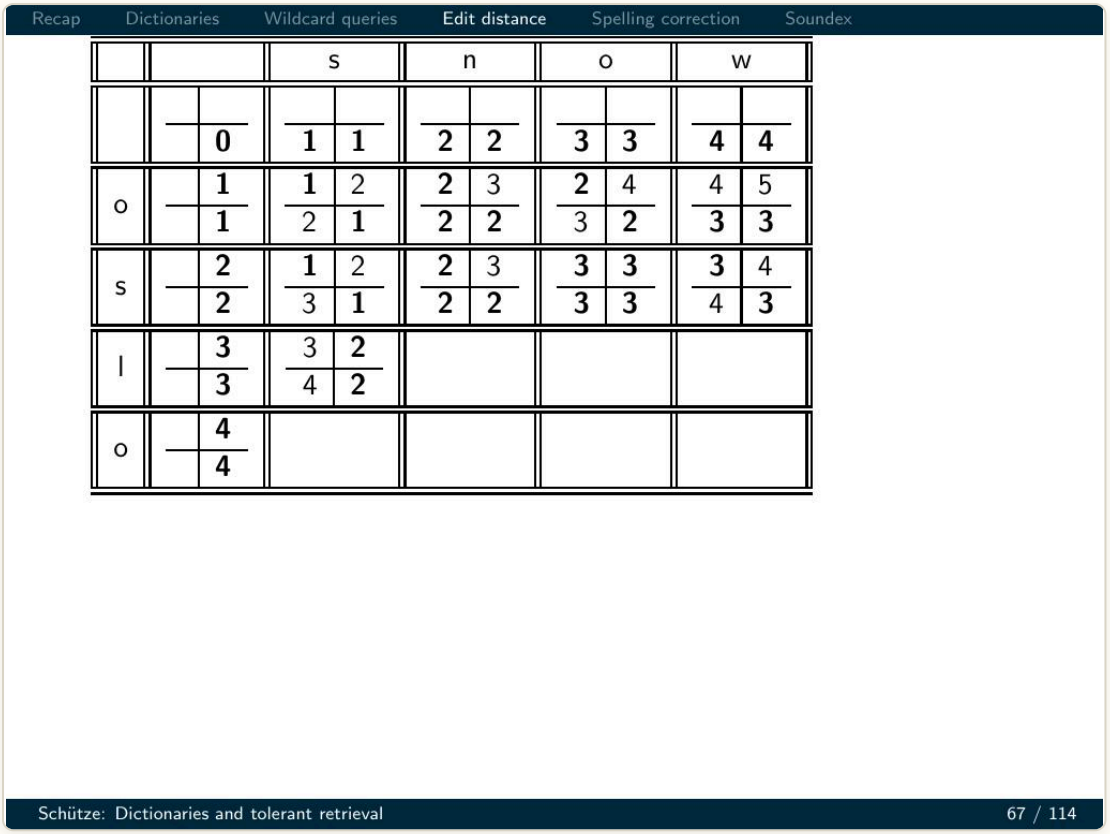


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

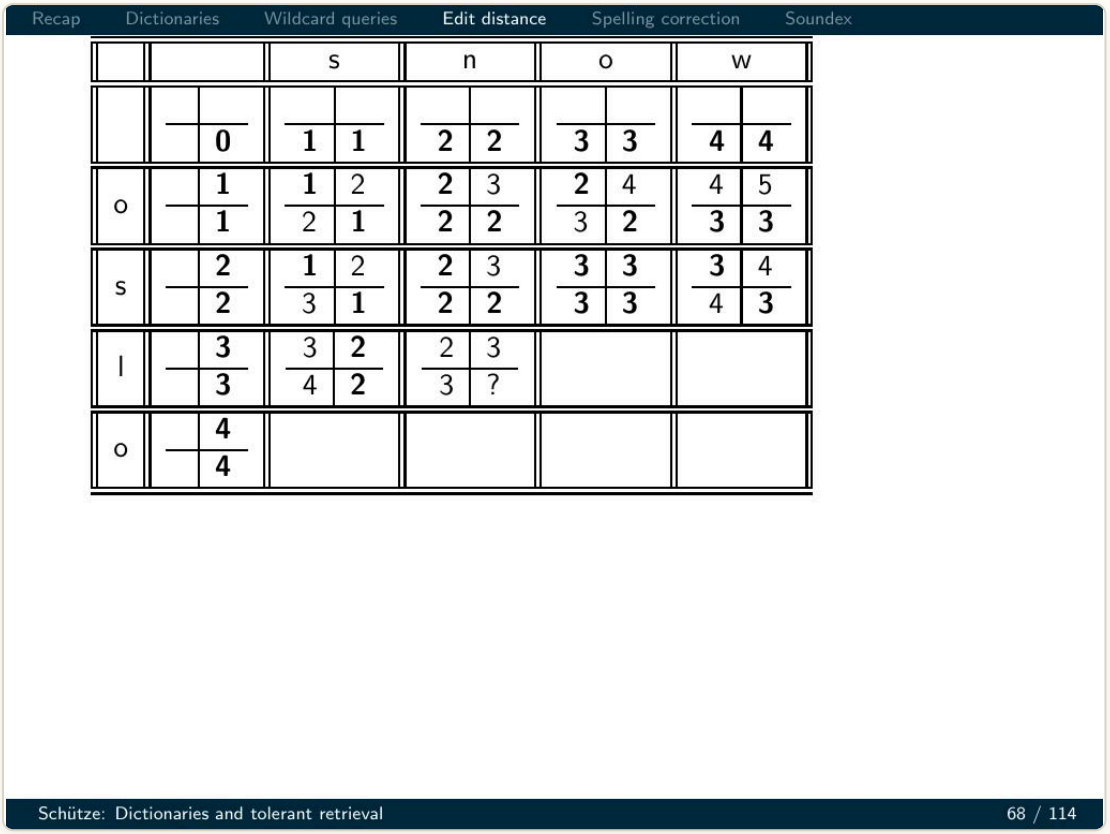


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

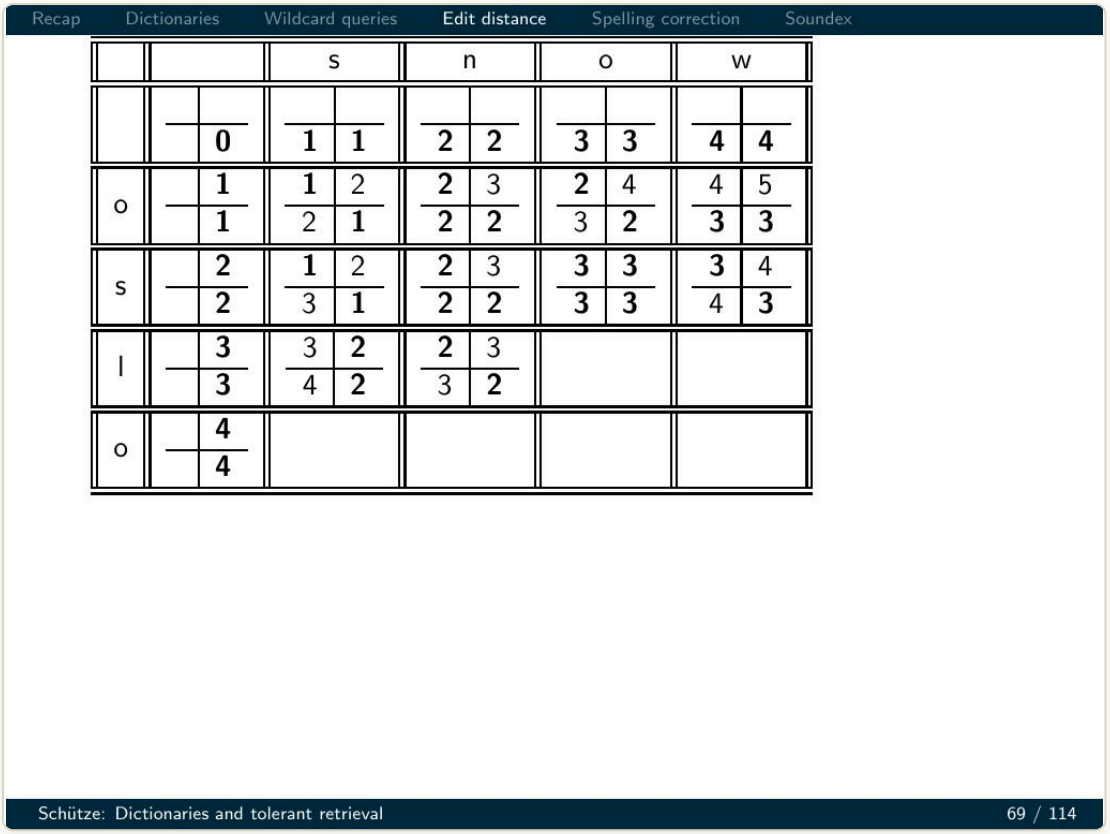


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

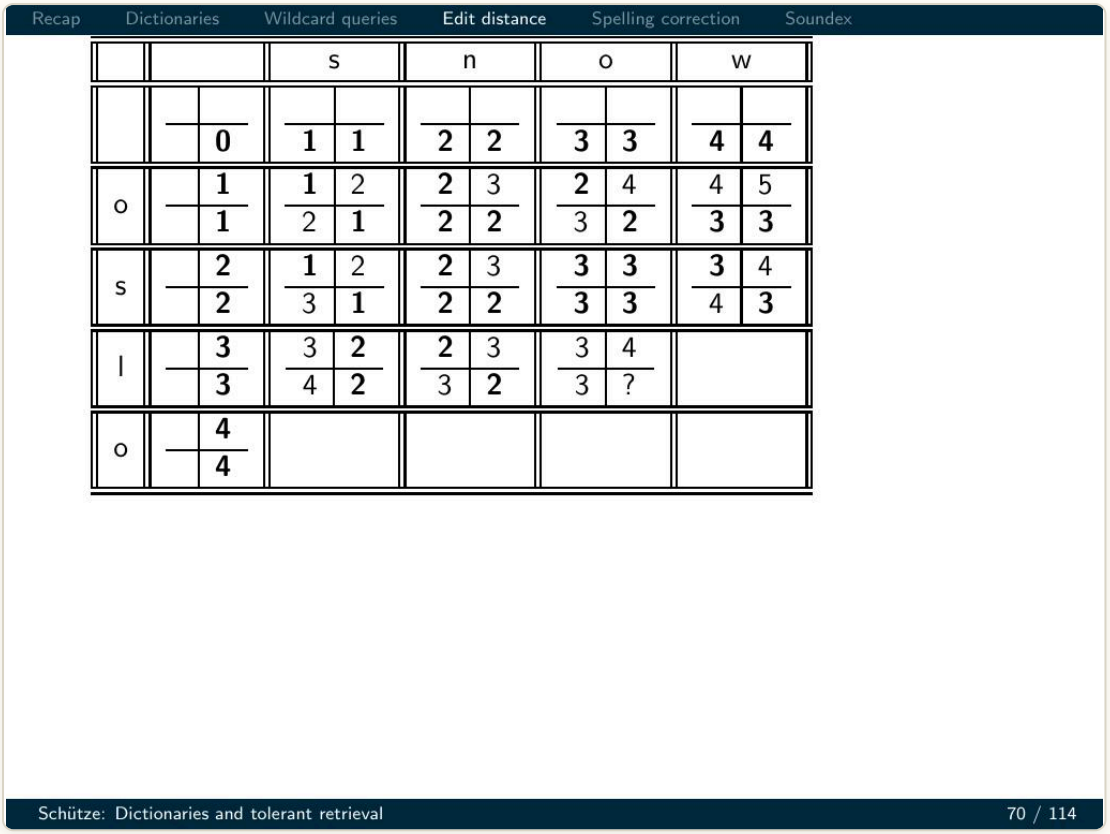


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

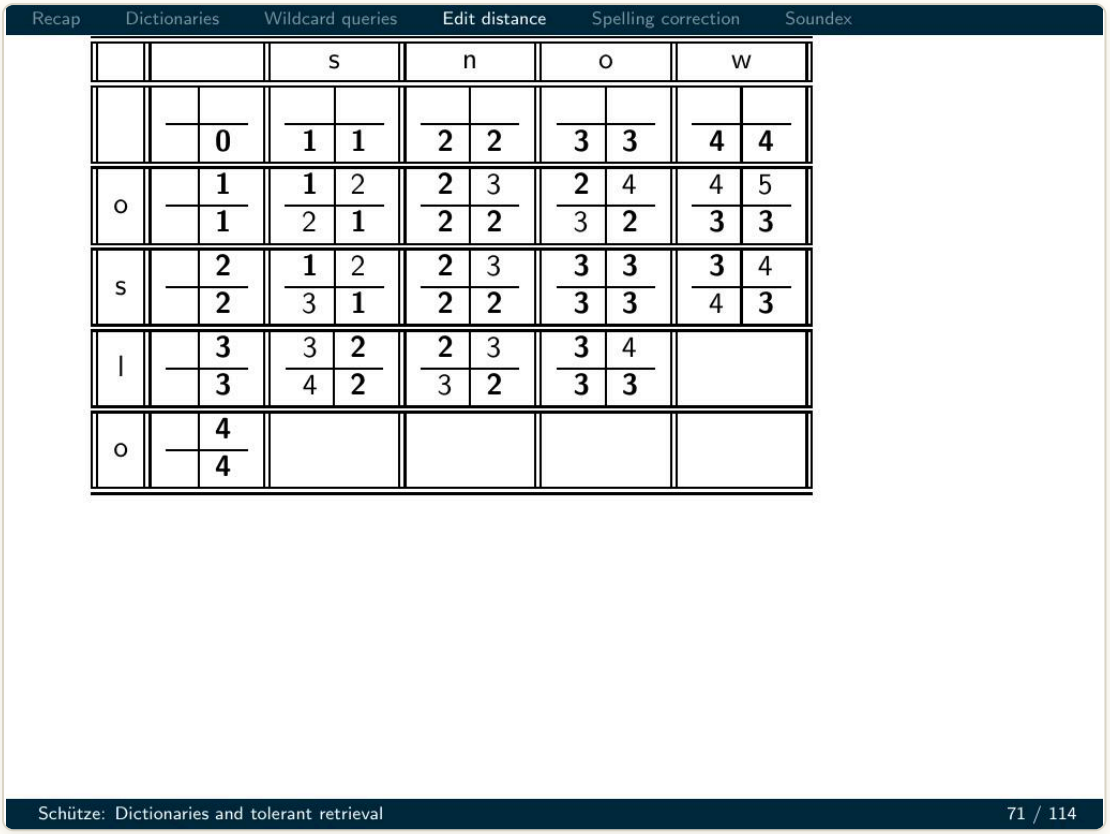


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

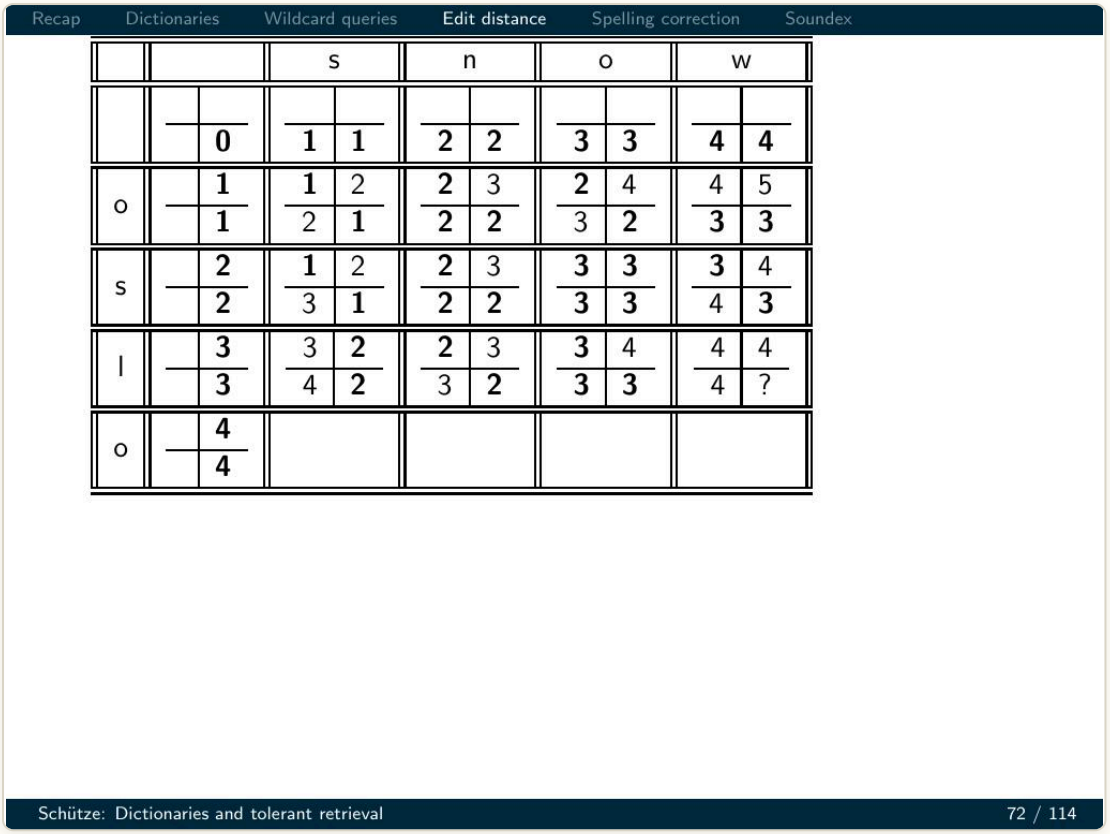


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

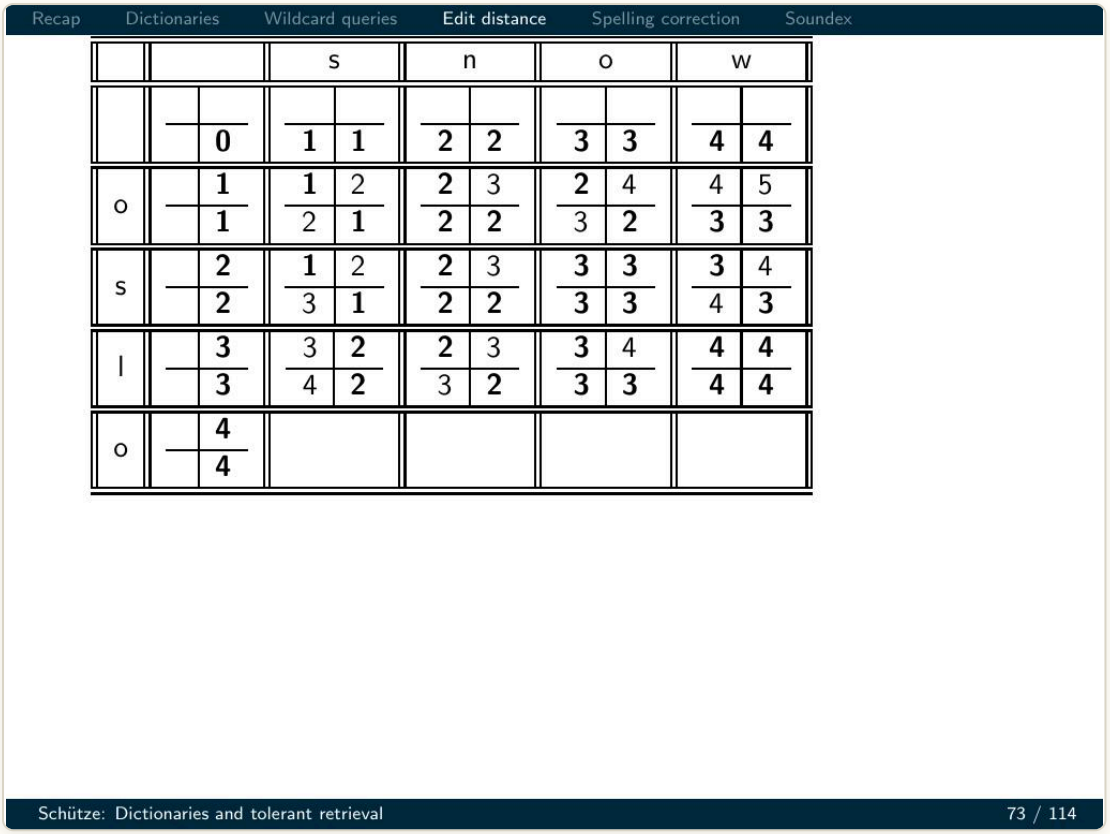


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

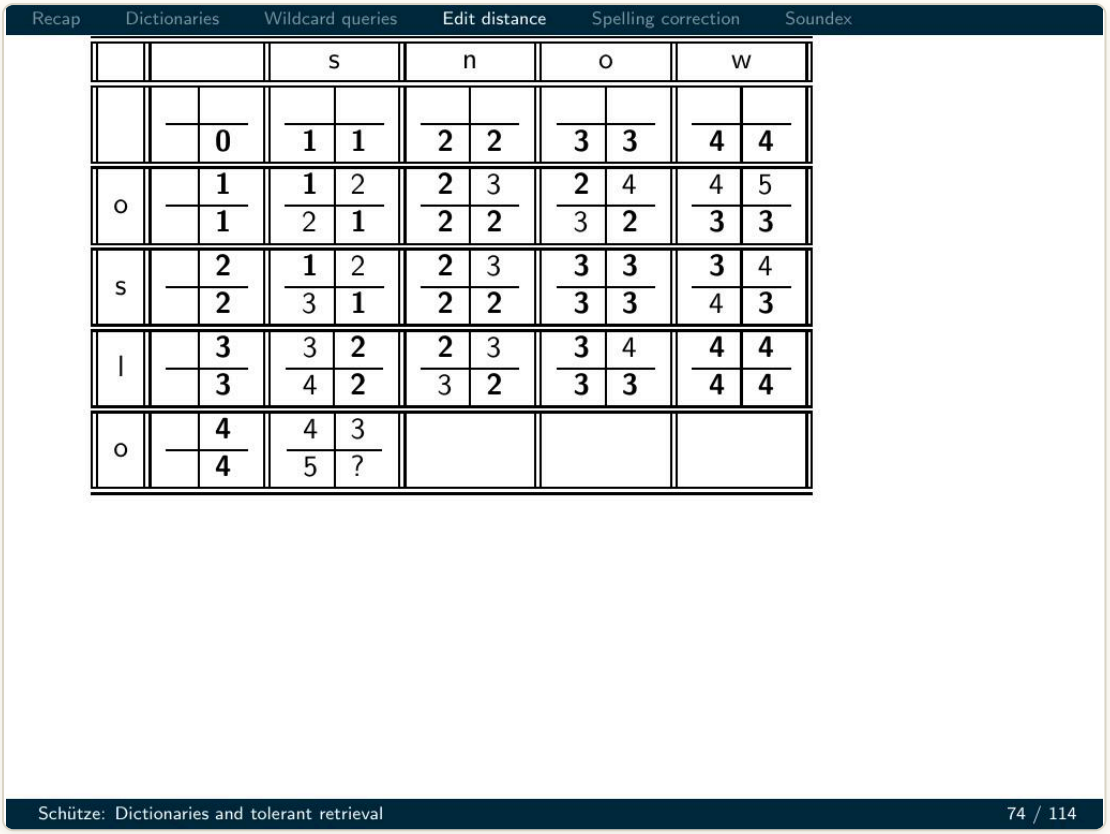


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽



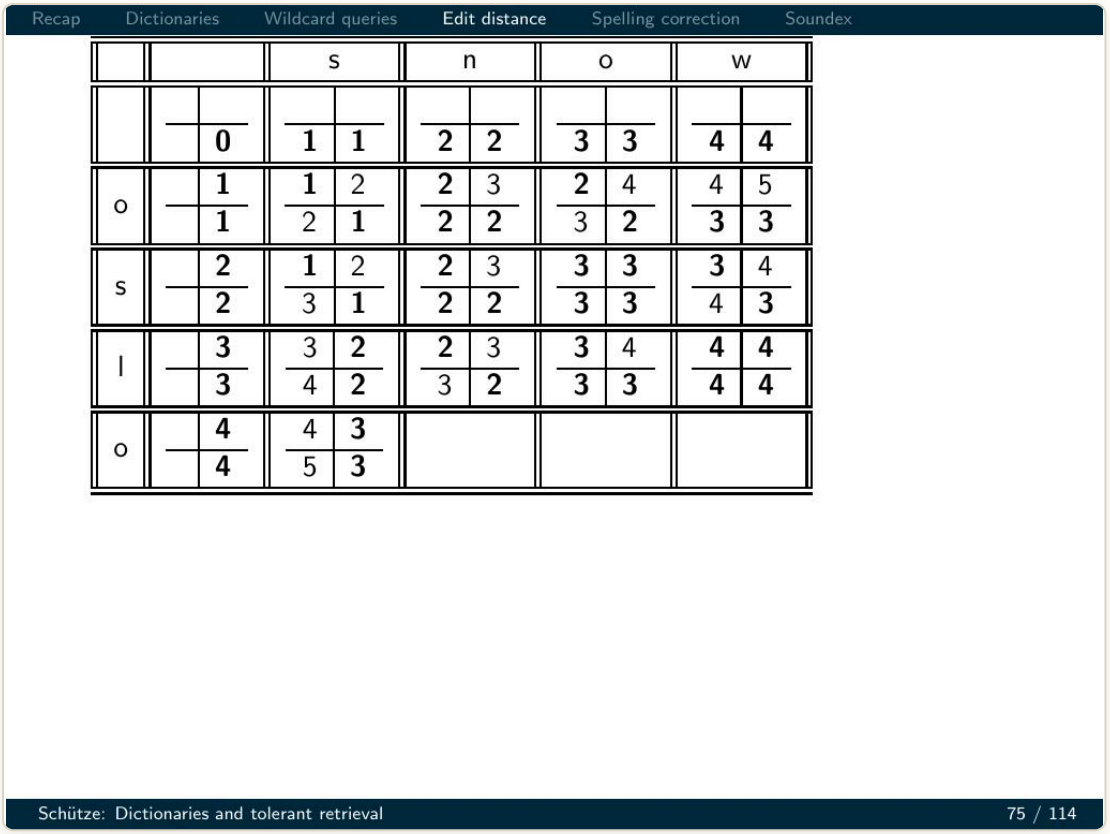
		s		n		o		w	
	0		**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	**1**	2	**2**	3	**2**	4	4	5
	1		2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
	2		3	**1**	**2**	**2**	**3**	**3**	4	**3**
l	**3**	**3**	3	**2**	**2**	3	**3**	4	**4**	**4**
	3		4	**2**	3	**2**	**3**	**3**	**4**	**4**
o	**4**	**4**	4	3						
	4		5	?						

연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

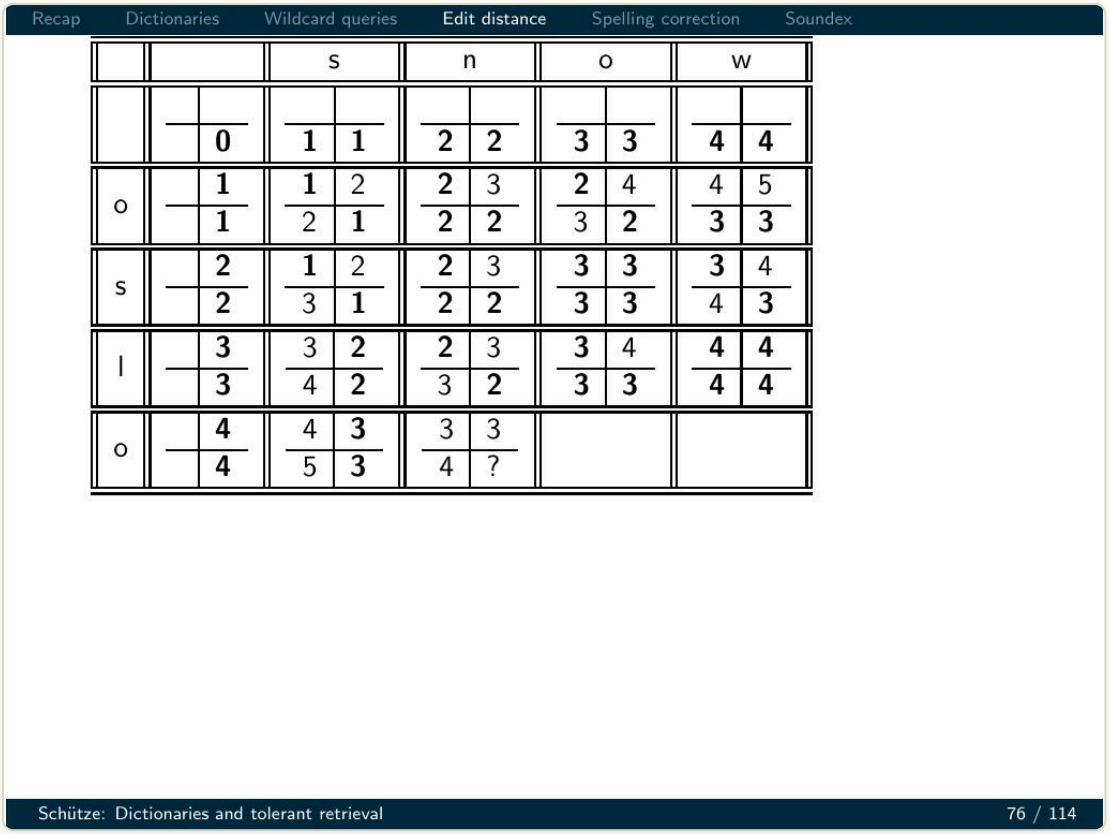


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽



			s		n		o		w	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	**1**	2	**2**	3	**2**	4	4	5
		1	2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
		3	**1**		**2**	**2**	**3**	**3**	4	**3**
l	**3**	**3**	3	**2**	**2**	3	**3**	4	**4**	**4**
		4	**2**		3	**2**	**3**	**3**	**4**	**4**
o	**4**	**4**	4	**3**	3	3				
		5	**3**		4	?				

연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

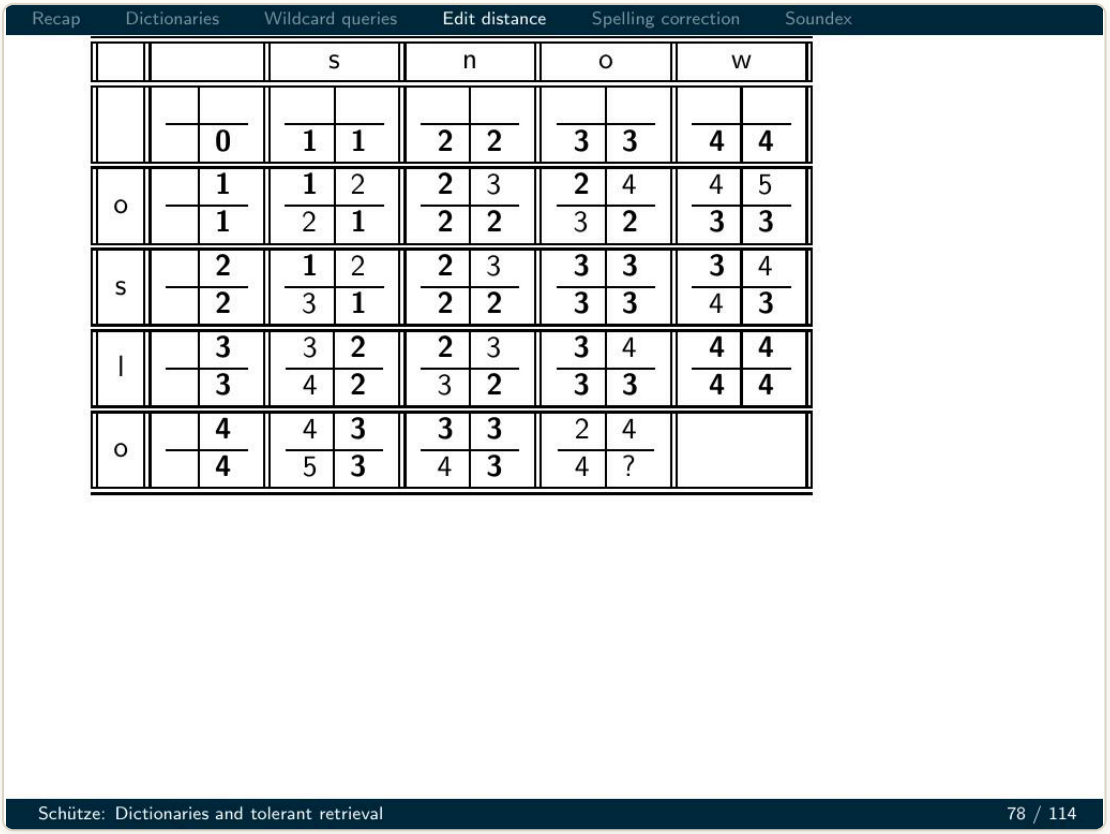
			s		n		o		w	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	**1**	2	**2**	3	**2**	4	4	5
	1	**1**	2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
	2	**2**	3	**1**	**2**	**2**	**3**	**3**	4	**3**
l	**3**	**3**	3	**2**	**2**	3	**3**	4	**4**	**4**
	3	**3**	4	**2**	3	**2**	**3**	**3**	**4**	**4**
o	**4**	**4**	4	**3**	**3**	**3**				
	4	**4**	5	**3**	4	**3**				

연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

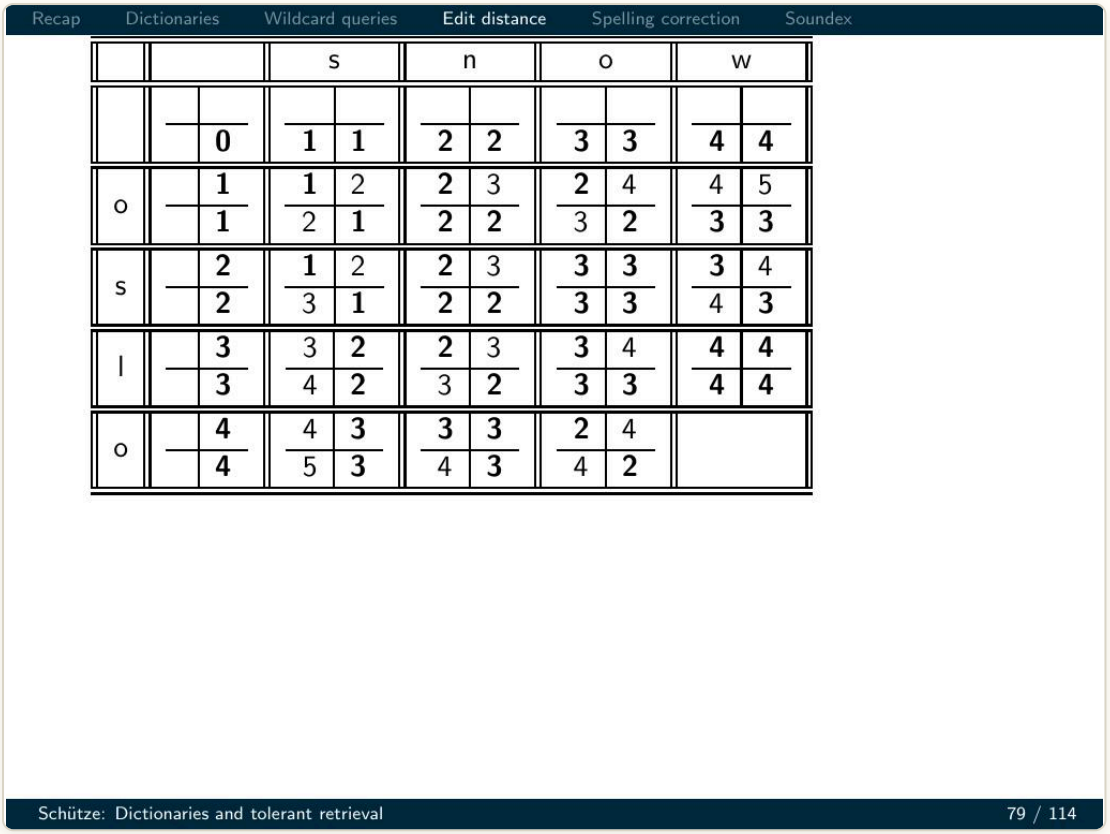


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽

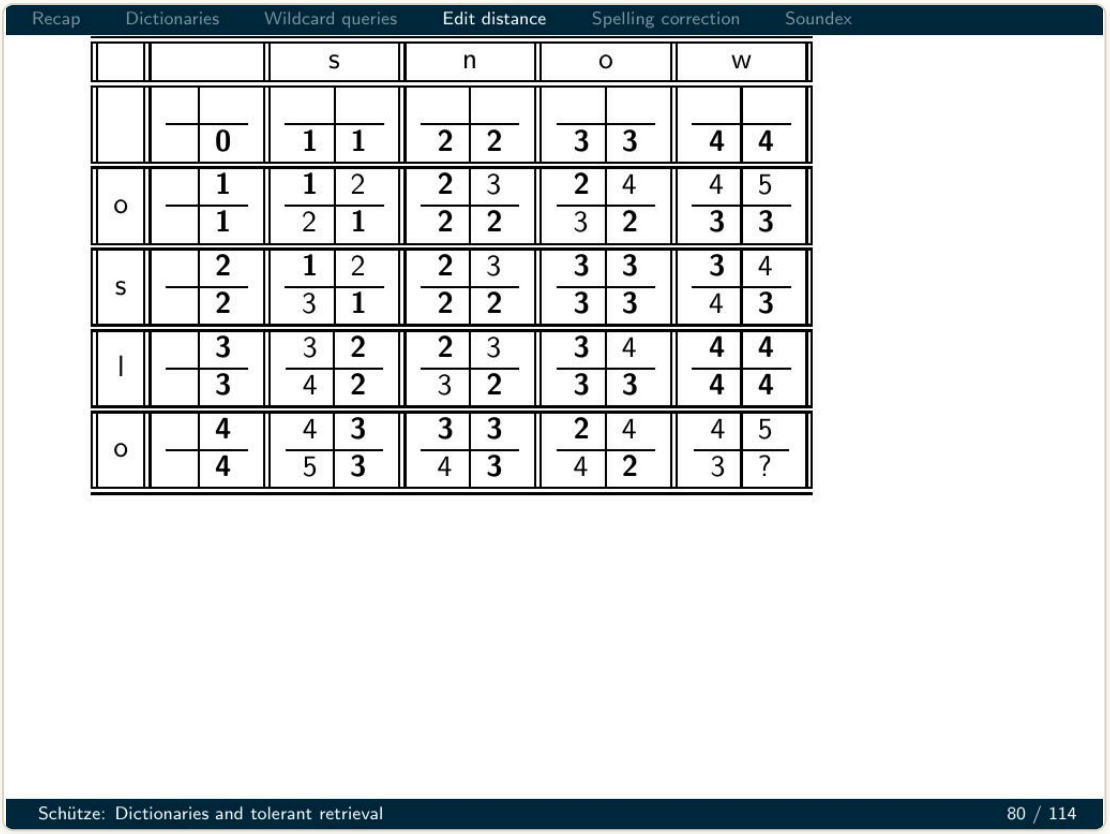


연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽



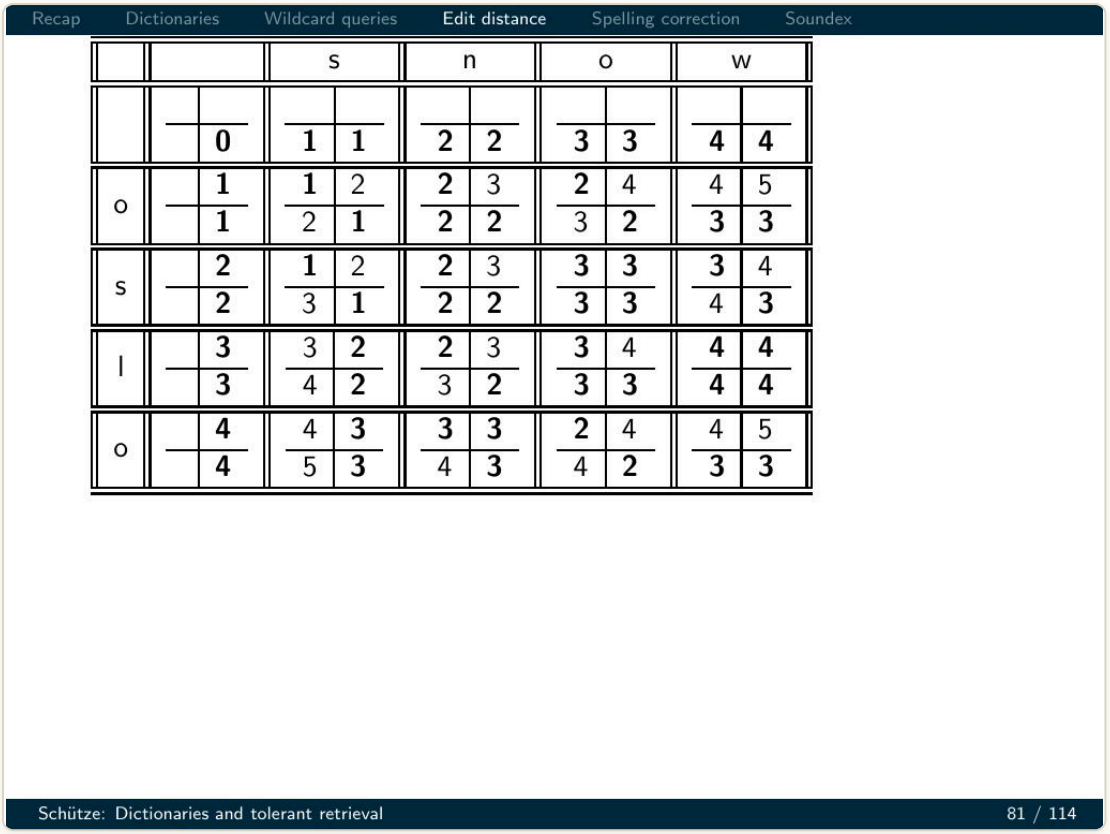
			s		n		o		w	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	**1**	2	**2**	3	**2**	4	4	5
		1	2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
		2	3	**1**	**2**	**2**	**3**	**3**	4	**3**
l	**3**	**3**	3	**2**	**2**	3	**3**	4	**4**	**4**
		3	4	**2**	3	**2**	**3**	**3**	**4**	**4**
o	**4**	**4**	4	**3**	**3**	**3**	**2**	4	4	5
		4	5	**3**	4	**3**	4	**2**	3	?

연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽



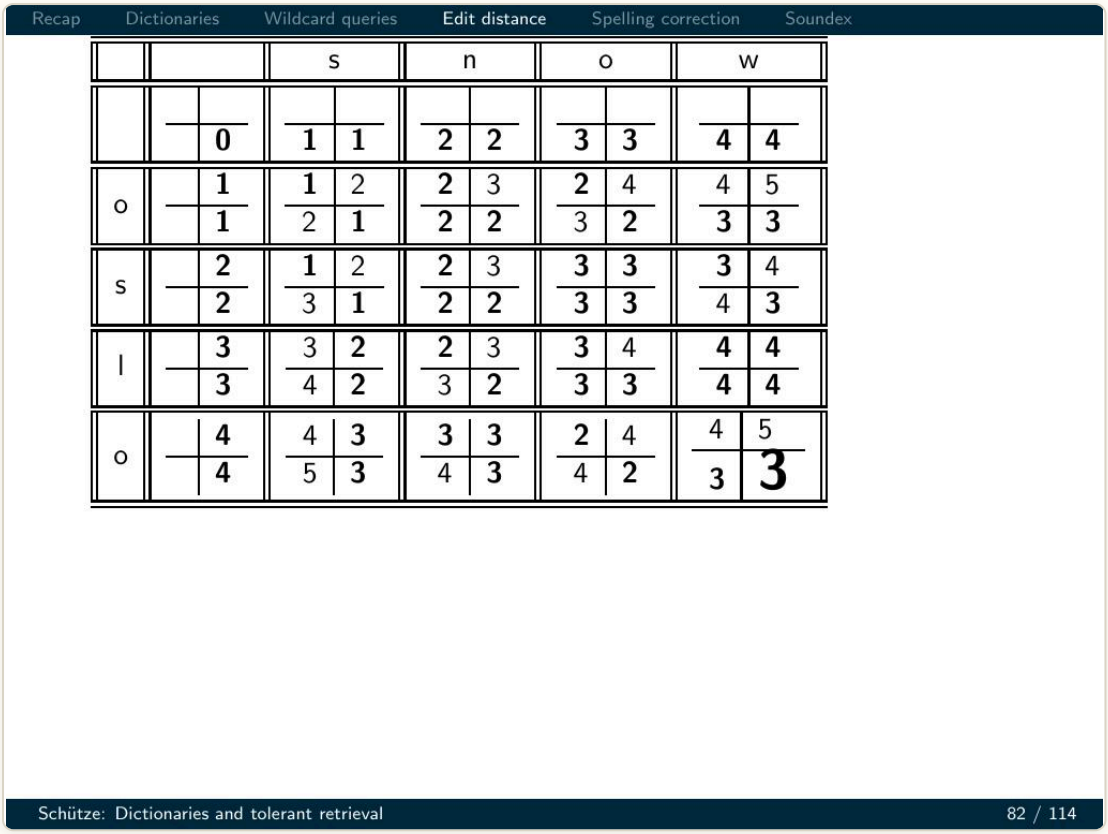
			s		n		o		w	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	**1**	2	**2**	3	**2**	4	4	5
	2	**1**	2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
	3	**1**	3	**1**	**2**	**2**	**3**	**3**	4	**3**
l	**3**	**3**	3	**2**	**2**	3	**3**	4	**4**	**4**
	4	**2**	4	**2**	3	**2**	**3**	**3**	**4**	**4**
o	**4**	**4**	4	**3**	**3**	**3**	**2**	4	4	5
	5	**3**	5	**3**	4	**3**	4	**2**	**3**	**3**

연습: OSLO → SNOW 행렬 채우기

- 표를 왼쪽 위부터 한 칸씩 채워 나가는 과정 (애니메이션)
- 칸마다 위(+1, 삭제) · 왼쪽(+1, 삽입) · 대각선(+0 복사 / +1 교체) 중 **최소**

✍ 메모

— 넘어가도 되는 쪽



		s	n	o	w
	0	**1** **1**	**2** **2**	**3** **3**	**4** **4**
o	**1** **1**	**1** 2 / 2 **1**	**2** 3 / **2** **2**	**2** 4 / 3 **2**	4 5 / **3** **3**
s	**2** **2**	**1** 2 / 3 **1**	**2** 3 / **2** **2**	**3** **3** / **3** **3**	**3** 4 / 4 **3**
l	**3** **3**	3 **2** / 4 **2**	**2** 3 / 3 **2**	**3** 4 / **3** **3**	**4** **4** / **4** **4**
o	**4** **4**	4 **3** / 5 **3**	**3** **3** / 4 **3**	**2** 4 / 4 **2**	4 5 / **3** **3**

OSLO → SNOW: 완성된 행렬

- o: 1 1 2 3 4 → ... 마지막 행 o: 4 3 3 2 **3**
- → oslo와 snow의 레벤슈타인 거리 = **3**

✍ 메모

★ 시험 답 3

			s		n		o		w	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	1	2	2	3	2	4	4	5
	1	2	**1**	**2**	**2**	3	**2**	4	**3**	**3**
s	**2**	**2**	1	2	2	3	3	3	3	4
	2	3	**1**	**2**	**2**	3	**3**	3	4	**3**
l	**3**	**3**	3	2	2	3	3	4	4	4
	3	4	**2**	3	**2**		**3**	3	**4**	**4**
o	**4**	**4**	4	3	3	3	2	4	4	5
	4	5	**3**	4	**3**		4	**2**	**3**	**3**

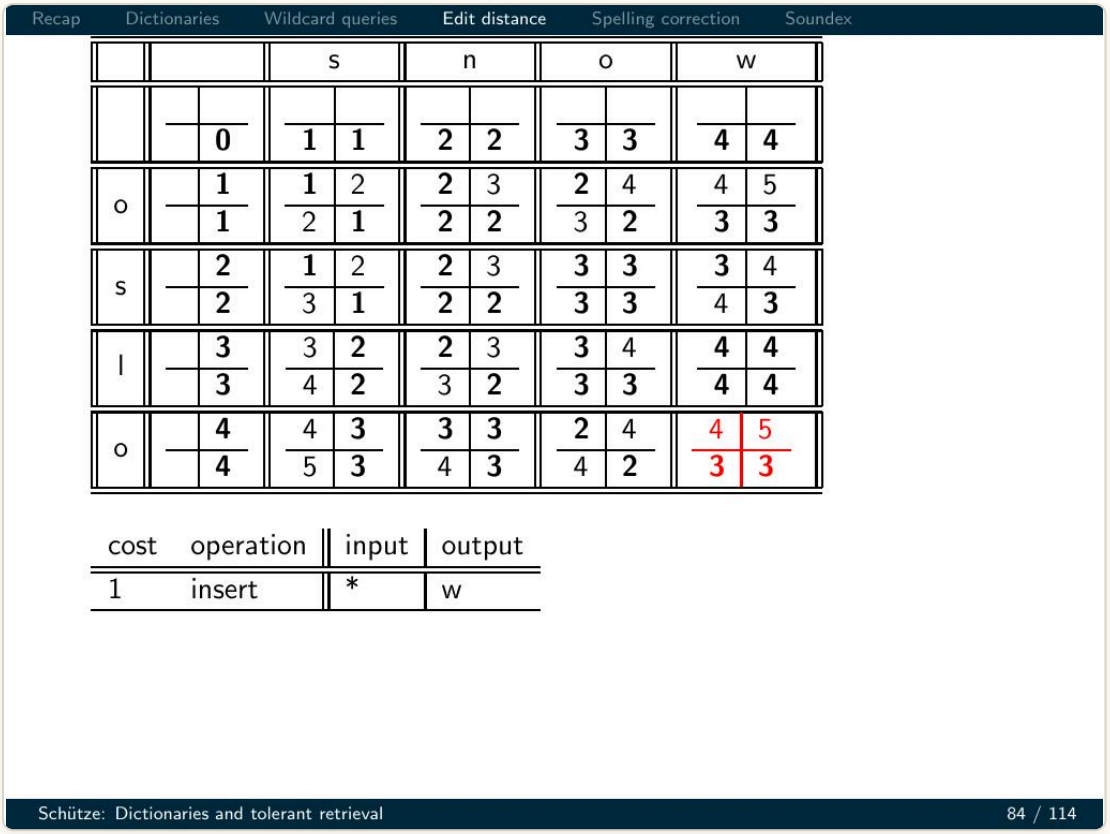
How do I read out the editing operations that transform OSLO into SNOW?

OSLO → SNOW: 편집 연산 읽어 내기

- 완성된 행렬
- oslo를 snow로 바꾸는 편집 연산을 어떻게 읽어 낼까?

✍️ 메모

💡 이해 오른쪽 아래에서 출발해 '최소값을 만든 이웃'으로 거슬러 올라간다 (backtrace)

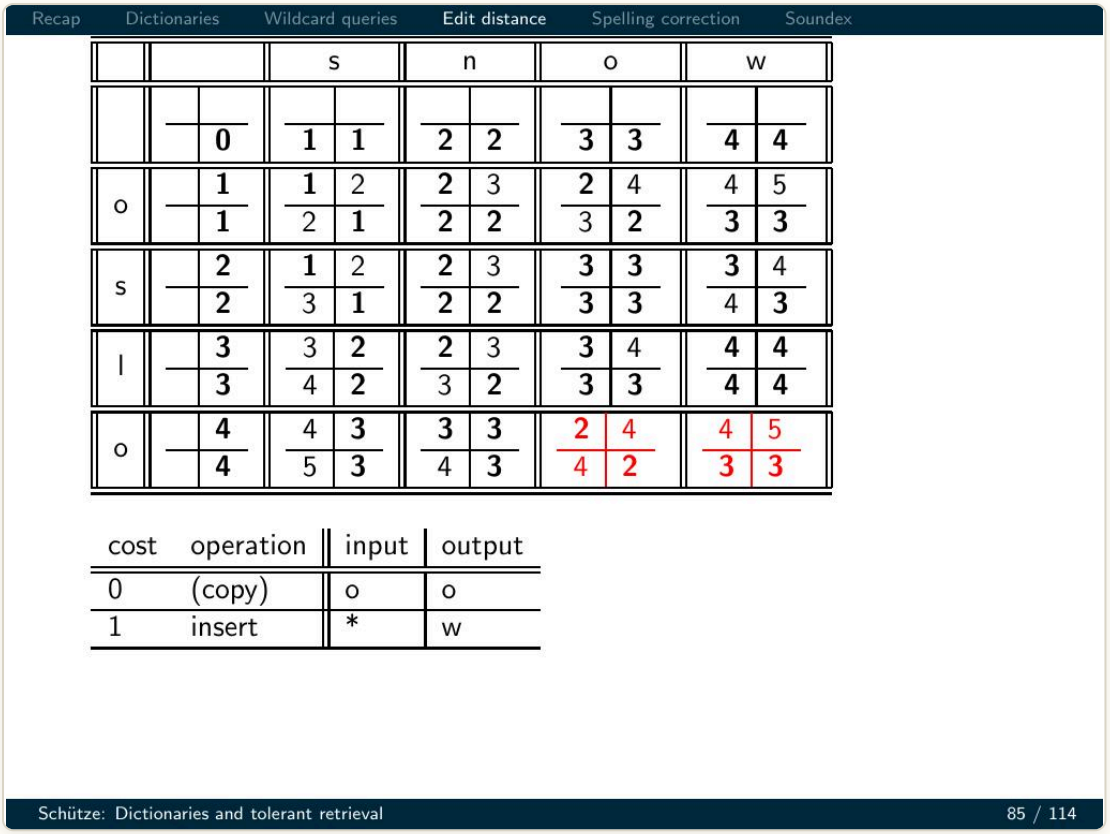


OSLO → SNOW: 편집 연산 읽어 내기

- 오른쪽 아래 칸(3)에서 시작해, 그 값을 만든 이웃 칸으로 **거꾸로** 따라간다
- 대각선 = 복사(0) 또는 교체(1), 위 = 삭제, 왼쪽 = 삽입

✍ 메모

— 넘어가도 되는 쪽



OSLO → SNOW: 편집 연산 읽어 내기

- 오른쪽 아래 칸(3)에서 시작해, 그 값을 만든 이웃 칸으로 **거꾸로** 따라간다
- 대각선 = 복사(0) 또는 교체(1), 위 = 삭제, 왼쪽 = 삽입

✍ 메모

— 넘어가도 되는 쪽

		s		n		o		w	
	0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	2	**2**	3	**2**	4	4	5
	1	2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
	2	3	**1**	**2**	**2**	**3**	**3**	4	**3**
l	**3**	3	**2**	2	3	**3**	4	**4**	**4**
	3	4	**2**	3	**2**	**3**	**3**	**4**	**4**
o	**4**	4	**3**	**3**	**3**	2	4	4	5
	4	5	**3**	4	**3**	4	**2**	**3**	**3**

cost	operation	input	output
1	replace	l	n
0	(copy)	o	o
1	insert	*	w

OSLO → SNOW: 편집 연산 읽어 내기

- 오른쪽 아래 칸(3)에서 시작해, 그 값을 만든 이웃 칸으로 **거꾸로** 따라간다
- 대각선 = 복사(0) 또는 교체(1), 위 = 삭제, 왼쪽 = 삽입

✍ 메모

— 넘어가도 되는 쪽

		s		n		o		w	
	0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
o	**1**	**1**	2	**2**	3	**2**	4	4	5
	1	2	**1**	**2**	**2**	3	**2**	**3**	**3**
s	**2**	**1**	2	**2**	3	**3**	**3**	**3**	4
	2	3	**1**	**2**	**2**	**3**	**3**	4	**3**
l	**3**	3	**2**	**2**	3	**3**	4	**4**	**4**
	3	4	**2**	3	**2**	**3**	**3**	**4**	**4**
o	**4**	4	**3**	**3**	**3**	2	4	4	5
	4	5	**3**	4	**3**	4	**2**	**3**	**3**

cost	operation	input	output
0	(copy)	s	s
1	replace	l	n
0	(copy)	o	o
1	insert	*	w

OSLO → SNOW: 편집 연산 읽어 내기

- 오른쪽 아래 칸(3)에서 시작해, 그 값을 만든 이웃 칸으로 **거꾸로** 따라간다
- 대각선 = 복사(0) 또는 교체(1), 위 = 삭제, 왼쪽 = 삽입

✍ 메모

— 넘어가도 되는 쪽

		s	n	o	w
	0	1 1	2 2	3 3	4 4
o	1 / 1	1 2 / 2 1	2 3 / 2 2	2 4 / 3 2	4 5 / 3 3
s	2 / 2	1 2 / 3 1	2 3 / 2 2	3 3 / 3 3	3 4 / 4 3
l	3 / 3	3 2 / 4 2	2 3 / 3 2	3 4 / 3 3	4 4 / 4 4
o	4 / 4	4 3 / 5 3	3 3 / 4 3	2 4 / 4 2	4 5 / 3 3

cost	operation	input	output
1	delete	o	*
0	(copy)	s	s
1	replace	l	n
0	(copy)	o	o
1	insert	*	w

OSLO → SNOW: 편집 연산 (결과)

- 빨간 경로를 따라 읽은 연산
  - 비용 1 **삭제** o → *
  - 비용 0 (복사) s → s
  - 비용 1 **교체** l → n
  - 비용 0 (복사) o → o
  - 비용 1 **삽입** * → w
- 합계 **3**

✍ 메모

★ 시험 삭제 o, 복사 s, 교체 l→n, 복사 o, 삽입 w = 3

			c		a		t		c		a		t	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**	**5**	**5**	**6**	**6**
c	**1**	**1**	**0**	2	2	3	3	4	**3**	5	5	6	6	7
			2	**0**	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**	**5**	**5**
a	**2**	**2**	2	**1**	**0**	2	2	3	3	4	**3**	5	5	6
			3	**1**	2	**0**	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
t	**3**	**3**	3	**2**	2	**1**	**0**	2	2	3	3	4	**3**	5
			4	**2**	3	**1**	2	**0**	**1**	**1**	**2**	**2**	**3**	**3**

연습: CAT → CATCAT

- cat(행) × catcat(열)의 레벤슈타인 행렬을 채운다
- 최종 거리 = **3**

✍ 메모

— 넘어가도 되는 쪽

			c		a		t		c		a		t	
		0	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**	**5**	**5**	**6**	**6**
c	**1**	**1**	**0**	2	2	3	3	4	**3**	5	5	6	6	7
			2	**0**	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**	**5**	**5**
a	**2**	**2**	2	**1**	**0**	2	2	3	3	4	**3**	5	5	6
			3	**1**	2	**0**	**1**	**1**	**2**	**2**	**3**	**3**	**4**	**4**
t	**3**	**3**	3	**2**	2	**1**	**0**	2	2	3	3	4	**3**	5
			4	**2**	3	**1**	2	**0**	**1**	**1**	**2**	**2**	**3**	**3**

cost	operation	input	output
1	insert	*	c
1	insert	*	a
1	insert	*	t
0	(copy)	c	c
0	(copy)	a	a
0	(copy)	t	t

연습: CAT → CATCAT

- cat(행) × catcat(열)의 레벤슈타인 행렬을 채운다
- 최종 거리 = **3**

✍ 메모

— 넘어가도 되는 쪽

		c	a	t	c	a	t
	0	**1** 1	**2** 2	**3** 3	**4** 4	**5** 5	**6** 6
c	**1** 1	**0** 2 / 2 **0**	2 3 / **1** **1**	3 4 / **2** **2**	**3** 5 / **3** **3**	5 6 / **4** **4**	6 7 / **5** **5**
a	**2** 2	2 **1** / 3 **1**	**0** 2 / 2 **0**	2 3 / **1** **1**	3 4 / **2** **2**	**3** 5 / **3** **3**	5 6 / **4** **4**
t	**3** 3	3 **2** / 4 **2**	2 **1** / 3 **1**	**0** 2 / 2 **0**	2 3 / **1** **1**	3 4 / **2** **2**	**3** 5 / **3** **3**

cost	operation	input	output
0	(copy)	c	c
1	insert	*	a
1	insert	*	t
1	insert	*	c
0	(copy)	a	a
0	(copy)	t	t

연습: CAT → CATCAT

- cat(행) × catcat(열)의 레벤슈타인 행렬을 채운다
- 최종 거리 = **3**

✍ 메모

— 넘어가도 되는 쪽

		c	a	t	c	a	t
	0	**1** 1	**2** 2	**3** 3	**4** 4	**5** 5	**6** 6
c	**1** 1	**0** 2 / 2 **0**	2 3 / **1** 1	3 4 / **2** 2	**3** 5 / **3** 3	5 6 / **4** 4	6 7 / **5** 5
a	**2** 2	2 **1** / 3 1	**0** 2 / 2 **0**	2 3 / **1** 1	3 4 / **2** 2	**3** 5 / **3** 3	5 6 / **4** 4
t	**3** 3	3 **2** / 4 2	2 **1** / 3 1	**0** 2 / 2 **0**	2 3 / **1** 1	3 4 / **2** 2	**3** 5 / **3** 3

cost	operation	input	output
0	(copy)	c	c
0	(copy)	a	a
1	insert	*	t
1	insert	*	c
1	insert	*	a
0	(copy)	t	t

연습: CAT → CATCAT

- cat(행) × catcat(열)의 레벤슈타인 행렬을 채운다
- 최종 거리 = **3**

✍ 메모

— 넘어가도 되는 쪽

		c	a	t	c	a	t
	0	**1** 1	**2** 2	**3** 3	**4** 4	**5** 5	**6** 6
c	**1** 1	**0** 2 / 2 **0**	2 **1**	3 2	**3** 3	5 4	6 5
a	**2** 2	2 **1** / 3 1	**0** 2 / 2 **0**	2 1	3 2	**3** 3	5 4
t	**3** 3	3 **2** / 4 2	2 **1** / 3 1	**0** 2 / 2 **0**	2 **1** / 3 1	3 **2** / 4 2	**3** 3 / 3 3

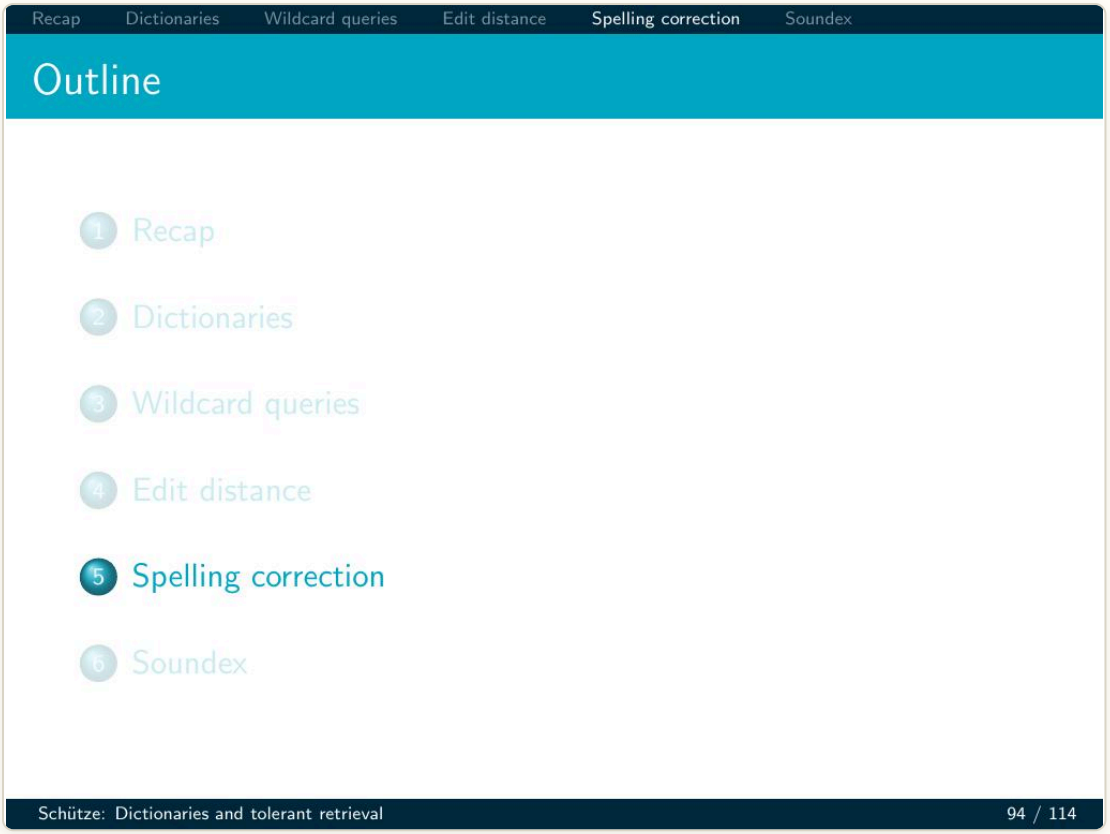
cost	operation	input	output
0	(copy)	c	c
0	(copy)	a	a
0	(copy)	t	t
1	insert	*	c
1	insert	*	a
1	insert	*	t

CAT → CATCAT: 편집 연산 (결과)

- 비용 0 (복사) c → c
- 비용 0 (복사) a → a
- 비용 0 (복사) t → t
- 비용 1 **삽입** * → c
- 비용 1 **삽입** * → a
- 비용 1 **삽입** * → t
- 합계 **3**

✍ 메모

⚠ 주의 같은 거리 3을 주는 다른 경로(앞에 cat 삽입)도 있다 — 경로는 하나가 아닐 수 있음

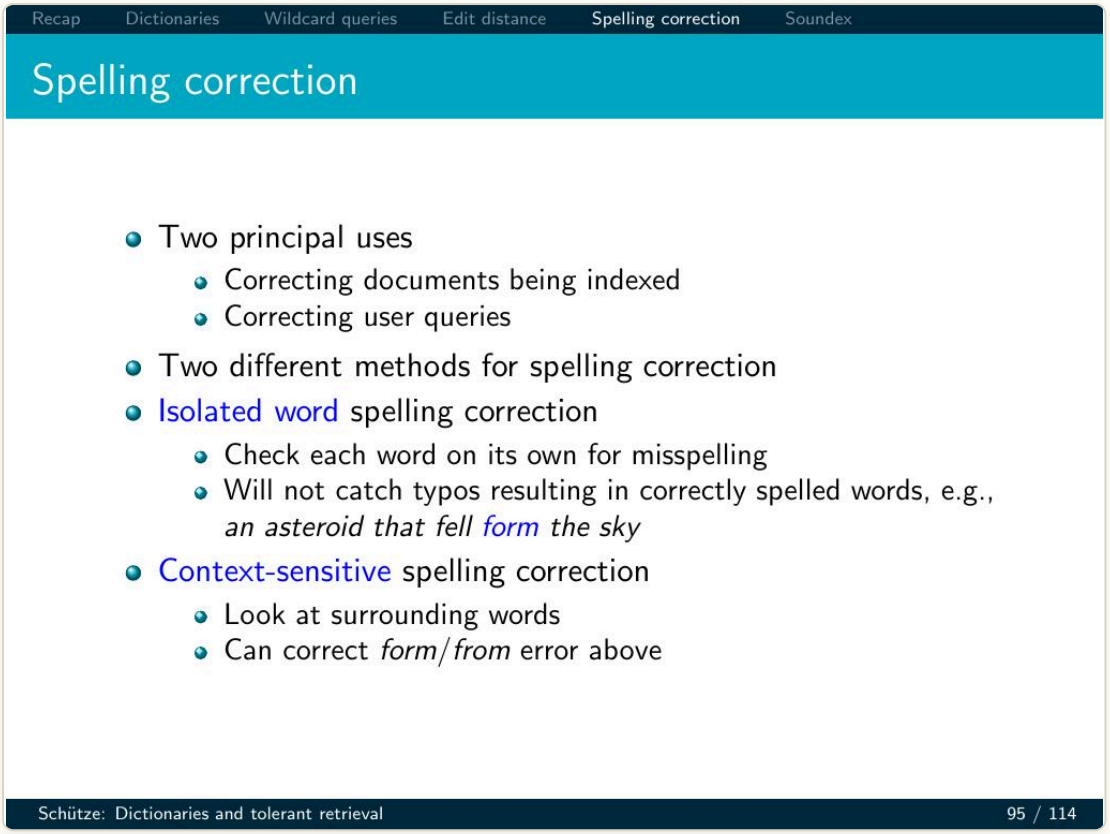


목차

- 복습 → 사전 → 와일드카드 질의 → 편집 거리 → 철자 교정 → Soundex
- 지금: **철자 교정**

✍ 메모

— 넘어가도 되는 쪽

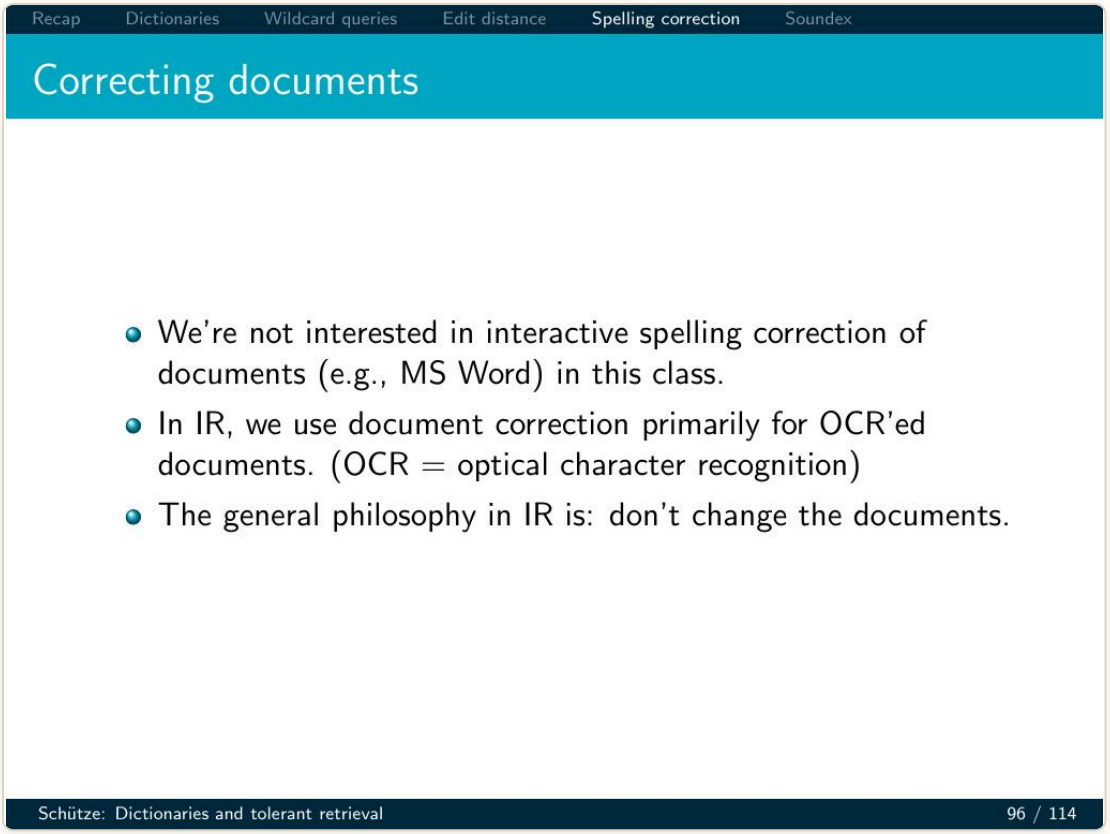


철자 교정 (Spelling correction)

- 주된 용도 두 가지
  - **색인할 문서** 교정
  - **사용자 질의** 교정
- 철자 교정의 두 방법
  - **고립 단어 교정** — 단어 하나하나를 따로 검사
    - 철자는 맞는 다른 단어가 된 오타는 못 잡는다 — an asteroid that fell **form** the sky
  - **문맥 의존 교정** — 주변 단어를 본다
    - 위의 form/from 오류를 고칠 수 있다

✍ 메모

★ 시험 고립 단어 vs 문맥 의존 — form/from 예

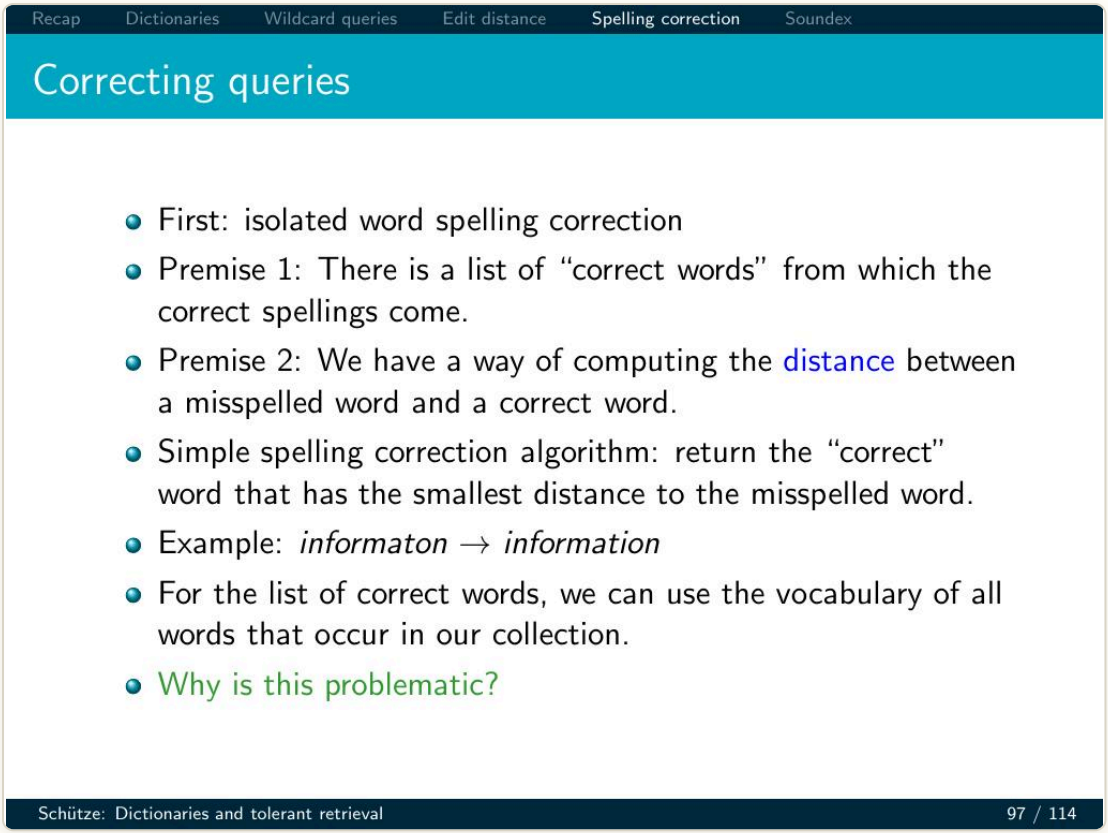


문서 교정

- 이 수업은 문서의 대화형 철자 교정(예: MS Word)엔 관심 없다
- IR에서 문서 교정은 주로 **OCR 문서**에 쓴다 (OCR = 광학 문자 인식)
- IR의 일반 철학: **문서는 바꾸지 마라**

✍ 메모

— 넘어가도 되는 쪽

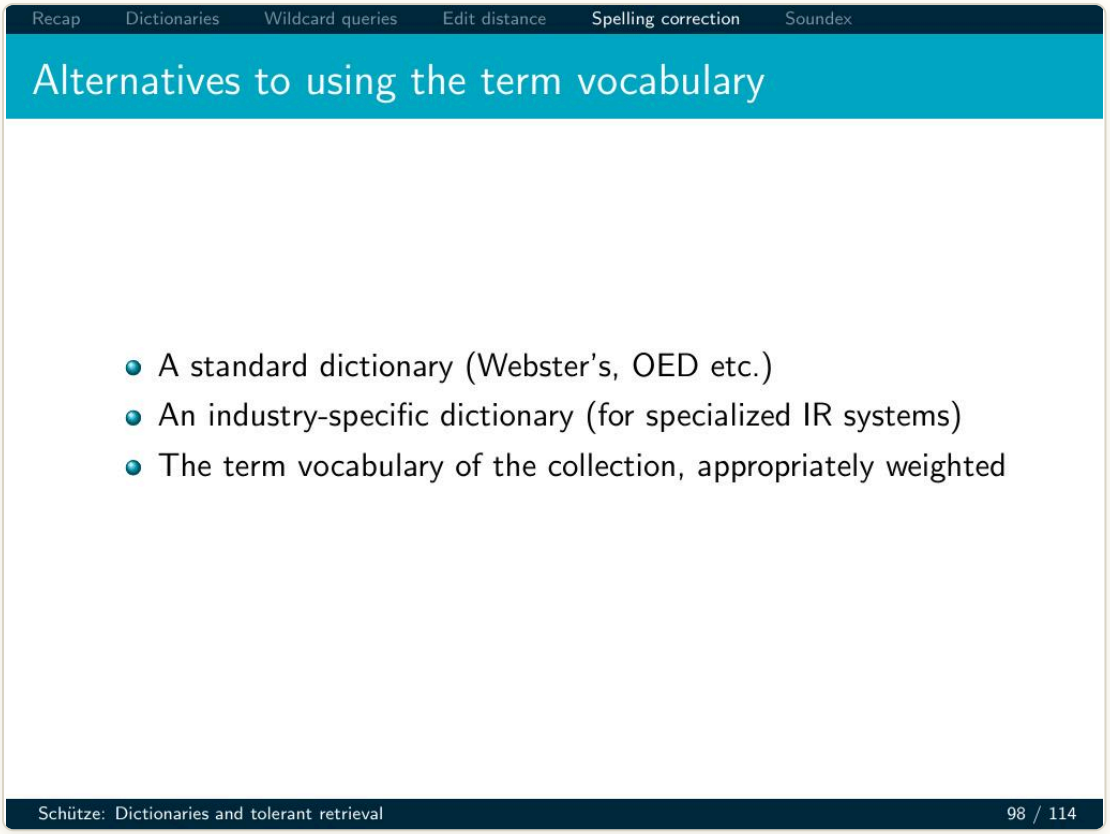


질의 교정

- 먼저: 고립 단어 철자 교정
- 전제 1: 올바른 철자가 나오는 **'올바른 단어' 목록**이 있다
- 전제 2: 틀린 단어와 올바른 단어 사이의 **거리를 계산**할 방법이 있다
- 간단한 교정 알고리즘: 틀린 단어와 **거리가 가장 작은** '올바른' 단어를 반환
  - 예: informaton → information
- 올바른 단어 목록으로 컬렉션에 나오는 **모든 단어의 어휘**를 쓸 수 있다
  - 그런데 이게 왜 문제일까?

✍️ 메모

💡 이해 컬렉션 어휘엔 오타도 들어 있다 → 그냥 쓰면 틀린 단어를 '올바르다'고 제안할 수 있음

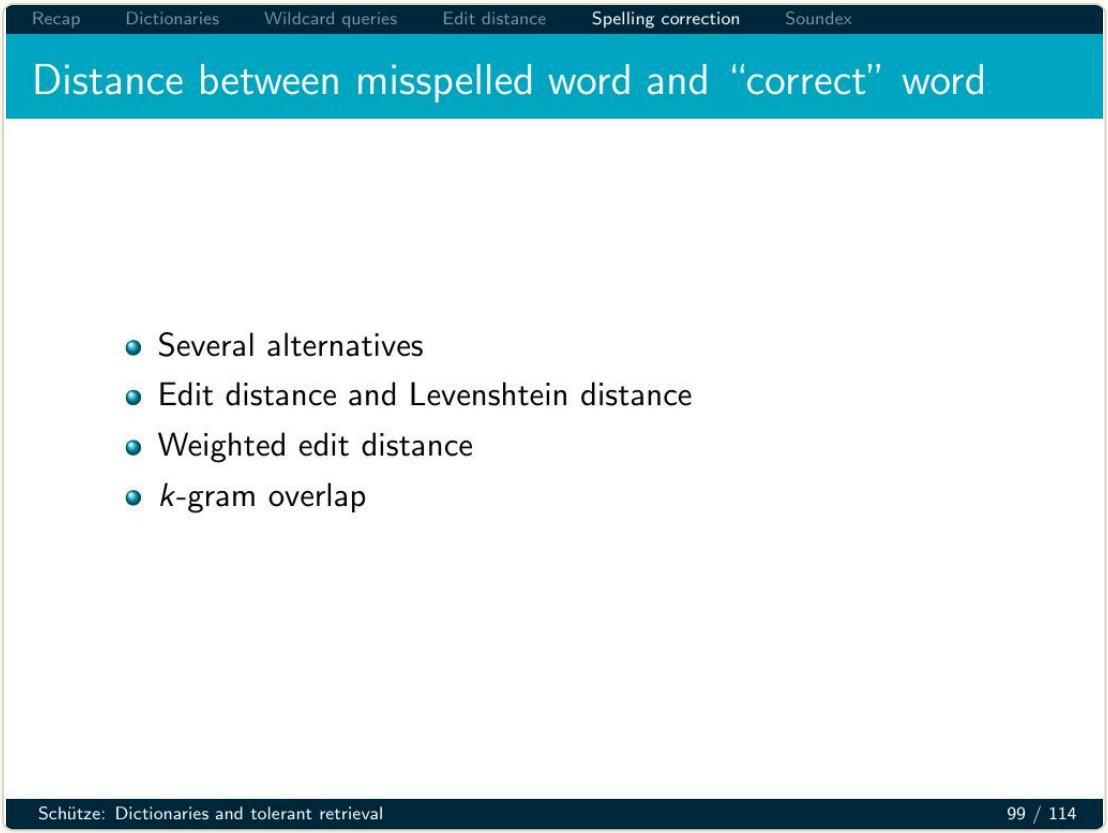


단어 어휘 대신 쓸 수 있는 것

- 표준 사전 (Webster's, OED 등)
- 업계 전용 사전 (특화된 IR 시스템용)
- **적절히 가중한** 컬렉션 단어 어휘

✍ 메모

— 넘어가도 되는 쪽

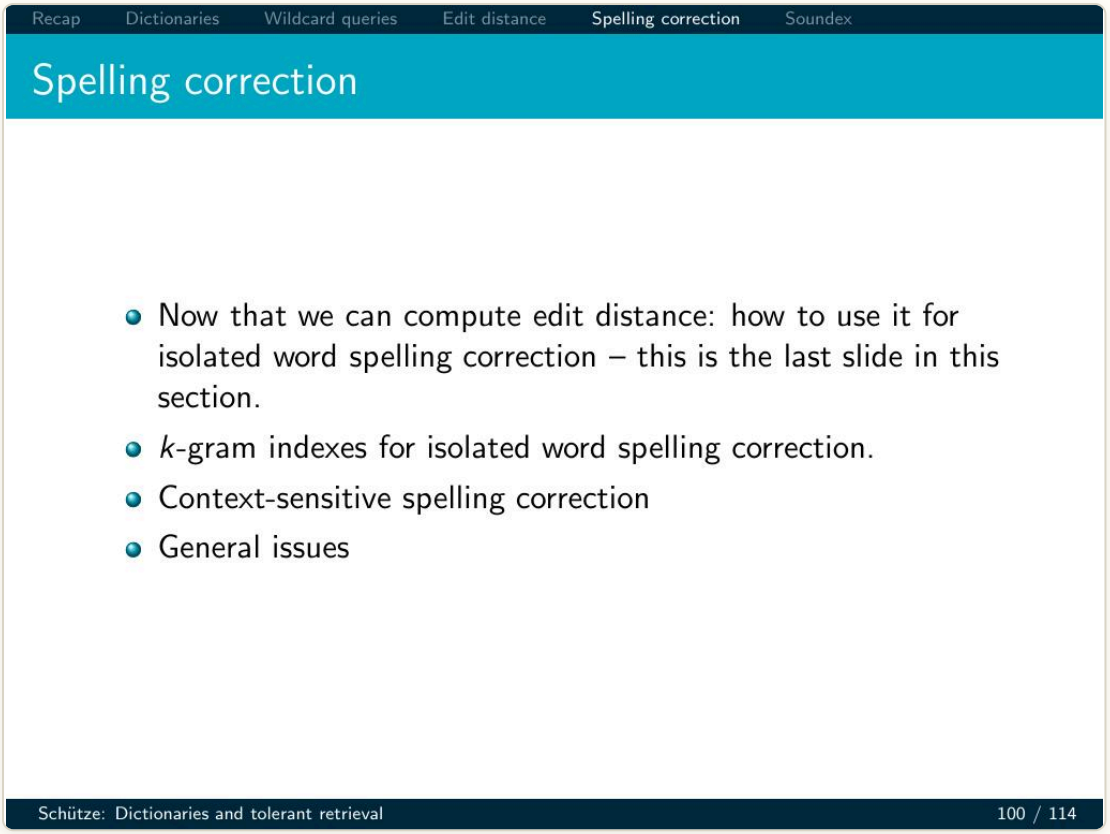


틀린 단어와 '올바른' 단어 사이의 거리

- 여러 대안
  - 편집 거리와 레벤슈타인 거리
  - 가중 편집 거리
  - **k-gram 겹침**

✍ 메모

— 넘어가도 되는 쪽

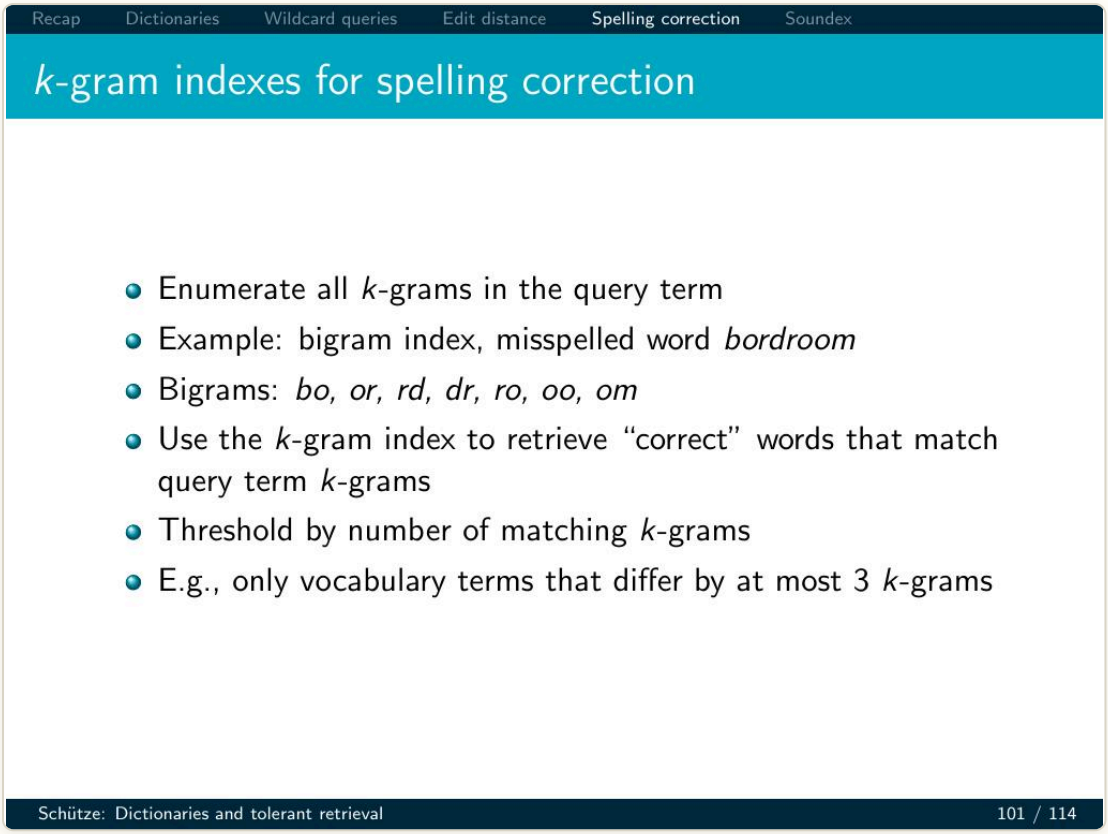


철자 교정 — 남은 내용

- 편집 거리를 고립 단어 교정에 쓰는 법은 앞에서 봤다
- **k-gram 색인**으로 고립 단어 교정
- **문맥 의존** 철자 교정
- 일반적 쟁점

✍ 메모

— 넘어가도 되는 쪽

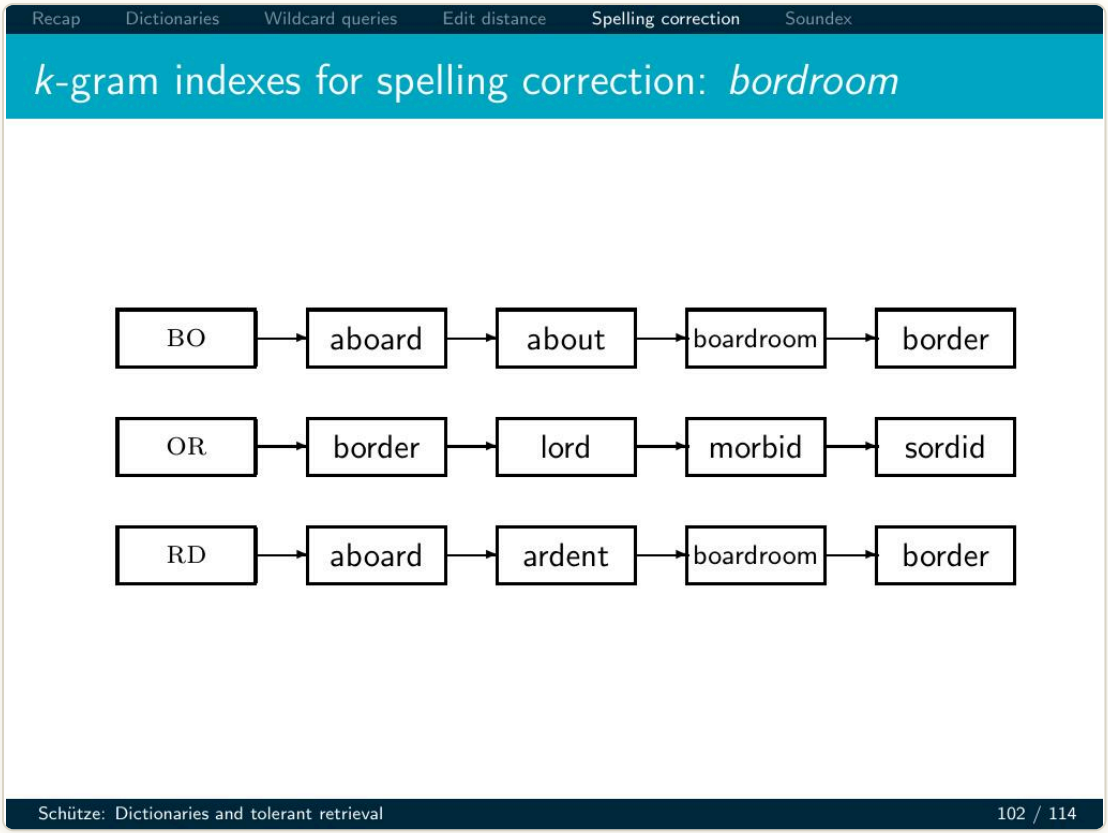


철자 교정용 k-gram 색인

- 질의어의 모든 k-gram을 나열
- 예: 바이그램 색인, 틀린 단어 **bordroom**
  - 바이그램: bo, or, rd, dr, ro, oo, om
- k-gram 색인으로 질의어 k-gram과 맞는 '올바른' 단어를 가져온다
- 맞는 k-gram **개수로 임계값**
  - 예: k-gram이 최대 3개까지 다른 어휘 단어만

✍ 메모

— 넘어가도 되는 쪽

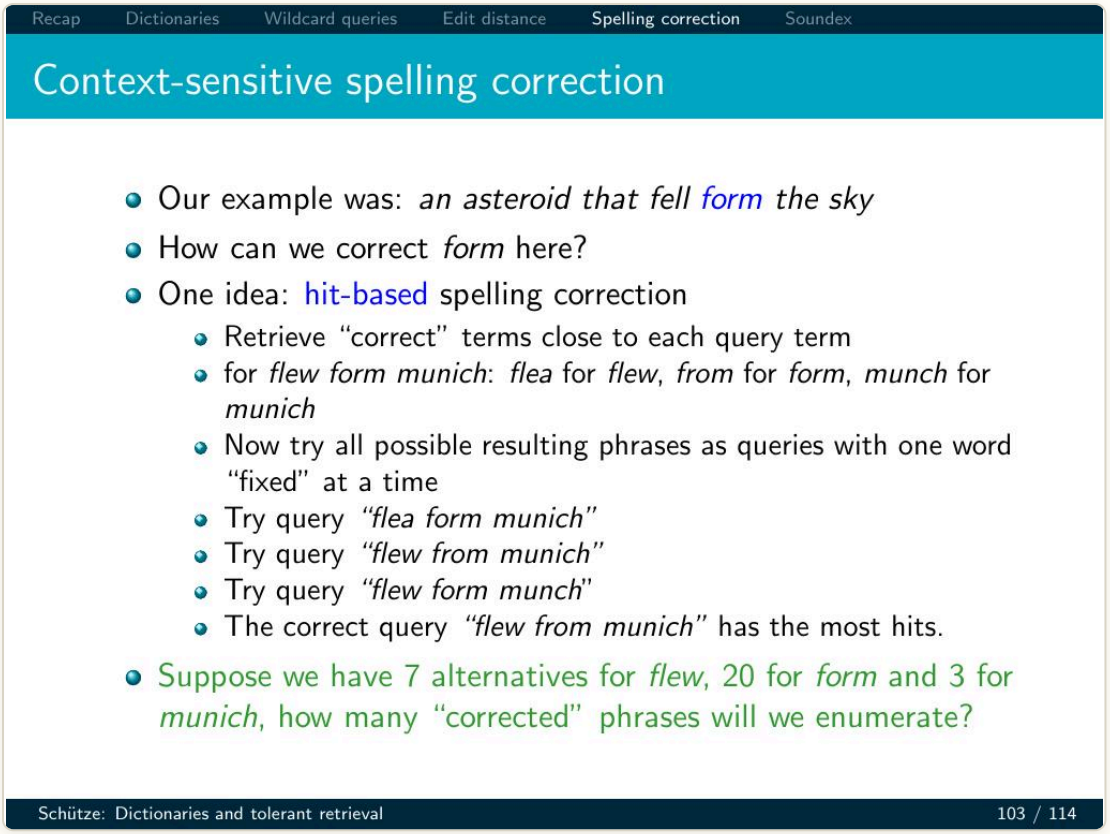


철자 교정용 k-gram 색인: bordroom

- BO → aboard, about, boardroom, border
- OR → border, lord, morbid, sordid
- RD → aboard, ardent, boardroom, border
- → 여러 바이그램에 공통으로 나오는 **boardroom, border**가 후보

✍️ 메모

💡 이해 실제로는 자카드 계수 같은 겹침 비율로 순위를 매긴다

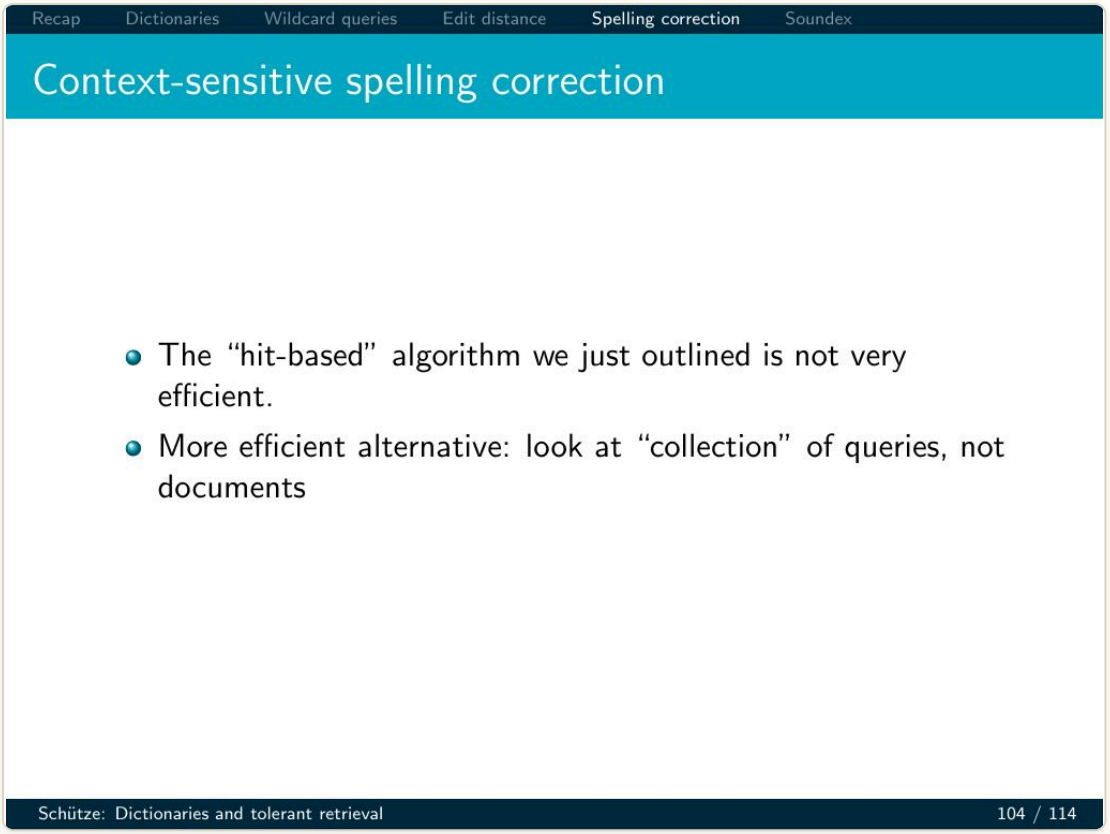


문맥 의존 철자 교정

- 예: an asteroid that fell **form** the sky — 여기서 form을 어떻게 고칠까?
- 아이디어: **결과 수 기반(hit-based)** 교정
  - 각 질의어에 가까운 '올바른' 단어를 가져온다
  - flew form munich → flew 대신 flea, form 대신 from, munich 대신 munch
  - 한 번에 **한 단어만 고친** 모든 구를 질의로 시도
    - "flea form munich" / "flew from munich" / "flew form munch"
  - 올바른 질의 "flew from munich"가 **결과가 가장 많다**
- flew 대안 7개, form 20개, munich 3개면 '교정된' 구를 몇 개 나열하게 될까?

✍ 메모

✏ 예·풀이 한 번에 한 단어만 고치면 7 + 20 + 3 = 30개 (모든 조합이면 7 × 20 × 3 = 420)

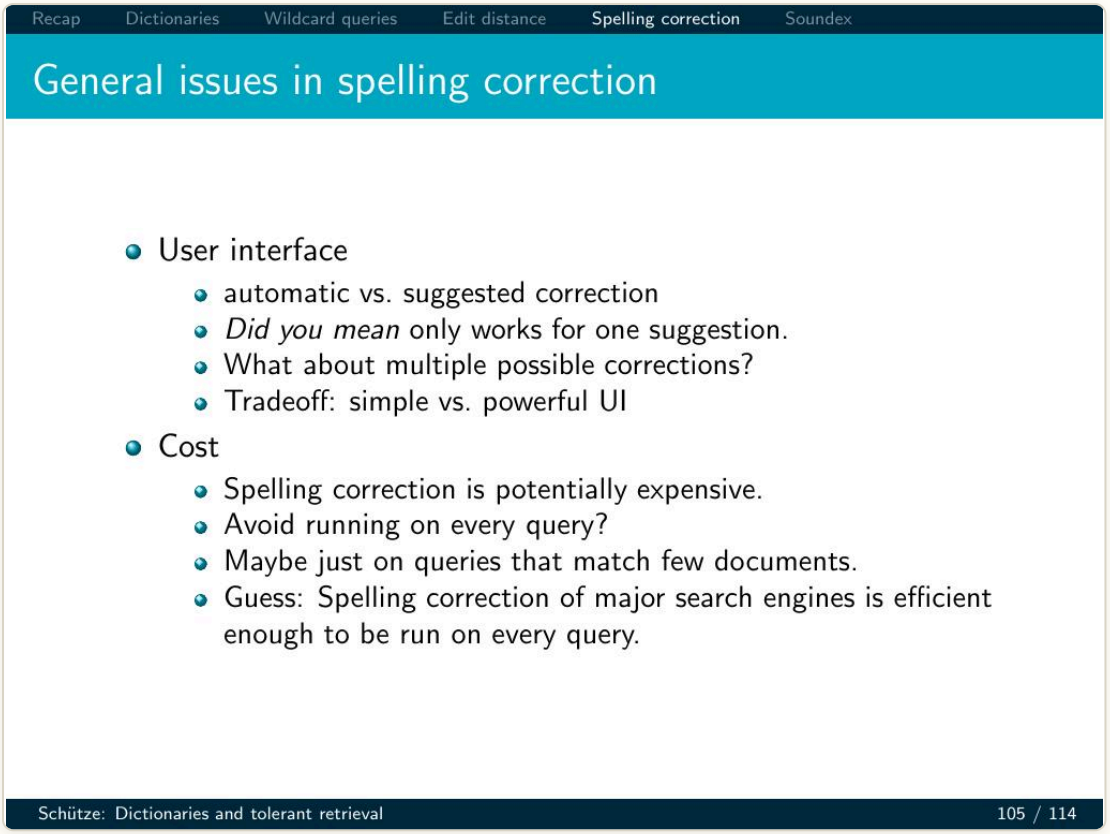


문맥 의존 교정 — 효율

- 방금 본 hit-based 알고리즘은 **그리 효율적이지 않다**
- 더 효율적인 대안: 문서가 아니라 **질의 모음(query log)**을 본다

✍ 메모

— 넘어가도 되는 쪽

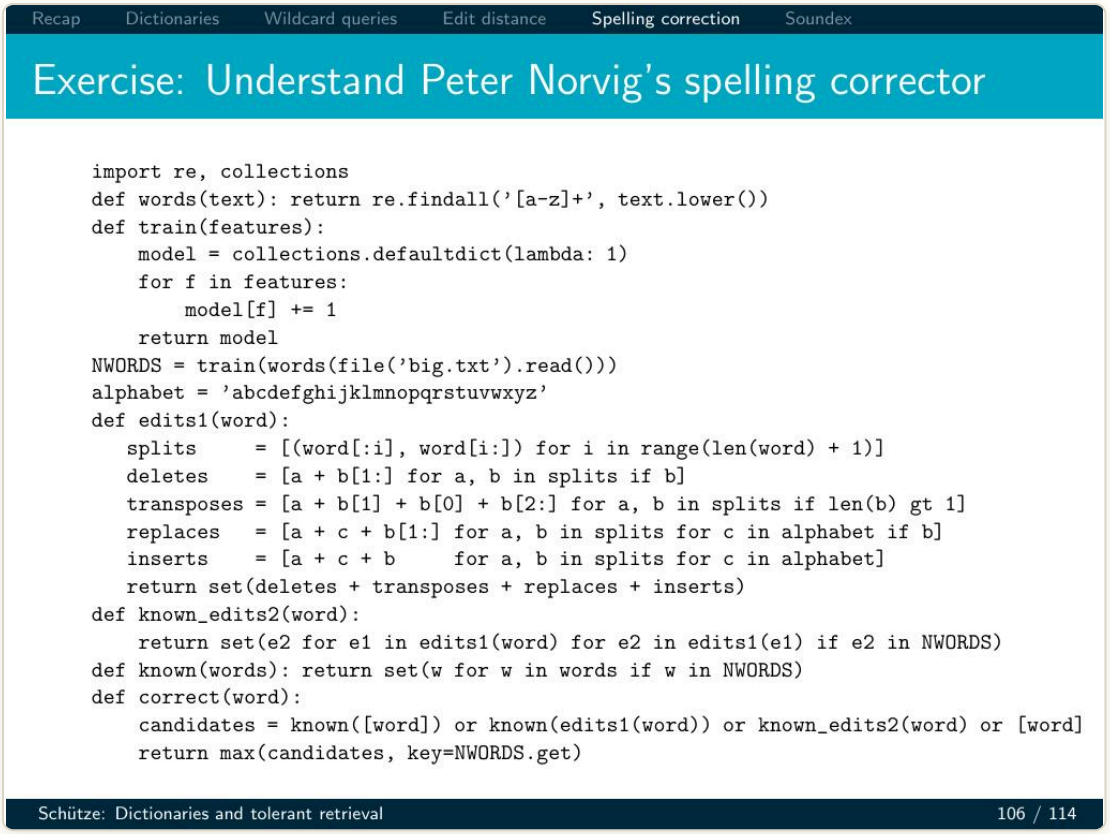


철자 교정의 일반 쟁점

- **사용자 인터페이스**
  - **자동** 교정 vs **제안**
  - 'Did you mean'은 제안 **하나**일 때만 통한다
  - 여러 교정 후보가 있으면?
  - 단순한 UI vs 강력한 UI의 절충
- **비용**
  - 철자 교정은 잠재적으로 비싸다
  - 모든 질의에 돌리지 말까?
  - 맞는 문서가 **적은 질의**에만?
  - 추측: 주요 검색 엔진의 철자 교정은 모든 질의에 돌릴 만큼 효율적이다

✍️ 메모

— 넘어가도 되는 쪽



연습: Peter Norvig의 철자 교정기 이해하기

- 파이썬 20줄 남짓

`words()`: 소문자 단어 추출 / `train()`: 단어 빈도 모델 `NWORDS`

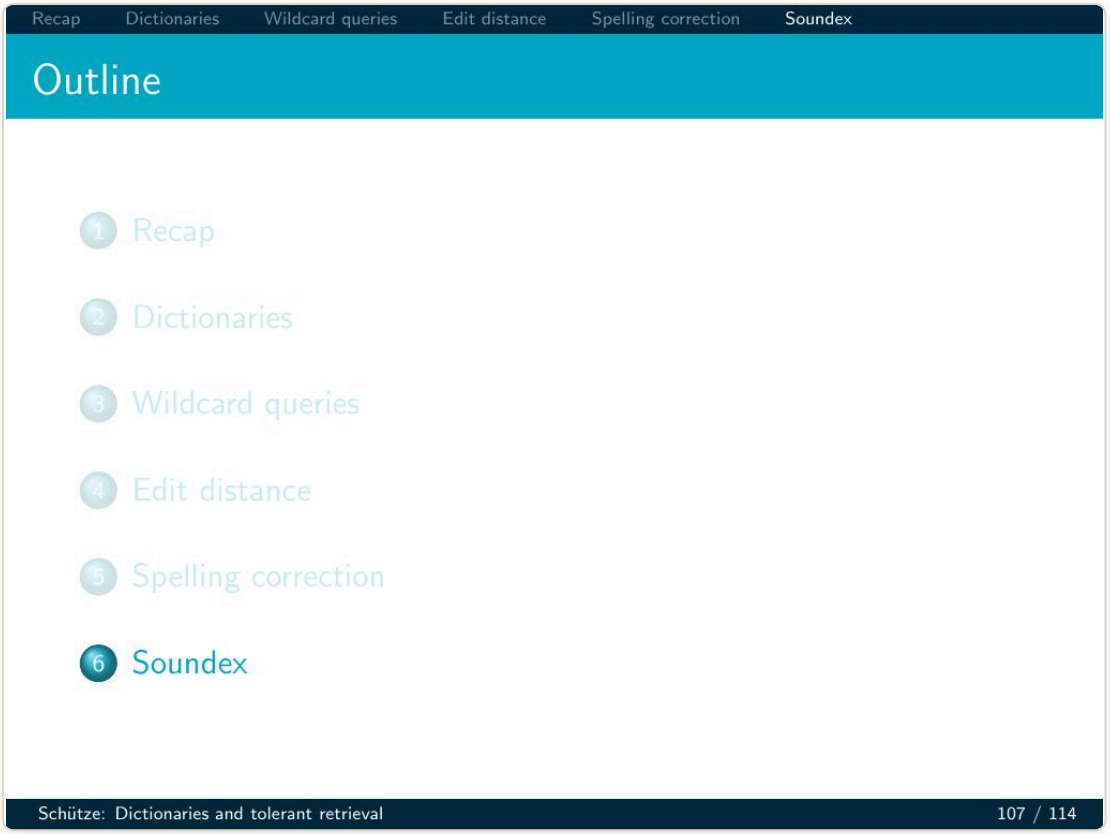
`edits1(word)`: 편집 거리 1인 모든 문자열 (삭제·전치·교체·삽입)

`known_edits2(word)`: 편집 거리 2 중 사전에 있는 것

`correct(word)`: 원래 단어 → 거리 1 → 거리 2 순으로 사전에 있는 후보 중 **빈도가 가장 높은** 것

✍️ 메모

💡 이해 편집 거리 후보 + 빈도(언어 모델)로 고르는 구조

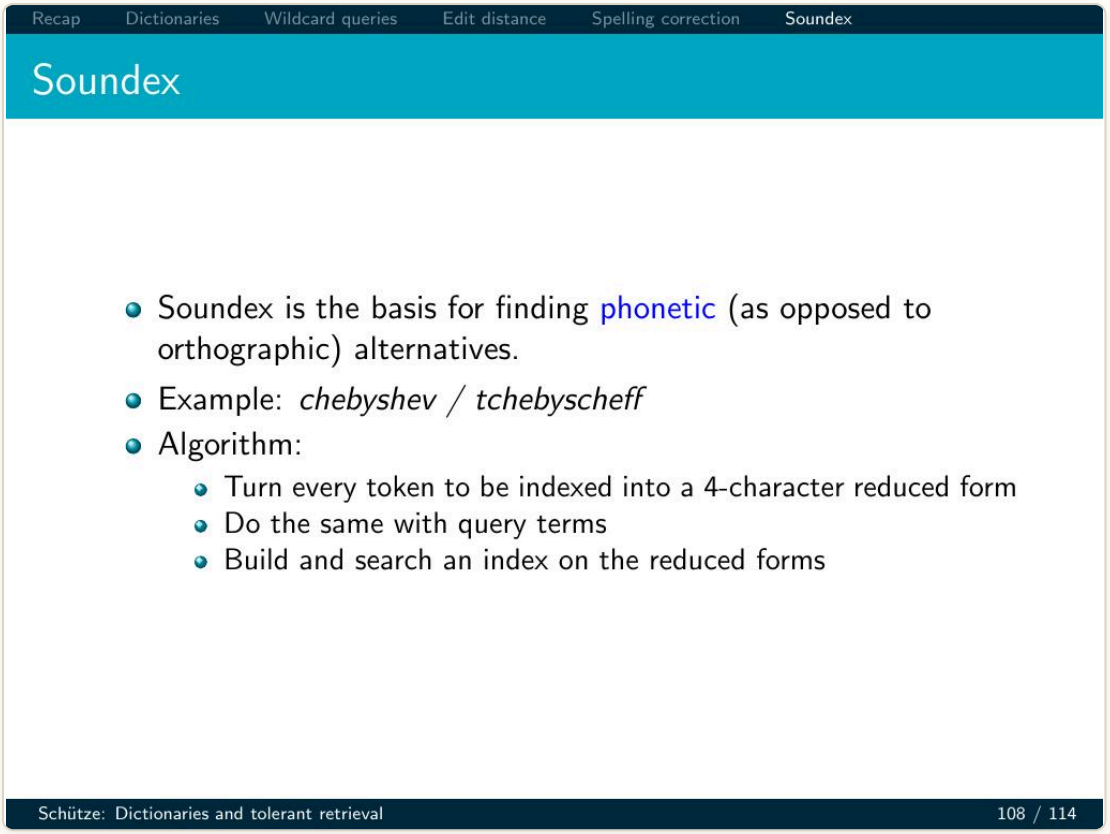


목차

- 복습 → 사전 → 와일드카드 질의 → 편집 거리 → 철자 교정 → Soundex
- 지금: **Soundex**

✍ 메모

— 넘어가도 되는 쪽

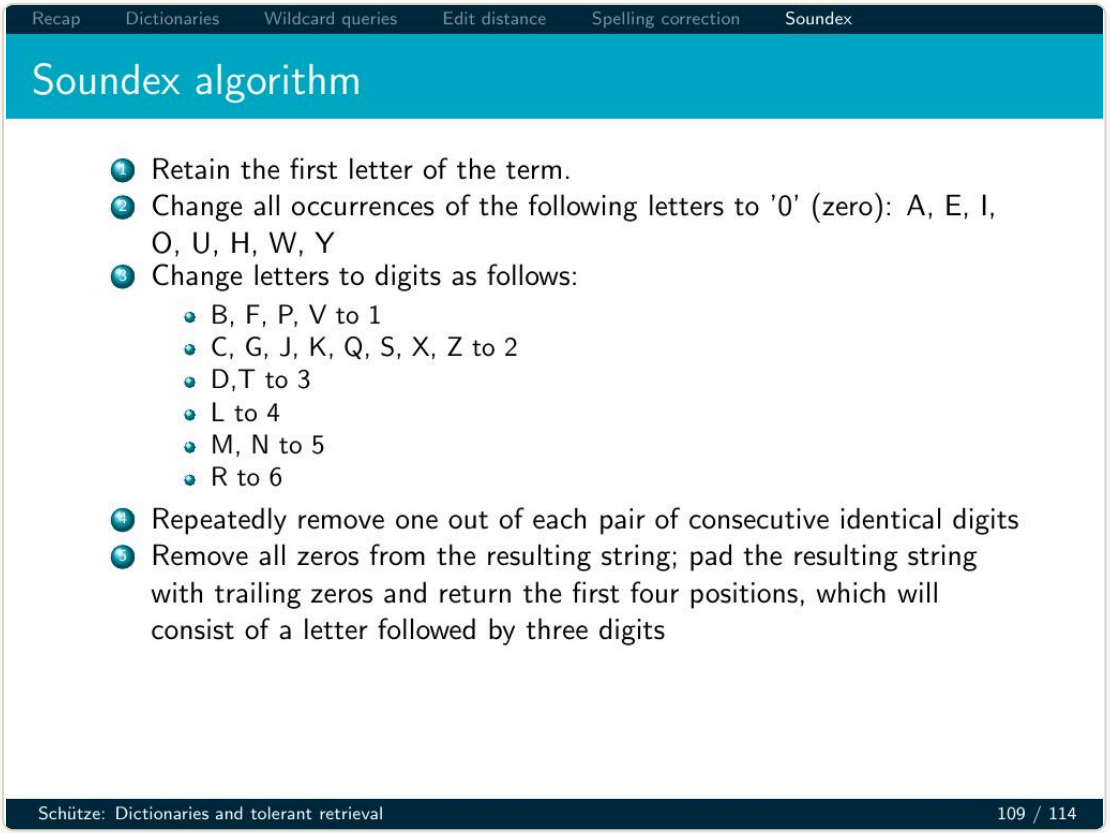


Soundex

- Soundex는 철자(orthographic)가 아니라 **발음(phonetic)** 상의 대안을 찾는 기반
- 예: chebyshev / tchebyscheff
- 알고리즘
  - 색인할 모든 토큰을 **4글자 축약형**으로 바꾼다
  - 질의어도 똑같이
  - 축약형으로 색인을 만들고 검색

✍ 메모

— 넘어가도 되는 쪽

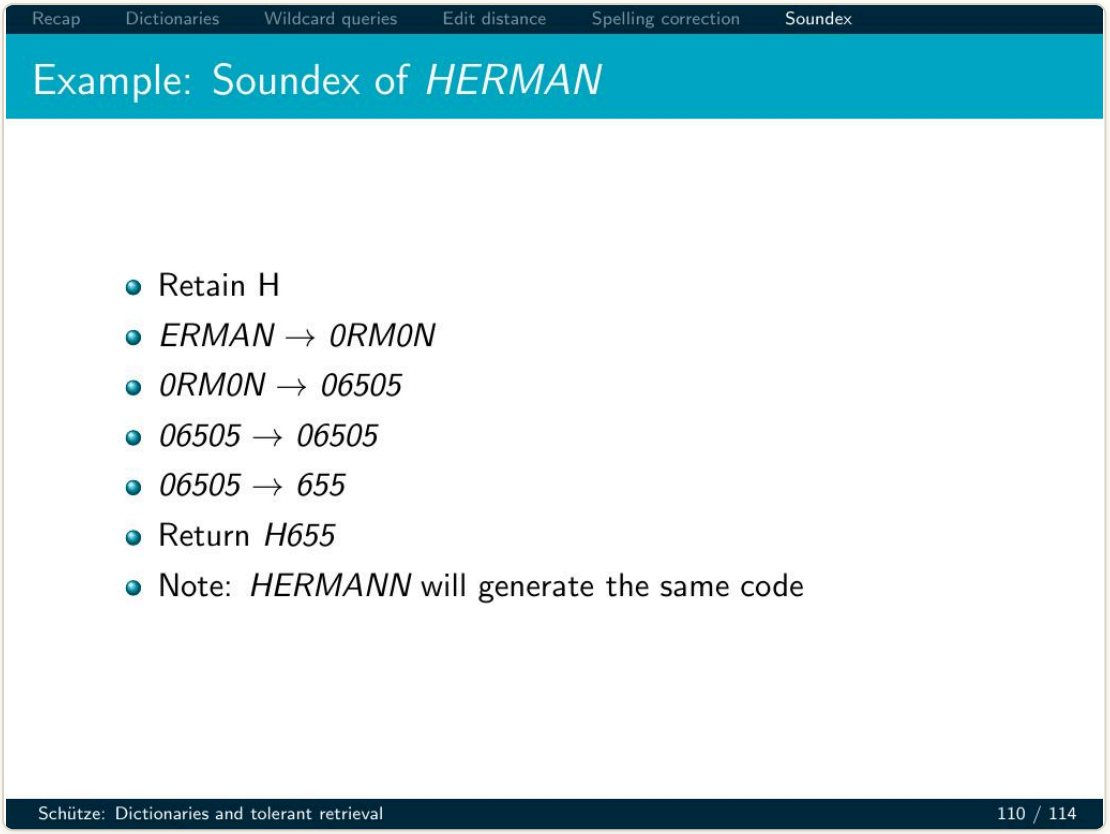


Soundex 알고리즘

- ① 단어의 **첫 글자는 그대로**
- ② A, E, I, O, U, H, W, Y는 모두 **'0'**으로
- ③ 나머지 글자를 숫자로
  - B, F, P, V → **1**
  - C, G, J, K, Q, S, X, Z → **2**
  - D, T → **3**
  - L → **4**
  - M, N → **5**
  - R → **6**
- ④ 같은 숫자가 **연속**하면 하나만 남김
- ⑤ 0을 모두 지우고, 뒤에 0을 채워 **앞 4자리**(글자 1 + 숫자 3) 반환

✍️ 메모

★ 시험 숫자표 1~6 외우기 — 계산 문제 확실

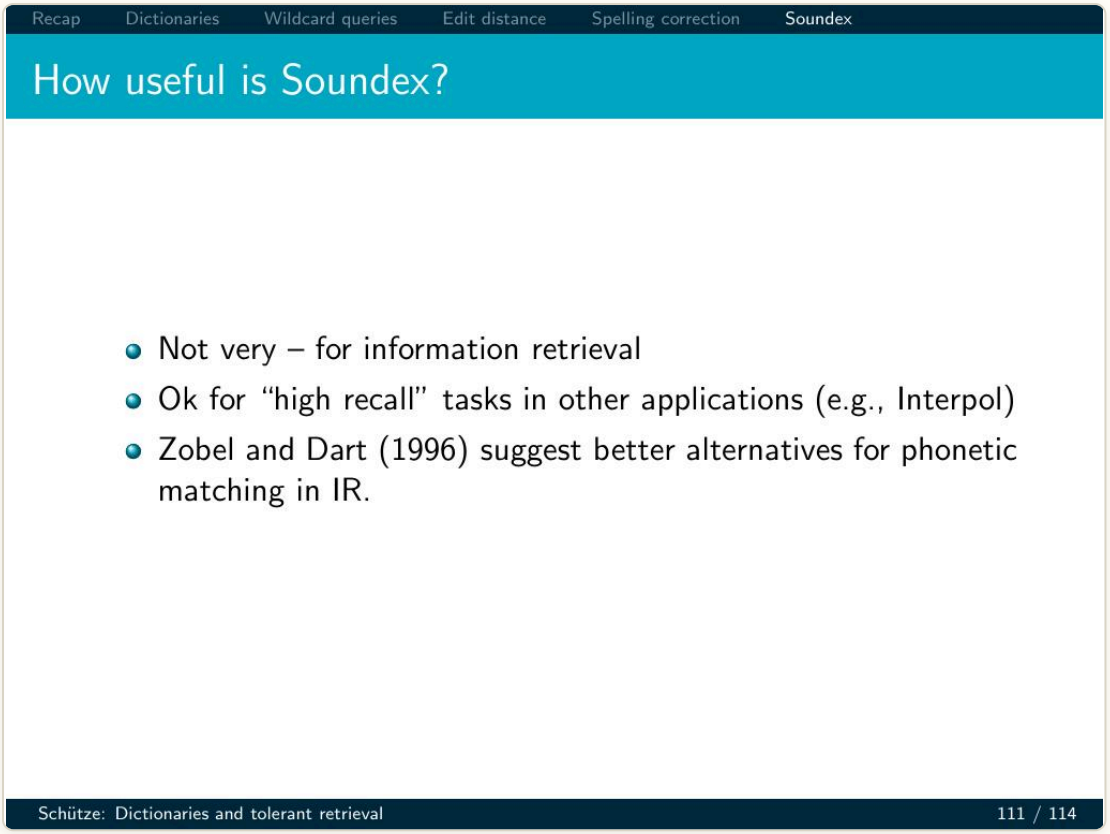


예: HERMAN의 Soundex

- H는 유지
- ERMAN → 0RM0N
- 0RM0N → 06505
- 06505 → 06505 (연속 같은 숫자 없음)
- 06505 → 655 (0 제거)
- → **H655**
- 참고: HERMANN도 같은 코드

✍ 메모

✏ 예·풀이 HERMANN: ...0RM0NN → 065055 → 06505 → 655 → H655

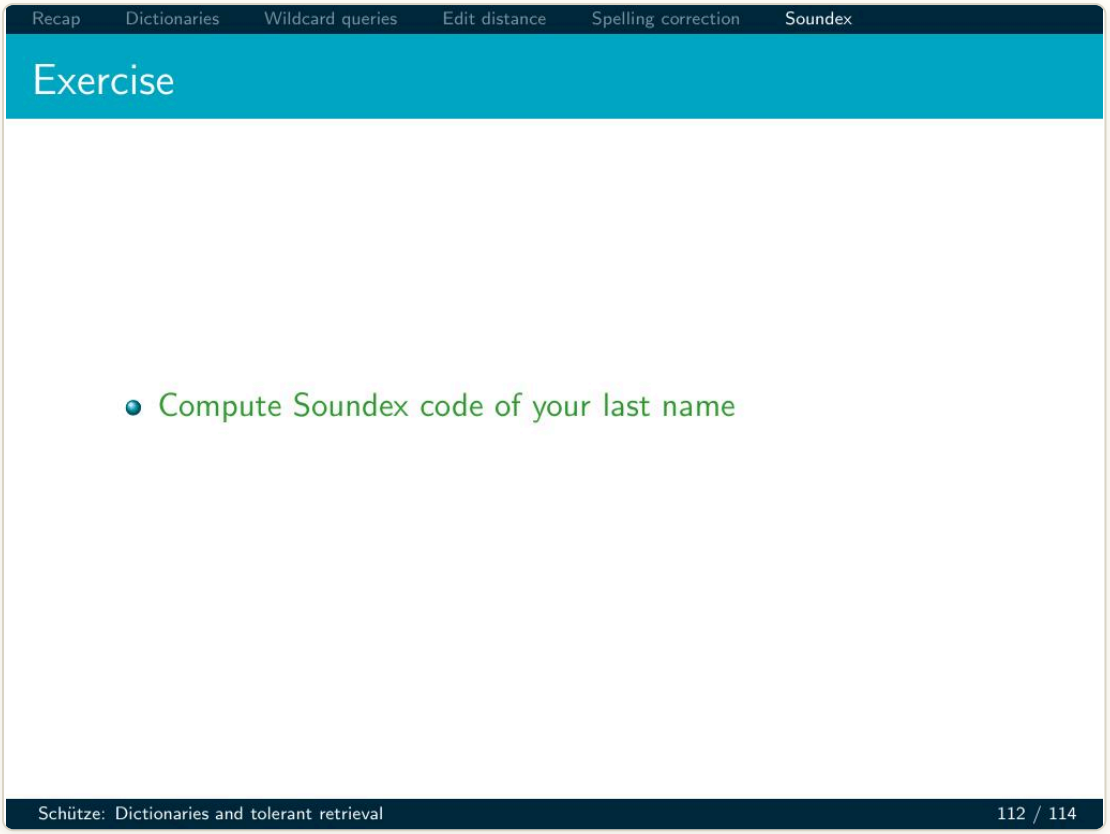


Soundex는 얼마나 유용한가?

- 정보검색에는 **그다지** 유용하지 않다
- 다른 응용의 **고재현율** 작업에는 괜찮다 (예: 인터폴)
- Zobel & Dart(1996)가 IR용 더 나은 발음 매칭 방법을 제안

✍️ 메모

— 넘어가도 되는 쪽

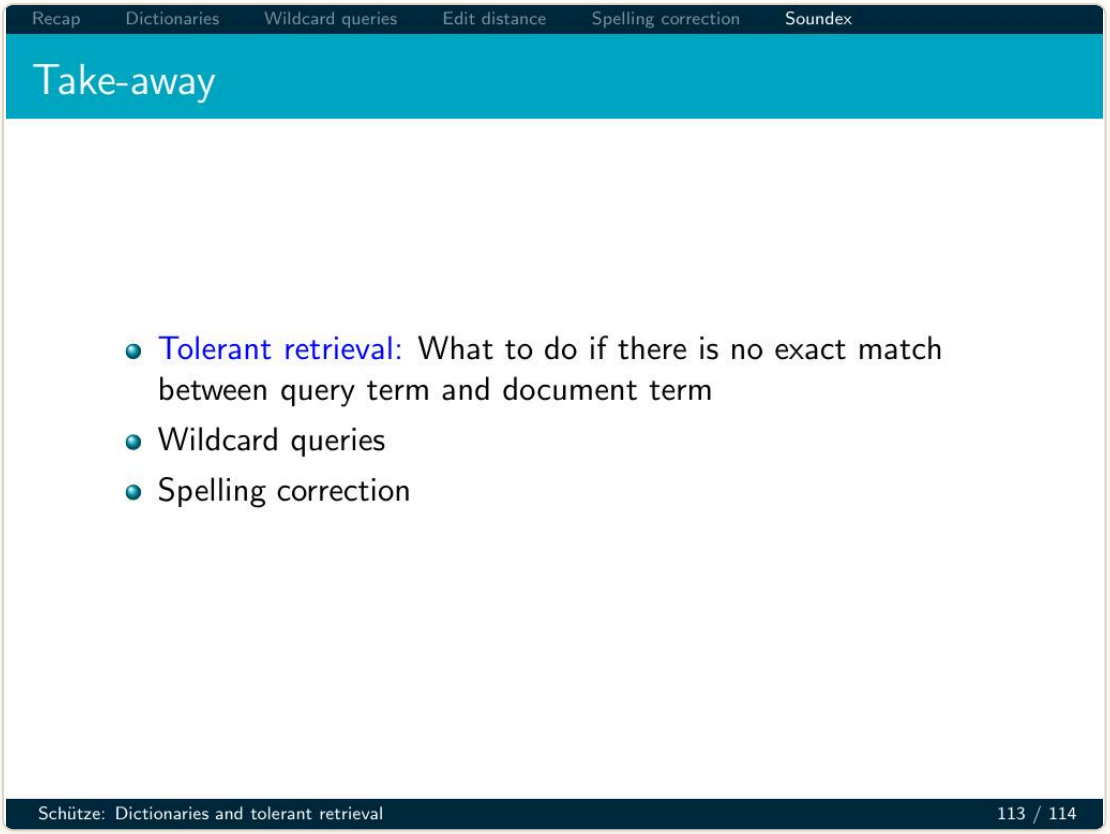


연습

- 자기 성(last name)의 Soundex 코드를 계산해 보라

✍ 메모

✏ 예·풀이 예: LEE → L, 0, 0 → 0 제거 → L000 (뒤에 0 채움)

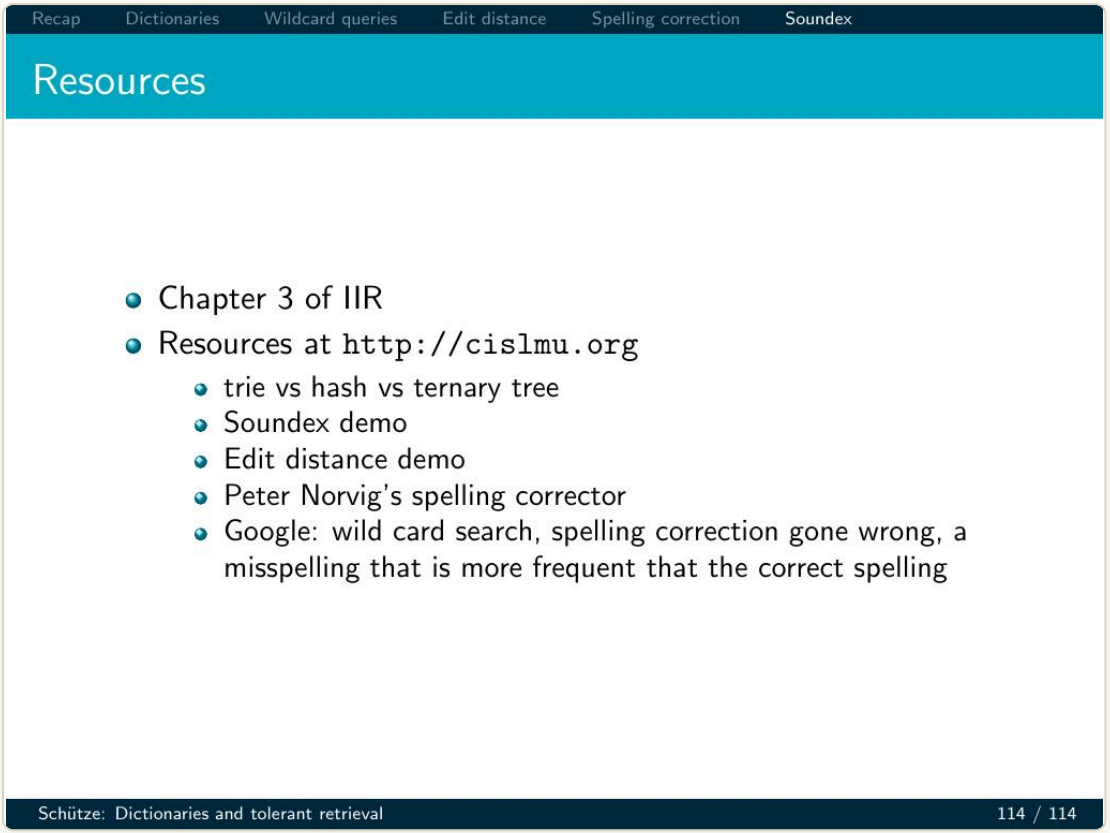


이번 강의에서 가져갈 것

- 오류 허용 검색: 질의어와 문서 단어가 정확히 일치하지 않을 때
  - 와일드카드 질의 (permuterm, k-gram)
  - 철자 교정 (편집 거리, k-gram, 문맥 의존, Soundex)

✍ 메모

— 넘어가도 되는 쪽



참고 자료

- IIR 3장, http://cislmu.org
- trie vs 해시 vs ternary tree, Soundex 데모, 편집 거리 데모, Peter Norvig 철자 교정기
- 구글: 와일드카드 검색, 잘못된 철자 교정, 올바른 철자보다 더 흔한 오타

✍ 메모

— 넘어가도 되는 쪽